

Multidimensional Time-Varying Signal Processing on Graphs

Dissertation submitted to

Tsinghua University

in partial fulfillment of the requirement

for the degree of

Doctor of Philosophy

in

Information and Communication Engineering

by

Yan, Yi

Dissertation Supervisor: Professor Ercan Engin Kuruoglu

May, 2024

ABSTRACT

Big data encountered in the real world are frequently in the form of dynamic and irregularly structured multidimensional signals. In this dissertation, our research goal is to extend the frontiers of knowledge in the field of Signal Processing by overcoming four challenges that hinder the effectiveness of applying classical Signal Processing to multivariate time-varying signals on topological structures: the representation of topological irregularity, the estimation of unknown data based on known data, the removal of noise present in the data, and the extraction of the cross-space-time variation. Our research approach is to develop algorithms that utilize Graph Signal Processing (GSP) and Topological Signal Processing (TSP) techniques to represent multidimensional signals that reside on the graph or network structures. Firstly, an analysis is conducted on GSP adaptive filters; they are able to accurately form online tracking of time-varying signals on the graph nodes, under the condition of partial signal observations being corrupted by Gaussian or impulsive non-Gaussian noises. Efficient and effective denoising of time-varying signals on graphs is accomplished by our proposed algorithms. Secondly, in addition to using adaptive filters to capture the time evolution of the graph signals, a TSP-based Joint Online Estimation (JOE) framework is designed to extract and represent the cross-space-time variation on simplicial signals that are represented by a time-varying regression based on simplicial diffusion. This TSP formulation allows us to push the GSP methods, which focus on the graph nodes only, to higher-ordered structures on graphs. Adaptive estimation of time-varying signals that reside on both the graph nodes and the graph edges is accomplished by specifying JOE to the nodes and edges of a graph. Thirdly, a discussion on the cases where the topologies dynamically change over time is explored by the proposal of a Dynamic Multi-Hop model. The Dynamic Multi-Hop model could infer a series of dynamic graph topologies from a fixed graph containing time-varying node and edge signals using TSP and Statistical Graphical Model. Real-world applications of the research proposed in this dissertation include but are not limited to meteorological data prediction, transportation network modeling, and pandemic spread simulation.

Keywords: Graph Signal Processing; Topological Signal Processing; Time-Varying Signals; Online Estimation; Dynamic Graphs

TABLE OF CONTENTS

ABSTRACT	I
TABLE OF CONTENTS	II
LIST OF FIGURES.....	V
LIST OF TABLES	VIII
LIST OF SYMBOLS AND ACRONYMS	IX
CHAPTER 1 RESEARCH BACKGROUND, CHALLENGES, AND EXPECTED OUT- COME	1
1.1 Research Background.....	1
1.2 Research Challenges	2
1.3 Research Objectives.....	4
1.4 Dissertation Outline	4
CHAPTER 2 LITERATURE REVIEW OF RELATED WORK	6
2.1 Foundations of Signal Processing on Graphs and Networks	6
2.2 Time-Varying Algorithms on Graphs with Gaussian Noise Assumption	7
2.3 Time-Varying Algorithms on Graphs with non-Gaussian Noise Assumption.....	10
2.4 Topological Signal Processing on High-order Structures	11
2.5 Neural Network Algorithms on Graphs and Higher-Ordered Structures	13
2.6 Topology Acquisition Techniques.....	15
2.7 Applications	17
CHAPTER 3 PRELIMINARY KNOWLEDGE AND TIME-INVARIANT SIGNAL PROCESSING ON TOPOLOGICAL STRUCTURES	19
3.1 Processing of Signals on the Graph Nodes.....	19
3.1.1 Graph Signals With a Single Feature	19
3.1.2 Graph Signals With Multiple Features	23
3.2 Processing of the Signals on Simplicial Complexes	25
3.3 Graph and Simplicial Neural Networks	31
3.4 Discussion	32

TABLE OF CONTENTS

CHAPTER 4 TIME-INVARIANT TOPOLOGICAL SIGNAL PROCESSING CASE STUDY	33
4.1 Bi-SCNN Derivation.....	33
4.2 Experiment Setting and Dataset Information	36
4.3 Estimation of Missing Simplicial Signals in Citation Complex.....	37
4.4 Classification of Ocean Drifter on Simplicial Complex.....	38
4.5 Discussion	41
CHAPTER 5 ONLINE ESTIMATION OF TIME-VARYING GRAPH SIGNALS UNDER IMPULSIVE NOISE.....	43
5.1 Introduction	43
5.2 Impulsive Distributions.....	44
5.3 Existing Adaptive GSP Algorithms.....	46
5.4 The Adaptive GNLMP Algorithm	48
5.5 The Adaptive Graph Sign Algorithm	50
5.6 Convergence Analysis of G-Sign Under Steady-State Estimation.....	52
5.7 Graph-Sign-Diffusion Algorithm.....	54
5.8 Experimental Results.....	55
5.8.1 The Effect of the Step Size Parameter.....	56
5.8.2 Steady-State MSD Under Non-Gaussian Noises.....	57
5.8.3 Time-varying Estimation Under Impulsive Noise	59
5.8.4 GSD and G-Sign Comparison	61
5.9 Discussion and Future Work	62
CHAPTER 6 ADAPTIVE ESTIMATION OF TIME-VARYING INTERACTING SIMPLICIAL SIGNALS	64
6.1 Introduction	64
6.2 Joint Online Estimation of Two Interacting Simplicial Signals	65
6.2.1 Time-varying Simplicial Regression	66
6.2.2 Spectral Analysis of the Time-varying Simplicial Regression	68
6.2.3 The Hodge NLMS Algorithm	69
6.2.4 Joint Online Estimation Algorithm	72
6.2.5 Simplicial Aggregation.....	74
6.2.6 TSP Spectral Analysis of the JOE	75

TABLE OF CONTENTS

6.3 Specifying JOE Onto Graph Nodes and Edges	76
6.3.1 Joint Prediction of Edge and Node Signals	77
6.3.2 TSP Analysis for JOE on Edges and Nodes	77
6.4 Edge Signal Estimation via Line Graph Transformation.....	79
6.5 Experiment Results and Discussion.....	81
6.5.1 Experiment Set Up	81
6.5.2 Hodge-NLMS Analysis	82
6.5.3 JOE on Transportation Network.....	84
6.5.4 Covid-19 Case Prediction in Population Mobility Network.....	88
6.6 Discussion and Future Work	90
CHAPTER 7 TIME-VARYING GRAPH REPRESENTATION VIA DYNAMIC MULTI-HOP TOPOLOGIES.....	91
7.1 Introduction	91
7.2 Dynamic Multi-Hop.....	93
7.2.1 1-Hop Node Signal Aggregation.....	93
7.2.2 P-Hop Node Signal Aggregation.....	94
7.2.3 Dense Topology Representation.....	94
7.3 From Time-Varying Edge Signal To Dynamic Graphs	95
7.3.1 Time-Varying Edge Signal Representation	95
7.3.2 Topology Pruning	97
7.4 Applying Dynamic Multi-Hop to Brain Networks.....	98
7.4.1 Dataset Formulation	99
7.4.2 Network Analysis Using Dynamic Multi-Hop	101
7.5 Discussion and Limitations	103
CHAPTER 8 CONCLUSIONS AND FUTURE WORK	106
8.1 Conclusion.....	106
8.2 Future Work	108
REFERENCES	110
ACKNOWLEDGEMENTS	119

LIST OF FIGURES

Figure 1.1	Dynamic graphs of confirmed COVID-19 cases stored on the nodes and population mobility stored on the edges on different days (self-loops are omitted). Left color bar: edges (log scale). Right color bar: nodes.	2
Figure 2.1	A gene interaction graph.	12
Figure 3.1	A random sensor graph and the graph signal.	21
Figure 3.2	An illustration of message passing.	22
Figure 3.3	A single time instance of a multi-feature time-varying graph signal.	25
Figure 3.4	An illustration of the 0-simplex, the 1-simplex, and the 2-simplex.	26
Figure 3.5	An illustration of a simplicial complex that consists of signals on nodes (green), edges (black), and triangles (blue).	28
Figure 4.1	An illustration of a 3-layer Bi-SCNN network.	33
Figure 4.2	An illustration of one Bi-SCNN layer.	34
Figure 4.3	The run time of the Bi-SCNN when compared against the baselines for 10% missing rate.	38
Figure 4.4	The Ocean Drifter Complex and five ocean drifters.	40
Figure 5.1	The PDF of various non-Gaussian distributions all with $\mu = 0$ in logarithmic scale.	46
Figure 5.2	The graph signal of a sensor graph and its topology.	56
Figure 5.3	The first time instance of time-varying temperatures as a graph signal and its graph topology. (a) Ground truth graph signal. (b) Noisy observation (S α S noise) with missing values (missing values are treated as 0 in the figure). (c) Recovered graph signal by G-Sign.	57
Figure 5.4	MSD performance for estimating a steady-state graph signal under S α S noise using different μ_s	58
Figure 5.5	MSD of the experiments and the theoretical MSD for estimating steady-state graph signal under different noises.	59
Figure 5.6	Estimation of one selected node from a time-varying graph signal under different noises.	60
Figure 5.7	Wind speed prediction using adaptive graph algorithms. (a) Reconstruction of one node. (b) The NMSD over time.	61

LIST OF FIGURES

Figure 6.1	A graph with time-varying signals on both the nodes and the edges. (left color bar: edges, right color bar: nodes).....	66
Figure 6.2	Performance of the Hodge-NLMS under fixed noise and different step sizes.....	84
Figure 6.3	Performance of the Hodge-NLMS under fixed step size and different noise levels.....	84
Figure 6.4	JOE compared with separate estimation on one of the edges (a) and one of the nodes (b) of the Sioux Falls network.	86
Figure 6.5	NMSE of the JOE compared against separate estimation in the Sioux Falls network (top: edge, bottom: node).	87
Figure 6.6	NMSE of the JOE compared against separate estimation in the Anaheim network (top: edge, bottom: node).....	87
Figure 6.7	Dynamic graphs that record COVID-19 cases on the nodes and population mobility on the edges (self-loops are omitted). Upper row: ground truth. Lower row: predicted signals. Left color bar: edges (log scale). Right color bar: nodes.	88
Figure 6.8	Average prediction error per region of daily COVID-19 cases in England .90	
Figure 7.1	The original graph topology and the topologies formed by directly connecting p -hop neighbors. Left color bar: edges. Right color bar: nodes.....	95
Figure 7.2	The topologies formed at two time instances $t = 0$ and $t = 30$ by directly connecting p -hop neighbors then pruning the edges. Left color bar: edges. Right color bar: nodes.....	98
Figure 7.3	The topologies formed at two time instances $t = 0$ and $t = 30$ by directly merging all $\bar{\mathbf{L}}^{(p)}$ s for $p = 1...6$. Left color bar: edges. Right color bar: nodes.....	98
Figure 7.4	The formed graph topologies by taking the correlation of all node data and then pruning. The edge signals are fixed, but the node signals are time-varying.....	100
Figure 7.5	Topologies formed by directly connecting p -hop neighbors of a normal subject. Left color bar: edges. Right color bar: nodes.....	101
Figure 7.6	Topologies formed by directly connecting p -hop neighbors of an Autism subject. Left color bar: edges. Right color bar: nodes.....	102

LIST OF FIGURES

Figure 7.7	Topologies formed from a normal subject by Dynamic Multi-hop. Left color bar: edges. Right color bar: nodes.	103
Figure 7.8	Topologies formed from an Autism subject by Dynamic Multi-hop. Left color bar: edges. Right color bar: nodes.....	104
Figure 7.9	Comparison between the topology at $t = 52$ and $t = 60$ mapped on the brain image. The purple edges have appeared at $t = 52$ but not in $t = 60$ and the red edges edges have appeared at $t = 60$ but not in $t = 52$	104
Figure 7.10	The average unweighted node degree over time.	105

LIST OF TABLES

Table 4.1	Experiment accuracy on citation complex from 10% to 50% data missing rate.	39
Table 4.2	Experiment run time (in seconds) on citation complex with K=5, from 10% to 50% data missing rate.....	40
Table 4.3	Classification (test) accuracy on the Ocean Drifter Dataset. The number after the activation denotes the number of simplicial layers.	41
Table 4.4	Experiment run time (in seconds) on the Ocean Drifter Dataset. The number after the activation denotes the number of simplicial layers.	42
Table 4.5	The number of trainable parameters of each network setting on all datasets.	42
Table 5.1	Computational Complexity Analysis.....	51
Table 5.2	Run Time Comparison of Steady-State Experiments	58
Table 5.3	Run Time Comparison of Time-varying Temperature Predictions.....	60
Table 5.4	Run Time Comparison of Wind Speed Experiments.....	62
Table 5.5	A List of Adaptive Filters on Graphs	62
Table 6.1	Parameters for LGLMS and JOE in nodes	86
Table 6.2	Parameters for LGLMS and JOE in edges	86

LIST OF SYMBOLS AND ACRONYMS

ARCH	Autoregressive Conditional Heteroscedasticity
BiGCN	Binary Graph Convolutional Network
BiSCNN	Binarized Simplicial Convolutional Neural Networks
FLOM	Fractional Lower Order Moment
GAT	Graph Attention Networks
GCN	Graph Convolutional Neural Network
GFT	Graph Fourier Transform
GGM	Gaussian Graphical Model
GLMP	Graph Adaptive Least Mean p^{th} Power
GLMS	Graph Adaptive Least Mean Squares
GNLMS	Graph Adaptive Normalized LMS
GNLMP	Graph Adaptive Normalized LMP
GNN	Graph Neural Networks
GSD	Graph Adaptive Sign Diffusion
GSP	Graph Signal Processing
G-Sign	Graph Sign
LMP	Least Means p^{th} power
LMS	Least Means Squares
LR	Leaky Relu
JOE	Joint Online Estimation
NLMP	Normalized LMP
NLMS	Normalized LMS
SAT	Simplicial Attention Networks
SCNN	Simplicial Convolutional Neural Networks
SFT	Simplicial Fourier Transform
SNN	Simplicial Neural Networks
$S\alpha S$	Symmetric α Stable Distribution
TSP	Topological Signal Processing
k	Order of a simplex or simplicial complex
$\mathcal{X}_k$	k -simplex (node: $k = 0$, edge: $k = 1$)
$\mathcal{K}$	simplicial complex
$\mathcal{G}$	graph

t	time index
$\mathbf{x}[t] \in \mathbb{R}^N$	graph (node) signal at time t
N	Number of nodes of $\mathcal{G}$ (GSP notation)
N_0	Number of nodes of $\mathcal{G}$ (TSP notation)
N_1	Number of edges of $\mathcal{G}$
$\mathbf{L} \in \mathbb{R}^{N \times N}$	Graph Laplacian matrix
$\mathbf{D}_S \in \mathbb{R}^{N \times N}$	Graph Sampling (Masking) Matrix
$\mathbf{U} \in \mathbb{R}^{N \times N}$	Eigenvector matrix of $\mathbf{L}$
$\mathbf{\Lambda} \in \mathbb{R}^{N \times N}$	Diagonal Eigenvector matrix of $\mathbf{L}$
$\mathcal{F}$	Bandlimited Frequency set with $ \mathcal{F} $ frequencies
$\mathbf{U}_{\mathcal{F}} \in \mathbb{R}^{N \times \mathcal{F} }$	Bandlimited eigenvector matrix of $\mathbf{L}$
$\mathbf{M}_n \in \mathbb{R}^{ \mathcal{F} \times \mathcal{F} }$	Normalization Matrix of GNLMs
$\mathbf{\Sigma}$	Spectral graph filter
$h(\mathbf{\Lambda})$	$\mathbf{\Sigma}$ in terms of $\mathbf{\Lambda}$
$\hat{\mathbf{x}}[t] \in \mathbb{R}^N$	Estimated graph (node) signal at time t
$\tilde{\mathbf{x}}[t] \in \mathbb{R}^N$	Estimation error at time t
$\hat{\mathbf{s}}[t] \in \mathbb{R}^N$	GFT of $\hat{\mathbf{x}}[t]$
$\tilde{\mathbf{s}}[t] \in \mathbb{R}^N$	GFT of $\tilde{\mathbf{x}}_k[t]$
$\boldsymbol{\eta}_k[t] \in \mathbb{R}^N$	Noise on the graph (node) signal
$\hat{\mathbf{y}}[t] \in \mathbb{R}^N$	Noisy observation of $\mathbf{x}[t]$
μ_{lms}	Update step size of the GLMS algorithm
μ_n	Update step size of the GNLMs algorithm
μ_{lmp}	Update step size of the GLMP algorithm
μ_{nlmp}	Update step size of the GNLM algorithm
μ_s	Update step size of the Graph-Sign algorithm
$\mathbf{x}_k[t] \in \mathbb{R}^{N_k}$	k -simplicial signal at time t
$\mathbf{L}_{LG} \in \mathbb{R}^{N_1 \times N_1}$	Graph Laplacian of Line Graph
$\mathbf{A}_{LG} \in \mathbb{R}^{N_1 \times N_1}$	Adjacency Matrix of Line Graph
N_k	Cardinality of $\mathbf{x}_k[t]$
$\mathbf{L}_k \in \mathbb{R}^{N_k \times N_k}$	Hodge Laplacian matrix
$\mathbf{H}_k \in \mathbb{R}^{N_k \times N_k}$	Simplicial convolution or aggregation
$\hat{\mathbf{L}}_k[t] \in \mathbb{R}^{N_k \times N_k}$	Time-varying Simplicial 1-hop Regression matrix defined based on $\mathbf{L}_k$ and $\mathbf{x}_k[t]$
$\hat{\mathbf{H}}_k[t] \in \mathbb{R}^{N_k \times N_k}$	Time-varying Simplicial p -hop Regression matrix defined based on $\hat{\mathbf{L}}_k[t]$
$\mathbf{D}_{k,S} \in \mathbb{R}^{N_k \times N_k}$	Simplicial Sampling (Masking) Matrix
$\mathbf{U}_k \in \mathbb{R}^{N_k \times N_k}$	Eigenvector matrix of $\mathbf{L}_k$
$\mathbf{U}_{k,\mathcal{F}} \in \mathbb{R}^{N_k \times \mathcal{F}_k }$	Bandlimited eigenvector matrix of $\mathbf{L}_k$

LIST OF SYMBOLS AND ACRONYMS

$\mathbf{\Lambda}_k \in \mathbb{R}^{N_k \times N_k}$	Diagonal Eigenvector matrix of $\mathbf{L}_k$
$\mathbf{B}_k \in \mathbb{R}^{N_{k-1} \times N_k}$	Incidence matrix
$\mathbf{I}$	Identity matrix
$\mathbf{\Sigma}_k$	Spectral simplicial filter
$\hat{\mathbf{x}}_k[t] \in \mathbb{R}^{N_k}$	Estimated k -simplicial signal at time t
$\mathbf{R}_{k,l}$	Lower adjacency aggregation of JOE
$\mathbf{R}_{k,u}$	Upper adjacency aggregation of JOE
$\mathbf{R}_{k,b}$	Boundary aggregation of JOE
$\mathbf{R}_{k,c}$	Coboundary aggregation of JOE
$\hat{\mathbf{s}}_k[t] \in \mathbb{R}^{N_k}$	SFT of $\hat{\mathbf{x}}_k[t]$
$\boldsymbol{\eta}_k[t] \in \mathbb{R}^{N_k}$	Noise of the k -simplicial signal
$\hat{\mathbf{y}}_k[t] \in \mathbb{R}^{N_k}$	Noisy observation of $\mathbf{x}_k[t]$
μ_k	Update step size of the Hodge-NLMS algorithm
r_k	Aggregation weight of the Hodge-NLMS algorithm
Ω	Graph Aggregation
$\mathbf{M}_k^p$	k -simplex feature normalization of the p^{th} layer of Bi-SCNN
$\mathbf{Z}_k^p$	k -simplex output of the p^{th} layer of SCNN
$\hat{\mathbf{Z}}_k^p$	k -simplex output of the p^{th} layer of Bi-SCNN
$\bar{\mathbf{L}} \in \mathbb{R}^{N \times N}$	Pruned Graph Laplacian of Dynamic Multi-Hop

CHAPTER 1 RESEARCH BACKGROUND, CHALLENGES, AND EXPECTED OUTCOME

1.1 Research Background

Graph Signal Processing (GSP) is an interdisciplinary field that merges concepts from classical signal processing with graph theory to handle data that reside on irregular and non-Euclidean structures^[1-4]. Additionally, data in reality are often collected over time, meaning that data can contain time dimensions. Traditional signal processing algorithms primarily deal with time- or space-based signals that are not well-suited for graph-structured data due to not having effective graph representations^[2].

The fundamental graph representation is to record non-Euclidean data as nodes (vertices) and edges; high-order structures can be represented using more generalized topological structures such as simplicial complexes^[5-6], hypergraphs^[7], and cell complexes^[8]. Data can be residing on the nodes, the edges, or the high-order structures that are composed or inferred from the nodes and the edges. In machine learning, these data that reside on graph nodes are mostly treated as the node features^[4]. In signal processing, these data on the nodes are considered as graph signals^[3]. The signals on graphs can be multidimensional from multiple perspectives. The topological structure of a graph provides a multidimensional and spatial representation^[5]. Time-varying signals have a time dimension in addition to the spatial connections of a graph^[9]. For the signals on graphs, in addition to the multivariate nature, there may exist multiple features^[10]. In classical signal processing, signals can be processed in both the time domain and the spectral (frequency) domain. For the signals on graphs, we can make the same analogy where these signals can be in different domains. Just as the Fourier Transform provides a frequency decomposition for classical signal processing tasks, the Graph Fourier Transform decomposes graph signals in terms of the eigenvalues and eigenvectors of a graph^[11].

Graphs have shown remarkable success at solving many real-world tasks where they can be used to analyze data residing on complex networks like the brain connection^[12], human psychological behavior^[13], and the stock market^[14]. Graphs can be used to monitor traffic in road networks^[15], model electrical grids^[16], and identify anomalies^[17]. Images can be turned into graphs and processed using GSP methods, especially for tasks that need to consider local relations in the image^[18]. Graphs are essential building blocks

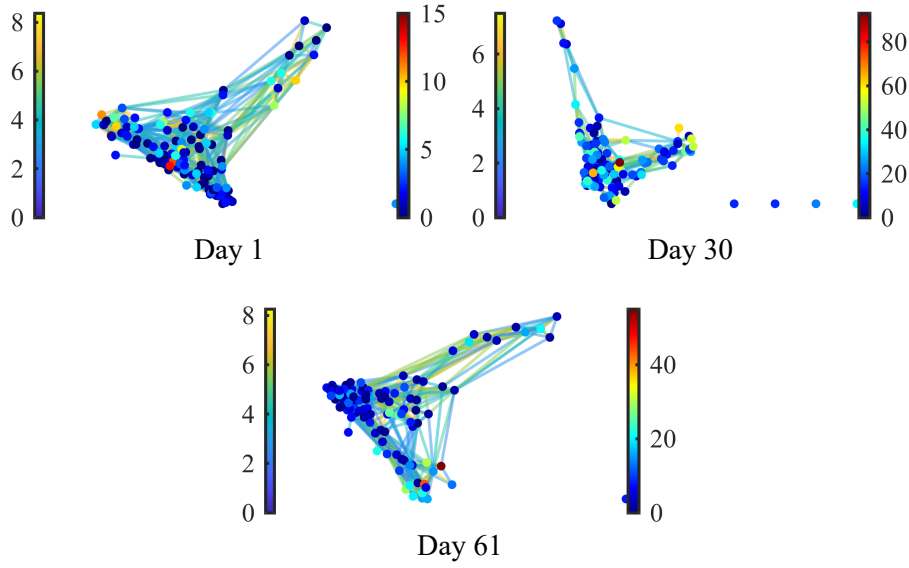


Figure 1.1 Dynamic graphs of confirmed COVID-19 cases stored on the nodes and population mobility stored on the edges on different days (self-loops are omitted). Left color bar: edges (log scale). Right color bar: nodes.

in many Artificial Intelligence applications to solve tasks including but not limited to classification^[18-20], clustering^[21-22], and regression^[23-24]. Graphs have also entered the field of deep learning, giving birth to the graph analogies of some well-known deep learning concepts from fields such as Computer Vision and Natural Language Processing. To name a few of such concepts, there are graph versions of the convolutional neural networks^[18-19], graph versions of the word2vec (node2vec)^[25], and graph version of the attention mechanism^[26].

To summarize, graph representations are ubiquitous in many fields of research and have a vast amount of application areas. In this dissertation, we will identify a few research challenges that are specific to the multidimensional time-varying signal on graph-like structures, and give a few solutions from the signal processing aspect as attempts to overcome these challenges.

1.2 Research Challenges

Four major challenges could impede the performance of classical signal processing algorithms on multivariate time-varying signals that reside on graph and topological structures. We will be giving an introduction to the four challenges in this chapter. A more detailed discussion will be conducted in later chapters that focus on the attempts to obtain problem-solving solutions.

- The representation of topological multidimensionality and irregularity: Graphs form a relationship from the connection of nodes and edges. These basic components can be manipulated into high-order topological structures such as multi-graphs, 3D shapes, simplicial complexes, polyhedral complexes, delta complexes, point clouds, and cubical complexes. These data representations are different from the traditional vector-like or grid-like structures seen in fields such as computer vision and natural language processing, which poses a challenge in representing data irregularity. Each element within the previously mentioned topological structures, for example, the nodes, the edges, and the triangles formed by the edges, may have signals residing on them. Each signal can have multiple features or channels, which increases the data dimension to another level. This nature of graph-structured data makes graphs naturally multidimensional multivariate data. In addition, real-world graphs can be very large, so making the computations efficient is an additional challenge.
- The extraction of the cross-space-time variation: Many real-world data on graphs and other topological structures could have time-varying behaviors. In addition, the topologies themselves could evolve over time. Conventional signal processing algorithms must be modified into adaptive or time-varying models on graphs to handle this dynamic nature and capture the space-time evolution.
- The reduction of the (extreme and impulsive) noise presence: Data gathered from the real world is often noisy due to various kinds of measurement errors, and graph-structured data is not an exception. However, noise can introduce errors, meaning that noisy data inputs can cause inaccuracies and lead to incorrect algorithmic outputs. Moreover, the popular noise assumption is that noise follows a Gaussian distribution, but it has been shown that noise in many real-world applications has extreme and impulsive behaviors that could not be modeled by a Gaussian distribution. These impulsive noises may cause instabilities in the applications that heavily rely on data or signals to operate. Considering the fact that the signals on graphs and networks are irregular, multidimensional, and multivariate, signal-denoising tasks become considerably more challenging when compared to some commonly seen non-graph tasks.
- The estimation of unknown data based on known data: The prediction, estimation, and interpolation of missing or unknown data are among the most common tasks

in the application of machine learning algorithms in real-world scenarios. Other than the possible presence of noise in the data gathered from the real world, data may be incomplete or contain missing values, and graphs are no exception. In addition, in some forecasting or prediction tasks, people are often required to make fast estimations of future data based on past and current observations. Thus, it is necessary to obtain efficient and effective methods for the inference, estimation, or prediction of unknown data based on known data that are designed specifically for graph-like topological structures.

1.3 Research Objectives

- The development of signal estimation and denoising algorithms that could operate under impulsive noise.
- Algorithms that utilize GSP to handle multidimensional time-varying signals residing on the nodes of a graph.
- Further extend the graph node algorithms to high-order structures using Topological Signal Processing (TSP) techniques.
- A framework that could extract and represent the cross-space-time variation as a dynamic graph model.

1.4 Dissertation Outline

The dissertation is organized as follows. In Chapter 2, we will conduct a literature review of notable and useful literature that is closely related to signal processing and learning techniques on graph or network structures in general. Chapter 3 contains discussions on time-invariant GSP and TSP techniques, serving as the preliminaries before diving into the time-varying processing. Our very recently proposed neural network, the Binarized Simplicial Convolution Neural Networks, is introduced in Chapter 4, serving as a demonstration of using the time-invariant techniques shown in Chapter 3 in practice. Starting from Chapter 5, we will be delving into the realm of time-varying signal processing on the graph nodes; we will be approaching this problem from two of our proposed algorithms: the Adaptive Graph Normalized Least Mean p^{th} algorithm, and the Adaptive Graph-Sign algorithm. In Chapter 6, we further extend adaptive GSP algorithms into the field of TSP, formulating a Joint Online Estimation Framework, enabling us to process

signals on high-order topological structures. In Chapter 7, a Dynamic Multi-Hop model is built to represent the dynamic node interactions, which is different from the conventional fixed-graph representation. Chapter 8 concludes the dissertation and discusses several potential future research directions. The list of publications supporting this dissertation can be found in the Resume chapter at the end of the dissertation.

CHAPTER 2 LITERATURE REVIEW OF RELATED WORK

2.1 Foundations of Signal Processing on Graphs and Networks

In the field of Graph Signal Processing (GSP), graphs have demonstrated their capacity to represent information in a variety of real-world scenarios and disciplines of study. Graphs have been studied extensively and attracted considerable attention due to their irregular structure, as well as their effectiveness in representing interactions among data^[1-3]. Early graph-related research has mostly focused on the graph structures themselves without considering the possibilities of graphs containing data other than the topological connections created by linking the nodes^[27]. Techniques within GSP emerged from spectral graph theory by considering the coexistence of topology and signals on the graph nodes. Spectral methods on the graph nodes based on graph theory were first proposed by Hammond et al., in which the wavelets were constructed from the spectral decomposition of the discrete graph Laplacian^[28]. The graph wavelets gave birth to this interdisciplinary research topic, sprouting the groundbreaking combination of classical signal processing concepts with graph theory, handling data residing in irregular and non-Euclidean spaces. In GSP, operations are often defined using graph shifts, which are defined based on the graph topology and also serve as graph representations^[3]. The two most common graph shift operators are the graph adjacency matrix and the graph Laplacian matrix. In classical signal processing, spectral operations are often defined in the spectral domain by transforming the data into the spectral domain using the (classical) Fourier Transform. In GSP, similar operations can be done by the spectral decomposition of the graph Laplacian Matrix. This spectral decomposition of the graph Laplacian is known as the Graph Fourier Transform (GFT) because its structure resembles the Discrete Fourier Transform in classical signal processing^[3]. The discussion on Finite Impulse Response (FIR) graph filters was conducted by Ortega et al.^[3], and the discussion on Infinite Impulse Response (IIR) graph filters can be found in the work of Shi et al.^[29]. When viewing the graph shift operator from the spatial domain, these operations make the signals on the graph nodes propagate to and aggregate into their neighbors, which are one edge away, giving us the "shift" when such propagation is done to all the nodes on the entire graph. This signal propagation can also be viewed as a diffusion or an aggregation^[30]. To link together the spatial and the spectral graph techniques, one can rely on using a spatial

approximation to the spectral methods, which could be done by the Chebyshev polynomial approximation^[31]. Currently, the spatial graph methods are preferred, especially in the deep learning community^[19,21,32]. The reason behind the preference for the spatial method is that it can reduce the computational complexity when the graph size is large or when there is a topological change, as spectral methods rely on eigenvector decomposition, which is computationally demanding and unstable for large graphs. However, it should be emphasized that the spatial methods are interpretable using the GSP spectral analysis. A review of the methods to generalize fundamental operations such as graph filtering, graph translation, graph modulation, graph dilation, and graph downsampling was given, and a survey on the localized, multiscale transforms that are proposed to efficiently extract features from high-dimensional data on graphs was provided^[2]. Signal restoration from noisy data is considered for signals on graphs using the GSP models^[1], where graph signal denoising is presented as a convex optimization problem leading to a closed-form solution in the form of an inverse graph filter. This GSP denoising idea became the building block of many other GSP algorithms that appeared later. GSP-based algorithms have the capability of solving many classical machine learning tasks such as classification^[33] and clustering^[22]. However, these algorithms can only handle static tasks that do not operate in real time; this drawback of the static models urges the need for online processing techniques and the consideration of time-varying data.

2.2 Time-Varying Algorithms on Graphs with Gaussian Noise Assumption

Signals in reality often contain time-dimension information; when processing data on graph nodes that evolve with time, one could rely on adaptive graph filters, which are a combination of classical adaptive filters with graph shift operations. In classical signal processing, adaptive filters are algorithms that utilize filters that have the ability to perform self-adjustments during the online estimation of signals. The update scheme of adaptive filters is usually derived according to some optimization problems, which is often done either at the signal level or the filter level. On the signal level, the optimization problem is usually set based on the minimization of certain error measures between the observed signal and the estimated signal. On the filter level, the optimization problem is usually to update the filter weights based on the error measure calculated at the signal level. Through this alternative implementation, iterative application of the adaptive filter

could achieve noise reduction during the online estimation process^[34]. In GSP, a combination of classical adaptive filters with graph shift operations performs efficiently on the online processing of time-varying graph signals owing to its simplicity of implementation. One could avoid the time-consuming process of learning or designing the filters, as seen in classical adaptive filters, by utilizing a predefined bandlimited filter on the global level derived from GFT based on the graph smoothness or graph bandlimited assumption^[3,35]. This means that in conventional adaptive graph algorithms, the update is done only at the signal level, in which the algorithm will update in the direction opposite to the error at each time step based on convex optimization. The first of such proposals is the adaptive graph Least Mean Squares (GLMS) algorithm^[35], where the algorithm uses a fixed, predefined bandlimited filter obtained from GFT to update in the direction opposite to the error based on an l_2 -norm optimization at each time step, with the assumption of Gaussian noise^[35]. Notice that as we mentioned in Section 2.1, GSP algorithms are not limited to only processing the node signals using spectral techniques. The definition of Adaptive filters using GSP could also be accomplished using spatial techniques. Taking the GLMS as an example, the spectral domain filtering operation in the GLMS can be approximated using a series of Chebyshev polynomials, resulting in the spatial domain graph diffusion LMS^[30]. Building on top of the success of GLMS, other least-squares-based adaptive GSP algorithms such as the Graph Normalized LMS (GNLMS)^[36] and the Graph Recursive Least Squares (GRLS) algorithm^[37] are GSP analogues of classical adaptive filtering algorithms developed for faster convergence than the GLMS. Recently, the distributed version of GSP algorithms has also gained attention. The idea is to define each operation distributively and locally on the graph nodes by the Chebyshev approximation of the spectral graph filters^[31]. This will give us distributed versions of the GRLS algorithm and GLMS algorithm^[37-38]. Similarly, distributed kernel GLMS was introduced, which extended the linear graph adaptive algorithm to a nonlinear version^[39].

To tackle the issues of temporal variability and to represent space-time interactions, another approach is to merge time series analysis methods with GSP. Classical multivariate time series models include the Vector Autoregressive (VAR) model^[40] and the Vector Autoregressive Moving Average (VARMA) model^[40]. A VAR model is when the current output of a multivariate variable is a linear regression of previous values. The VARMA model further includes a moving average of the past noise (error) terms in addition to the VAR model, offering a more accurate and versatile model. Time-series analysis al-

gorithms were introduced into GSP by specifying their parameters in the spectral domain via GFT, using the multivariate time-series as graph signals and potentially decreasing parameter size compared to traditional VAR and VARMA^[24,41]. The GSP analogy of VAR and VARMA is the graph VAR model^[24] and the graph VARMA model^[42], respectively. However, these models have the limitation of having a constant covariance, which cannot account for time-varying features, such as volatility. In time series analysis, Autoregressive Conditional Heteroscedasticity (ARCH)^[43] and generalized ARCH (GARCH)^[44] have been proposed to deal with symmetric volatility; subsequently, the exponential GARCH (EGARCH) model^[45] and the GJR-GARCH model^[46] have been introduced to study asymmetric volatility. A unified model that defines the GARCH model on graphs using GSP can also be formulated^[47].

State-space models, such as the Kalman Filter^[48], can also be combined with GSP. The Kalman filter is a recursive algorithm used to estimate the state of a linear dynamic system under a Gaussian noise assumption. The Kalman filter recursively tracks the state estimation of the system and the uncertainty of the estimation. Each new measurement received will be used to update the predicted state and the uncertainty. On fixed graphs, the Kalman Filter can be established as the optimal estimation method at each node, where node communications are formed by the edges^[49]. By calculating the steady-state error covariance matrices in networks modeled in the form of trees, the Kalman Filter over graphs enables improved estimation accuracy by leveraging the topological structures compared to conventional Kalman filters. Very recently, graph Kalman filters are being improved to go beyond the Gaussian noise assumption^[50-51]. We will be discussing these in the next section of this Chapter.

A line of work known as Time-Vertex Signal Processing was proposed to take into account the time-domain information, addressing the time-varying nature of some graph data. The cost function of time-vertex signal processing algorithms is set up using approaches similar to the GLMS l_2 -norm cost function with an additional regularization term and utilizes the Joint Time-Vertex Fourier Transform for the inclusion of the time-domain information^[52]. However, the Time-Vertex algorithm is a batch algorithm that updates all signals from all-time instances at once, which may not be practical for certain real-world applications.

2.3 Time-Varying Algorithms on Graphs with non-Gaussian Noise Assumption

The Gaussian noise assumption is seen in most noise models, and l_2 -norm optimization is the go-to option for Gaussian noise because minimizing the squared error corresponds to the maximum likelihood estimate solution^[53]. However, the underlying noise in a variety of realistic applications, including meteorological recordings^[54] and powerline communication^[55], is verified to possess impulsive behaviors that could be represented by heavy-tailed, non-Gaussian distributions, such as generalized Gaussian, Student's t, and α -stable distributions^[53]. The impulsiveness presented in heavy-tailed distributions can be characterized by large or infinite variance, causing l_2 -norm optimization-based algorithms like GLMS and GNLMS to be unstable^[56].

Least-squares-based adaptive graph methods, namely the GLMS, the GRLS, and the GNLMS algorithms, assume the noise to be Gaussian, which is invalid for impulsive noise situations. The least-squares optimization is unstable and will diverge in the presence of large or infinite variance caused by the impulsiveness in the noise^[56]. The GRLS algorithm further requires prior knowledge of the covariance matrix, which is not acquirable in many cases. This means that the optimization problem should be defined by a loss that is more stable than the l_2 -norm. To avoid the drawbacks caused by l_2 -norm optimization when the noise distribution is impulsive and non-Gaussian, a robust adaptive estimation of graph signals is done by replacing the l_2 loss with Welsch loss^[57]. On the other hand, a simple approach can be taken where, instead of using the second power, a least-mean p^{th} power is used to define a loss based on the assumption that the noise follows a symmetric α -stable distribution (S α S). Under the S α S noise assumption, the optimal cost function can be derived based on the minimum dispersion criterion, leading to l_p -norm optimization instead of l_2 -norm optimization^[58]. To eliminate the downsides caused by performing l_2 -norm optimization under non-Gaussian noise in a graph signal, the graph Least Mean p^{th} power (GLMP) was proposed to use l_p -norm optimization instead^[59]. There are two drawbacks to the GLMP algorithm: there are additional computations to the already expensive GLMS algorithm, and its parameter selection is still based on prior knowledge of the noise. In Chapter 5, we conducted a thorough discussion on how to tackle these two problems and provided efficient solutions to solve these two problems.

In the previous section, we went over the Kalman filter on graphs, but, as in its classical signal processing analogy, the Kalman filter on graphs still relies on the Gaussian

noise assumption. The Extended Kalman filter extends the applicability of the standard Kalman filter from linear systems to non-linear systems by locally approximating them as linear^[60]. The graph version of the Extended Kalman filter is a graph process obeying a nonlinear state-space model that can be found by forming a GSP-based Bayesian filter whose goal is to minimize the computed covariance matrix as the Kalman gain of the Extended Kalman filter in the graph spectral domain^[50]. A deep learning approach has been taken to formulate conventional and Extended Kalman filters where, instead of directly building them on graphs, one can use a neural network to encode the dynamic graph structure and also train the weights using neural networks^[61]. The Unscented Kalman Filter addresses the non-linearity of systems by representing the mean and covariance of the state using a set of deterministically chosen sample points to avoid the need for linearization, which is different from the standard Kalman filter^[62]. This approach can also be applied to graph signals, which leads to the Unscented Kalman filter of graph signals by again using the decomposition of the graph Laplacian matrix to form the GFT, then the design of the Kalman gain matrix from the eigenvectors^[51].

2.4 Topological Signal Processing on High-order Structures

Despite the success of GSP, data in GSP is solely on the graph nodes, meaning that higher-order signals, such as signals on graph edges, are not being represented. Examples of signals existing on the graph but not only on the graph nodes can be found in the following literature: traffic data of road networks are recorded on the edges^[63], gene interactions over time are formed as a set of time-varying graph edges^[64], agent interactions in multi-agent systems are modeled using time-varying edge data^[65]. In existing GSP algorithms that process time-varying node signals, the interaction among the nodes is modeled using fixed edge weights at all time instances.

In the Signal Processing community, researchers have proposed the concept of Topological Signal Processing (TSP)^[5]. This recently emerged field of study has established an alternative point of view from GSP, overcoming the limitation of GSP by generalizing graphs to simplicial complexes. Instead of the graph Laplacian, a more generalized Laplacian known as the Hodge Laplacian is adopted in TSP to incorporate the representation of higher-order features that are beyond the graph nodes. The signals on the nodes of a graph are considered as signals of the 0^{th} - order simplex, while higher-order simplices could be used to represent signals on the edges and beyond^[5-6]. High-dimensional

online estimation framework addressing time-varying, multi-order, and multivariate data to bridge multi-order space-time gaps. The above-mentioned Hodge Laplacian-based algorithms demonstrated good performance in processing signals on graph edges, but we may encounter additional challenges in more complicated situations. First and foremost, prior TSP and GSP literature primarily focused on signals in a single simplicial order. The consideration of processing signals that reside in multiple orders of simplices needs to gain more attention because, in reality, signals may appear and interact in multiple orders of simplices. Additionally, a deeper discussion should be conducted on time-varying signal models using the Hodge Laplacian or TSP since numerous real situations require online time-varying data models; static algorithms may not perform optimally to capture the dynamic change in the time-varying data. On a side note, data gathered from the real world is often noisy, meaning that the denoising of simplicial data should be considered when formulating simplicial representations. These challenges comprise addressing topological irregularities, revealing data interactions across multiple dimensions, eliminating data noise, and capturing time-varying patterns. Therefore, it is a pressing necessity for an online algorithm that focuses on time-varying multi-order and multivariate data on simplicial complexes to bridge the cross-space-time signal gaps.

2.5 Neural Network Algorithms on Graphs and Higher-Ordered Structures

Even though the above-mentioned GSP approaches have shown successful results in the online estimation of graph signals, unlike classical adaptive filters that adaptively update the filter weights, the performance of adaptive graph filters relies on a fixed, predefined filter. However, an accurate definition of graph filters requires prior knowledge of the spectrum of the graph signal, which may not be known in all situations. By introducing nonlinear activation functions into GSP, architectures such as Graph Convolutional Neural Networks (GCN) and Graph Attention Networks (GAT) have extended the spatial and spectral GSP methods to machine learning tasks such as node classification, link prediction, and image classification^[18-19,21,77]. Graph Neural Network (GNN) methods such as GCN and GAT learn the filter from the given data through backpropagation. These GNN architectures can be generalized by Message Passing Neural Networks (MPNN), where the graph representations and operations are defined locally on the nodes by a message passing scheme instead of globally on the graph topology^[78-79]. Compared to specific ar-

chitectures such as the GCN, MPNN provides more degrees of freedom by having several choices of aggregation functions such as sum, mean, or max and allowing us to tune the magnitude of the localized aggregation flexibly^[79]. Compared to graph representation done by the adjacency matrix or the Laplacian matrix that gives the global view of the graph, MPNNs provide localized representations based on node neighborhood relationships. In addition, edge weights can also be incorporated into the aggregations through message passing.

The GNN techniques discussed above have shown success when the data is purely in the spatial dimension but not the time dimension, which leads to the requirement for additional treatment when the data also contains the time dimension. There are very few attempts at combining the GSP algorithms with other deep-learning algorithms to include the processing of time-varying graph signals. The Spatio-Temporal GCN (STGCN) uses a combination of GCN and gated CNN to process spatial features and temporal features^[80]. However, the STGCN does not directly deal with missing data in the spatial domain and measurement noise in the signal. Another approach proposed by Lewnfus et al. can be taken, where they transfer the graph signal to the spectral domain using the GFT, then process it by a sequence of Gated Recurrent Units (GRU) and transfer it back to the spatial domain^[81]. The major drawback of this empirical approach is that the GFT only acts as a domain transformation, and the performance relies only on the GRU, which is not optimized for graphs and has low interpretability.

Deep learning algorithms making use of simplicial complexes to process data on the edges have been proposed by introducing nonlinearity to TSP. A graph neural network for edge data based on the Hodge Laplacian can be formulated using TSP^[75]. The work of Yang et al. defined a convolutional neural network based on simplicial complexes^[82]. The attention mechanism is merged with the Hodge Laplacian^[32,83]. Algorithms that conduct convolution on the nodes of a graph, such as GCN^[19], operate only on the graph nodes, but not on k -simplices with $k \geq 1$. The Simplicial Neural Network (SNN)^[84] generalized the GCN to simplicial complexes by using TSP to extend the convolutional ability of GCN to process higher-order simplicial features. The Simplicial Convolutional Neural Networks (SCNN)^[85] further improve the performance of SNN by applying the Hodge decomposition to the simplicial convolution so that it can train the upper and lower simplicial adjacencies separately. The Simplicial Attention Network (SAT)^[86] and the Simplicial Attention Neural Network^[87] introduced the attention mechanism to SNN and

SCNN to process simplicial features, making them generalized versions of the GAT from graphs to simplicial complexes. Both the SNN^[84] and the SCNN^[85] are direct extensions of the GCN onto simplicial complexes; SNN is architecturally identical to the SCNN except that SNN does not use Hodge decomposition.

Although the simplicial complex-based neural networks have achieved numerous successes, the focus of the previous networks was on how to extend concepts such as convolution or attention from graphs onto simplicial complexes, which leads to two drawbacks. First, the recently proposed simplicial neural network structures do not prioritize time efficiency, resulting in inefficiency in terms of computational complexity and run time. For instance, in the context of GCNs, the Simplified GCN^[88] has shown that a trained GCN essentially is a low-pass filter in the spectral domain, and the complexity of GCN can be reduced by eliminating nonlinearities and collapsing weights. The Binary-GCN (Bi-GCN)^[89] was inspired by the Binary Neural Networks to use binary operations such as XOR and bit count to speed up training time. In the field of Graph Signal Processing, the Adaptive Graph-Sign algorithm^[9] derived from l_1 -optimization results in a simple yet effective adaptive sign-error update that has outstanding run speed and robustness. To the best of our knowledge, similar improvements that focus on the efficiency and complexity of neural network algorithms on the simplicial complex were previously absent. In their graph predecessors, the over-smoothed phenomenon was extensively analyzed^[90-91] but has not yet gained enough attention in the simplicial complex-based neural networks.

2.6 Topology Acquisition Techniques

As mentioned earlier, one of the ways to utilize graphs as a data structure is to represent multivariate signals. The underlying topological structures of a graph signal are often formed to represent the pairwise relationship between two nodes through the connection of edges. Since an edge between two nodes is a pairwise relationship by definition, we can intuitively form and remove edges based on some measures of the strength of the relationship between the two data points on each node. When this process is repeated on all the data pairs, a graph is formed on the global level.

In statistical graphical models, statistical measures between two data elements are used to define the pairwise relationship. For example, the covariance matrix and the precision matrix can be used to form a graph because covariance is a statistical measure of

the deviations between two variables, meaning that measuring and properly thresholding this pairwise relationship gives us a sparse graph^[92]. The Gaussian Graphical Model is a specific example of a statistical graphical model that represents a conditional dependency structure in a graph: the nodes of the graph are the multivariate Gaussian distributed variables, and edges between these variables are the conditional dependencies or non-zero partial correlation coefficients^[93-94]. In Graph signal processing, operations such as graph construction, transforms, sampling, prediction, and regularization can be realized from a probabilistic and statistical aspect using the Gaussian Graphical Model^[95]. The graph Laplacian can be learned using a merging of the Gaussian prior and a smoothness assumption on the multivariate signal, which is equivalent to learning the underlying topological structure^[96]. A multivariate signal can be assumed to be a sparse linear combination of atoms from a dictionary based on a polynomial of graph Laplacians that represents localized patterns from a sparse prior^[97]. Another sparse prior assumption topology learning approach can be found proposed by Belilovsky et al.: the Gaussian Graphical Model is used in a way that data is trained in a neural network to learn a function calculating the empirical covariance matrices to form graph structures^[98].

The pairwise relationships in graphs can be non-linear and dynamic. Nonlinearities and time-evolving processes over graphs can be captured by kernels formed by a preselected dictionary that fits the signal of interest, where partial correlation, partial Granger causality, non-linear structural equations, and vector autoregressive techniques are used along with assumptions of the graph signals such as low rank, sparsity, and smoothness^[99-100]. A dynamic graph can be formed from the correlated properties previously seen in static statistical graphical models in time-varying graph signals by considering the space-time interactions in the underlying signal^[101-103]. The topology inference on dynamic signals can be approached using a state-space formulation of a network process^[104]. The Graphical LASSO is another instance of a statistical graphical model that estimates the inverse covariance matrix (precision matrix) of a Gaussian Graphical Model with sparsity constraints enforced through an l_1 penalty^[105]. By combining Graphical LASSO with an ADMM-based algorithm, one can incorporate signals from neighboring time points and pairwise connections to learn the smoothly changing graphs^[14]. Another smoothness-based graph topology learning task is to use the Sobolev smoothness as a measure to define a new smoothness function for time-varying graph signals^[106]. The work of^[107] proposes to use the structural equation model to learn graph topologies

in an online fashion, and^[108] further proposes a combination of the Gaussian Graphical Model, the structural equation model, and the smoothness-based model into an all-in-one online topology learning framework. The causal relationships between data can be modeled using a VAR model, making it possible to infer a graph topology from the underlying time-series vector^[109]. A specific example of this approach can be seen in a time-varying gene activation graph formed by thresholding a regression matrix of a time-varying gene regression that reflects gene activation patterns over time, leading to a parametric representation of the time evolution and influence between pairs of data^[64]. Given an underlying graph structure at a single time instance, assuming that the dynamics of a time-varying graph only have some minor changes between time instances, a small perturbation analysis of the Laplacian matrix and its GFT can be done to model the time-evolution of the dynamic graph^[110]. Topology inference of a dynamic topology from time-varying data can be tackled from another angle using an iterative prediction-correction approach on a Gaussian Graphical Model^[111].

Real-world data represented on the nodes can be constructed based on real-world relationships. For example, a graph of 5G signal reception strength is formed by taking measurements of signal strength at different locations^[35]. When meteorological data are gathered, the altitude, longitude, and latitude at multiple weather stations can be collected and passed into a Gaussian Kernel to form a k-nearest-neighbor graph^[36].

2.7 Applications

GSP techniques that have emerged present themselves as highly efficient solutions for addressing the complexities and irregularities previously mentioned, particularly within the context of real-world applications. These techniques offer a promising direction for analyzing and interpreting data that is inherently irregular and multidimensional, thereby overcoming traditional limitations and unlocking new potentials in various application domains. Graph structures can be deduced from various real-life applications by embedding the features onto the nodes of a graph represented using variants of the graph adjacency matrix or the variants of the graph Laplacian matrix.

Network and graph-structured data have become increasingly popular across various research fields including computer science, social science, biology, engineering, and finance, owing to their unique ability to represent multivariate irregularities^[1,3-4,112]. Data measurements recorded by an array of sensors can be utilized to formulate sensor graphs.

These graphs are usually associated with the physical positions of the sensors, which serve as the coordination and neighborhood relationships for the graph to form the graph topology. Within this approach, the signals or features derived from the sensor data are embedded directly into the nodes of the graph. Examples of these sensor graphs include recordings of 5G signal reception strength^[35], air quality^[113], and nationwide temperature^[36]. Take the sensor graph in^[35] as an example; the graph topology is formed by connecting different spatial locations. This graph aims to collect data at different locations through the sensors and then estimate the distribution of power across space. In the field of bioinformatics, graphs could serve as a powerful modeling tool for representing complex biological systems and interactions. These graphical models can serve as powerful representations of time-varying gene interactions^[64], functional brain imaging^[12], brain functionality across regions of the brain^[114], monitoring sensor networks in smart cities^[16], structuring geometric data^[115], and biomedicine^[116]. In the domain of finance, graphical models have been proposed as a sophisticated method to represent the dynamics of the stock market by modeling the interactions among individual stocks^[14].

Signals and features on graphs are not limited to residing solely on the nodes; they can also appear on higher-order structures, such as the edges of the graph. Traffic flow could be recorded on either the edges^[76] or the nodes^[117] of a graph. A map of ocean drifter trajectories can be modeled using the data on graph edges^[118]. Citation graphs^[119] can be formed by linking the citation relationship between papers. In a citation complex, a collaboration among three or more authors is modeled using simplicial complexes^[84], which is a scenario that cannot be simply recorded using only graph nodes because a traditional node-edge representation can only represent a one-to-one relationship between two authors. More such examples where data appears on structures with higher-order than graph nodes include pandemic prediction based on population mobility among regions^[120] and dynamic gene interaction graphs^[64]. The prediction of protein-protein interactions in a living system can be made using GSP by a Markovian model of the signal on a graph that records connectivity properties of the proteins on nodes; the inferred edges correspond to interactions between proteins^[121].

CHAPTER 3 PRELIMINARY KNOWLEDGE AND TIME-INVARIANT SIGNAL PROCESSING ON TOPOLOGICAL STRUCTURES

In this chapter, we will be introducing several time-invariant methodologies on the graph nodes, which will become the building blocks of the time-varying algorithms we are going to propose in later chapters. Then, the Graph Signal Processing (GSP) techniques will be introduced to a more generalized scenario where signals will appear on higher-order structures than nodes. The discussion of extending these signal-processing techniques to neural networks is also included.

3.1 Processing of Signals on the Graph Nodes

3.1.1 Graph Signals With a Single Feature

Let us first define a graph topology denoted as $\mathcal{G}$. A graph $\mathcal{G}$ consists of two sets $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, in which the set $\mathcal{V}$ is the set of N nodes and the set $\mathcal{E}$ is the set of edges. We can easily record the pairwise connectivity of the nodes in a network by using a graph adjacency matrix $\mathbf{A}$. For an unweighted graph, the edge connectivity is recorded in the adjacency matrix, where the magnitude of the ij^{th} element is assigned a value of 1 if there is an edge connecting node v_i to node v_j , and a value of 0 otherwise. For a weighted graph, the specific edge weight from node v_i to node v_j is stored at the ij^{th} entry of the graph adjacency matrix $\mathbf{A}$, and it is marked as 0 if there is no pairwise connection between those two nodes. The degree matrix $\mathbf{D}$ of an undirected, unweighted graph is a diagonal matrix where the i^{th} diagonal value represents the number of edges connected to node v_i . As for a weighted graph, the degree matrix $\mathbf{D}$ is also a diagonal matrix, but unlike the unweighted case, the i^{th} diagonal entry is calculated as the sum of all the weights of the edges that are connected to node v_i . A self-loop is counted 2 times in the undirected case. The diagonal entries of the degree matrix are known as the node degrees. A graph signal $\mathbf{x}$ is the function values defined on the nodes of the graph $\mathcal{G}$; each element in $\mathbf{x}$ corresponds to a signal at a single node.

There is one more step needed to make the analogy to the famous Fourier Transform in classical signal processing in GSP. The graph Laplacian matrix $\mathbf{L}$ of an undirected graph

has the property of being a positive semi-definite matrix and is defined as

$$\mathbf{L} = \mathbf{D} - \mathbf{A}. \quad (3.1)$$

The GFT is defined using the eigenvector decomposition (EVD) of $\mathbf{L}$:

$$\mathbf{L} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T. \quad (3.2)$$

Here, the EVD will result in N distinct eigenvalue-eigenvector pairs: the matrix $\mathbf{\Lambda}$ is the eigenvalue matrix with N eigenvalues on the diagonals; the matrix $\mathbf{U}$ is composed of the N orthonormal eigenvectors of $\mathbf{L}$ ^[37]. The GFT of graph signal $\mathbf{x}$ is used to transform $\mathbf{x}$ from spatial domain to spectral domain and is defined as

$$\mathbf{s} = \mathbf{U}^T \mathbf{x}, \quad (3.3)$$

which is the projection of $\mathbf{x}$ onto $\mathbf{U}$. The inverse Graph Fourier Transform (IGFT) is formulated as

$$\mathbf{x} = \mathbf{U}\mathbf{s}, \quad (3.4)$$

which transforms $\mathbf{s}$ from spectral-domain to spatial domain.

GSP algorithms can benefit from sparsity defined in both the spatial domain and the spectral domain. A bandlimited graph signal is sparse in the spectral domain^[36-37] defined by a frequency set $\mathcal{F}$ with $|\mathcal{F}|$ spectral-domain elements. To get a bandlimited representation $\mathbf{x}_0$ of a graph signal $\mathbf{x}$, we apply a band limiting filter $\mathbf{\Sigma}$ based on $\mathcal{F}$, where $\mathbf{\Sigma} \in \mathbb{R}^{N \times N}$ is a diagonal matrix with idempotent and self-adjoint properties defined as $\Sigma_{ii} = 1$ if $i \in \mathcal{F}$ and 0 otherwise. The filter $\mathbf{\Sigma}$ is applied to a graph using graph convolution $\mathbf{U}\mathbf{\Sigma}\mathbf{U}^T$. To simplify notation and make use of the sparsity in the calculation, we define $\mathbf{U}_{\mathcal{F}}^T = \text{support}(\mathbf{\Sigma}\mathbf{U})^T$. We can see that by doing so, $\mathbf{U}$ is reduced to $\mathbf{U}_{\mathcal{F}}$: $\mathbf{U}$ is of size N by N and $\mathbf{U}_{\mathcal{F}}$ is of size N by $|\mathcal{F}|$. There is one property that we should pay close attention to:

$$\mathbf{U}\mathbf{\Sigma}\mathbf{U}^T = \mathbf{U}_{\mathcal{F}}\mathbf{U}_{\mathcal{F}}^T, \quad (3.5)$$

but

$$\mathbf{I} = \mathbf{U}_{\mathcal{F}}^T \mathbf{U}_{\mathcal{F}}. \quad (3.6)$$

The reason this property holds is that $\mathbf{U}_{\mathcal{F}}\mathbf{U}_{\mathcal{F}}^T$ is a convolution that applies a filter to the underlying signal. It essentially does three operations all at once: 1. forward GFT, 2. filtering the signal in the spectral domain, and 3. inverse GFT. Intuitively, this means that the calculation $\mathbf{U}_{\mathcal{F}}\mathbf{U}_{\mathcal{F}}^T$ is intended to make modifications to a signal that will be multiplied

by it. On the other hand, $\mathbf{U}_{\mathcal{F}}^T \mathbf{U}_{\mathcal{F}}$ only does an inverse GFT and then a forward GFT for a signal of size $|\mathcal{F}|$, which means that they are only conducting transformations without modifications. The bandlimited property of a graph signal $\mathbf{x}_0$ is that the signal only has components of frequencies $\mathcal{F}$, which establishes the relationship

$$\mathbf{x}_0 = \mathbf{U}_{\mathcal{F}} \mathbf{U}_{\mathcal{F}}^T \mathbf{x}_0 \quad (3.7)$$

as shown in^[37]. We can see that the bandlimited filter is a simple masking of the frequency components. Figure 3.1 gives an illustration of a bandlimited graph signal on a random sensor network with $N = 50$ nodes and $|\mathcal{F}| = 20$ frequencies.

A graph signal, when represented using only a few select sampled nodes, is notably sparse in the spatial domain. This sparse representation can be systematically obtained based on a designated sampling set $S \subseteq \mathcal{V}$ or randomly dropping signals. The design strategies are usually intended to maximize a selection of properties of the graph signal $\mathbf{x}$ by a reduction of the number of elements in the node signals from N to $|S| < N$ ^[37]. Once the sampling set is finalized, we can record it into a diagonal sampling or masking matrix $\mathbf{D}_S$. The i^{th} diagonal entries of $\mathbf{D}_S$ are equal to 1 when the signal on node v_i is sampled based on the records in the sampling set $S \subseteq \mathcal{V}$ and 0 when the signal on node v_i is not sampled. It is worth pointing out that the sampling matrix $\mathbf{D}_S$ has the properties of being idempotent and self-adjoint.

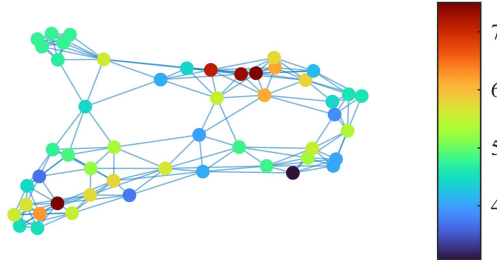


Figure 3.1 A random sensor graph and the graph signal.

Notice that both the graph Laplacian matrix $\mathbf{L}$ and the Adjacency matrix $\mathbf{A}$ are graph shift operators^[3]. Let us suppose we have a graph signal $\mathbf{x}$. To see the graph shift, let us use the graph Laplacian matrix as an example and look at the following calculation

$$\mathbf{L}\mathbf{x} = \text{vec} \left(\left. x_i - \sum_{j=0}^{N_j} e_{i,k} x_k \right|_{i=0 \dots N} \right), \quad (3.8)$$

where $\text{vec}()$ is the vectorize operator. We can see that for a node v_i , each of the signal components x_j on the neighborhood node v_j got shifted to node v_i . The difference be-

tween using the Laplacian matrix $\mathbf{L}$ and the adjacency matrix $\mathbf{A}$ is that the diagonal entries of the adjacency matrix are all zeros, meaning that the signals at the nodes themselves are not shifted. This graph-shifting operation may also be known as graph aggregation $\Omega^{[79]}$:

$$\text{agg}(v_i) = \Omega_{k=1\dots K}(z_i, m_{i,k}, x_k), \quad (3.9)$$

where x_k is the signal on the k^{th} neighboring node that is directly connected by an edge with a distance 1-hop away or the self-loop, and $m_{i,k}$ is the weight of the coefficient of the signal to be aggregated. For the calculation in (3.8), we can achieve it by setting $m_{i,k} = L_{i,k}$ and $m_{i,i} = L_{i,i}$. An illustration of the aggregation is shown in Figure 3.2.

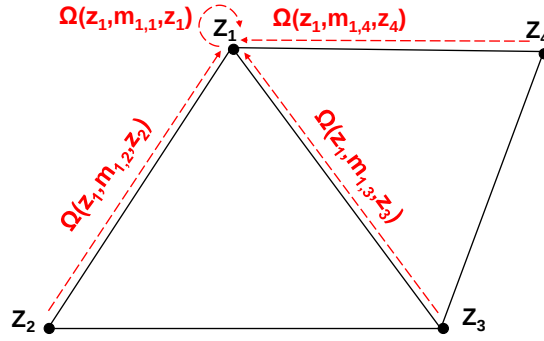


Figure 3.2 An illustration of message passing.

Most of the GSP algorithms have the assumption that the graph signals to be processed are smooth signals^[3]. To promote the smoothness of graph signals on a graph through convolution, one can attempt this by applying a low-pass filter in the spectral domain to the graph signal. The low-pass filter is a special case of the bandlimited filter. We can represent the low-pass filter as a function of $\mathbf{\Lambda}$; the ideal low-pass filter $h_l(\mathbf{\Lambda})$ has a cutoff frequency of λ_p . When $p < N$, it is defined as a diagonal matrix where the first p^{th} diagonal elements are 1, and the rest are 0. To conduct spectral graph filtering of graph signal $\mathbf{x}$, by the convolution property, filtering in the spectral domain can be done by multiplication:

$$\mathbf{x}' = \mathbf{U}h_l(\mathbf{\Lambda})\mathbf{U}^T\mathbf{x}. \quad (3.10)$$

However, direct implementation of the convolution shown in (3.10) requires the computation of the p smallest eigenvalues and eigenvectors. This is not to mention that when we are using a bandlimited filter instead of the ideal low-pass filter, the computation of all the eigenvalues and the eigenvectors is a necessity, making the computational complexity $O(n^3)$. Computing all the eigenvalues and the eigenvectors can become unstable as the size of the graph topology grows larger and larger, as analyzed in^[31]. Moreover,

each change in network topology requires a newly defined graph Laplacian to reflect even one additional node or edge, meaning that EVD will be repeated to formulate the Graph Fourier Transform (GFT) under the condition when a dynamic topology exists. Regrettably, this inherent limitation can potentially hinder the full-scale practical application of Graph Signal Processing (GSP) algorithms in certain scenarios. As a particular solution, one can effectively use a shifted Chebyshev polynomial to approximate the graph filters as shown in^[31], thereby circumventing the need for the time-consuming EVD. To begin, it can be shown that $\mathbf{U}h_l(\mathbf{A})\mathbf{U}^T = h_l(\mathbf{L})$ ^[31]. Then, to approximate $h_l(\mathbf{L})$ by a series of shifted Chebyshev polynomials, a Chebyshev polynomial $T_p(\mathbf{L})$ is defined over the interval $[0, \lambda_N]$:

$$T_p(\mathbf{L}) = \begin{cases} 1, & \text{if } p = 0 \\ \frac{2\mathbf{L} - \lambda_N}{\lambda_N}, & \text{if } p = 1 \\ \frac{4\mathbf{L} - 2\lambda_N}{\lambda_N} T_{p-1}(\mathbf{L}) - T_{p-2}(\mathbf{L}), & \text{if } p \geq 2, \end{cases} \quad (3.11)$$

where λ_N is the largest eigenvalue of $\mathbf{L}$. The approximation of the matrix $h_l(\mathbf{L})$ is formulated as

$$h_l(\mathbf{L}) \approx \theta_0 + \sum_{p=1}^P \theta_p T_p(\mathbf{L}) = \sum_{p=0}^P \hat{\theta}_p \mathbf{L}_0^p, \quad (3.12)$$

where P is the number of polynomials, θ_p and $\hat{\theta}_p$ are the weight of the p^{th} polynomial. We should point out that the calculation shown in (3.12) is a spatial method because it directly operates on the graph signal $\mathbf{x}$ using the spatial topological connections recorded in $\mathbf{L}$. Now, if we want to use the Chebyshev Polynomial approximation to approximate a low-pass filter, the low-pass filtering will aggregate the data from the 1-hop neighbors of each node in such a manner that the aggregation weights are determined to minimize the difference between the 1-hop neighbors, which is, in turn, a diffusion operation as mentioned in^[122]. In other words, both the spatial and spectral low-pass filtering can make the graph signals smoother. Similar concepts can be extended to bandlimited filters or high-pass filters by following the above approach.

3.1.2 Graph Signals With Multiple Features

The GSP discussed in the previous section is defined only on data with 1 feature, but data in the real world is often multidimensional and multi-featured. For example, in^[123] multiple sensors are placed at multiple locations of the human body to monitor various

vital signs and body motion, which could potentially be modeled using multi-feature GSP by treating the vital signals or the motions as the features and by constructing the graph topology using the location of the sensors. In^[124], a graph-theoretic model is given for a multi-channel EEG signal, but the multi-channel data is lost during the preprocessing phase by reducing the dimension to 1 using synchronization likelihood; the rest of the processing algorithm is done on the single-feature data. The GNLMS algorithm in^[36] is used to conduct online prediction of temperature using recordings from weather stations; it would be more beneficial if other data such as air quality, wind speed, precipitation, and humidity could be utilized simultaneously with the temperature. In^[113], air pollutant recordings in weather stations, including CO, NO₂, O₃, PM10, PM2.5, and SO₂, are monitored using a fusion of the attention mechanism and the GCN. In our recent work, we have discussed a formulation of the processing of graph signals with multiple features defined on a single graph topology^[10].

Let us consider a bandlimited graph signal with d features, where each feature is represented by an N -by-1 vector. The graph signal of interest $\mathbf{X}$ is a matrix of size N by d , with each column being one feature. An illustration of such a graph signal with $N = 197$ nodes and $d = 2$ features is shown in Fig. 3.3. The underlying graph topology is constructed using the 7-nearest neighbors approach of the geographic locations of 197 weather stations across the U.S., recorded for 95 hours by the weather stations^[125]. The multi-feature time-varying graph signal consists of two features, representing the temperature in Fig. 3.3(a) and the wind speed in Fig. 3.3(b) at the first hour of the dataset.

One can try to treat different features as separate graph signals and process them independently, which essentially is viewing a multi-feature graph signal as d separate graph signals defined over the d graph topologies that are identical. But in this way, the processing of d features will be done using d separate runs. We expanded the single-feature setting to process a multidimensional graph signal $\mathbf{X}$ to allow all d features to be processed at the same time.

When comparing to the single-feature case, we can see that the spectral domain filtering formulation is unchanged, as seen in (3.10):

$$\mathbf{X}' = \mathbf{U}h_l(\Lambda)\mathbf{U}^T\mathbf{X}. \quad (3.13)$$

This is because $\mathbf{U}_{\mathcal{F}}$ depends on the graph Laplacian, which is only related to the graph topology that is shared among all d features. Mathematically, $\mathbf{U}h_l(\Lambda)\mathbf{U}^T$ is of size N

by N and $\mathbf{X}$ is of size N_0 by d , meaning that we do not need to change the spectral formulation. As for the spatial method, we need to rearrange the weights θ in (3.12) in order to assign different weights for different features:

$$\mathbf{X}' = h_l(\mathbf{L})\mathbf{X} \approx \sum_{p=0}^P T_p(\mathbf{L})\mathbf{X}\boldsymbol{\theta}_p = \sum_{p=0}^P \mathbf{L}^p \mathbf{X} \hat{\boldsymbol{\theta}}_p, \quad (3.14)$$

where the weights $\boldsymbol{\theta}_p$ and $\hat{\boldsymbol{\theta}}_p$ are now d by d matrixes instead of scalars.

Using the above setting, all d features could be processed simultaneously instead of processing d one-dimensional graph signals separately. For simplicity, most of the derivations in later sections are conducted using $d = 1$ features, but keep in mind that for cases when $d > 1$, we can always switch the problem formulation using the approach discussed in this section.

3.2 Processing of the Signals on Simplicial Complexes

The spatial and spectral techniques are the basic building blocks of GSP algorithms. However, GSP only considered the case where the features reside on the graph nodes and neglected the possibility that signals can appear on higher-order structures such as graph edges. Here we list a few real-world examples in which GSP could not extract the full information within the data due to the fact that mapping the data onto the graph nodes does not completely represent the data. In the context of a citation complex, a collab-

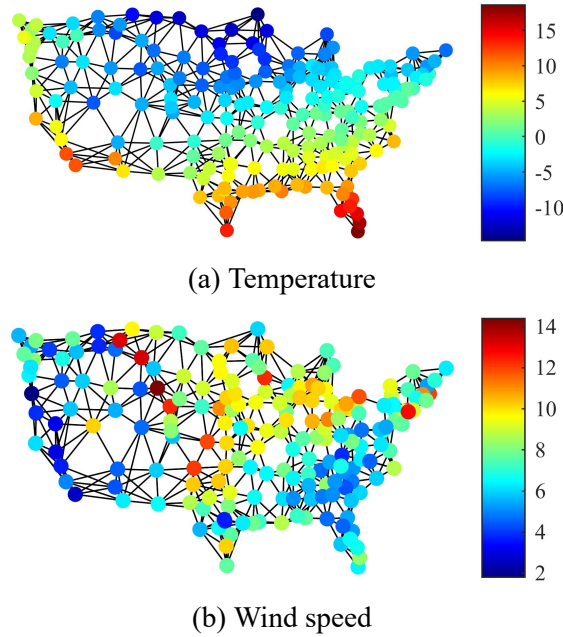


Figure 3.3 A single time instance of a multi-feature time-varying graph signal.

oration that involves three or possibly even more authors is accurately modeled using simplices^[84]; this particular example presents a challenge that cannot be merely captured using only the graph nodes or graph edges. The reason is that a conventional node-edge representation in standard GSP is inherently limited to representing a pairwise relationship between two authors, thus excluding more complicated relationship representations. The number of confirmed cases in a pandemic in multiple regions recorded on the graph nodes can be predicted using the additional information of the inter-regional population mobility recorded on the graph edges^[120]. The traffic flow on roads can be recorded on either the edges, as detailed in^[76], or on the nodes, as mentioned in^[117], of a given road network. A map of ocean drifter trajectories can be represented along the directed graph edges, as detailed in^[118].

Recently, researchers have proposed the concept of Topological Signal Processing (TSP) in the Signal Processing community^[5]. A more generalized Laplacian, called the Hodge Laplacian, is used to define spatial and spectral techniques in TSP instead of the graph Laplacian seen in GSP. High-dimensional topological signals, like signals on the graph edges and signals on the triangles, can be embedded into simplicial complexes using approaches similar to graph (node) embedding^[66]. To represent higher-order signals beyond just graph nodes, operations like sampling, spatial diffusion, spectral filtering, and convolution are defined on simplicial complexes in TSP^[5,71]. In TSP, topological

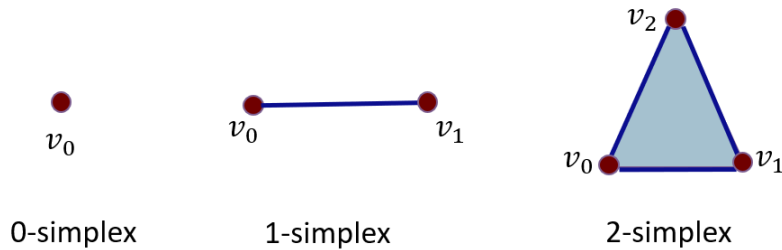


Figure 3.4 An illustration of the 0-simplex, the 1-simplex, and the 2-simplex.

structures such as graphs are considered as simplicial complexes defined by a vertex set $\mathcal{V} = \{v_0 \dots v_{N_0}\}$. A k -simplex $\mathcal{X}_k$ is defined as a subset of $\mathcal{V}$ with cardinality $k + 1$ that forms the simplest possible polytope. A collection of k -simplices $\{\mathcal{X}_{k,1} \dots \mathcal{X}_{k,N_k}\}$, with $k = 0 \dots K$ is a simplicial complex $\mathcal{K}$ of order K . An important property of the simplicial complex is that for a simplex $\mathcal{X}_k \in \mathcal{K}$, if another simplex of a lower order $\mathcal{X}_{k-1}$ is a subset of $\mathcal{X}_k$, then $\mathcal{X}_{k-1} \in \mathcal{K}$. The TSP graph representation of a graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ is a special case of simplicial complex: the node set $\mathcal{V}$ becomes a set of the 0-simplices and

the edge set $\mathcal{E} = \{e_1 \dots e_{N_1}\}$ becomes a set of the 1-simplices. A subscript k is used to denote which k -simplex each variable belongs to, so we can distinguish among different orders of simplices or simplicial complexes. The variable N_k will be used to record the cardinality of simplices in $\mathcal{K}$. For example, N_2 is the number of triangles, $N_1 = |\mathcal{E}|$ is the number of edges, and $N_0 = |\mathcal{V}|$ is the number of nodes.

The incidence matrix $\mathbf{B}_k \in \mathbb{R}^{N_{k-1} \times N_k}$ is used to record the relationship between two simplices of order k and $k-1$ in a simplicial complex. The columns of $\mathbf{B}_k$ correspond to the N_k elements in $\mathcal{X}_k$ and the rows of $\mathbf{B}_{k-1}$ correspond to the N_{k-1} elements in $\mathcal{X}_{k-1}$. Here is the values assignment logic of $\mathbf{B}_k$: if an element $x_{k-1}(i)$ in $\mathcal{X}_{k-1}$ is a neighboring element to $x_k(j)$ in $\mathcal{X}_k$, then 1 will be the magnitude of the ij^{th} entry of $\mathbf{B}_k$, with its sign set by the pre-established orientation in the simplicial complex. Here, the definition of neighboring is when two simplices are connected by their upper or lower simplex. For example, two nodes are neighboring nodes if they are connected by the same edge, and two edges are neighboring edges when they are connected by the same node or within the same triangle. We should emphasize that the incidence matrix $\mathbf{B}_k$ is a boundary operator due to the fact that $\mathbf{B}_k$ represents how a $k-1$ -simplex is bounded by its upper adjacent k -simplices. The dual of the boundary operator, which is a coboundary operator, is defined by the transpose of the incidence matrix $\mathbf{B}_k^T$.

A simplicial signal in a simplicial complex living on the k -simplices, denoted as $\mathbf{x}_k = [x_{k,0} \dots x_{k,N_k}]^T$, is a mapping from the k -simplex to $\mathbb{R}^{N_k}$. The GFT analogy in TSP is the Simplicial Fourier Transform (SFT). Instead of using the graph Laplacian matrix as seen in GSP, the Hodge Laplacian matrix plays the role of the graph Laplacian matrix. Let us make the analogy of GSP spectral operations on the node signal $\mathbf{x}_0$ in TSP. The Hodge Laplacians can be defined for a simplicial complex with order K as follows^[5]:

$$\mathbf{L}_k = \begin{cases} \mathbf{B}_1 \mathbf{B}_1^T, & k = 0, \\ \mathbf{B}_k^T \mathbf{B}_k + \mathbf{B}_{k+1} \mathbf{B}_{k+1}^T = \mathbf{L}_{k,l} + \mathbf{L}_{k,u}, & 0 < k < K, \\ \mathbf{B}_K^T \mathbf{B}_K, & k = K. \end{cases} \quad (3.15)$$

In (3.15), we can see that the Hodge Laplacian $\mathbf{L}_k$ is composed of two parts: the upper Hodge Laplacian $\mathbf{L}_{k,u} = \mathbf{B}_{k+1} \mathbf{B}_{k+1}^T$ and the lower Hodge Laplacian $\mathbf{L}_{k,l} = \mathbf{B}_k^T \mathbf{B}_k$. For a k -simplex $\mathcal{X}_k$ in the simplicial complex, this definition provides the neighborhood relationship from the upper-adjacency and the lower-adjacency of $\mathcal{X}_k$, respectively^[71]. It deserves emphasis that for $k = 0$, there is no lower adjacency and $\mathbf{L}_0$ is the graph Laplacian.

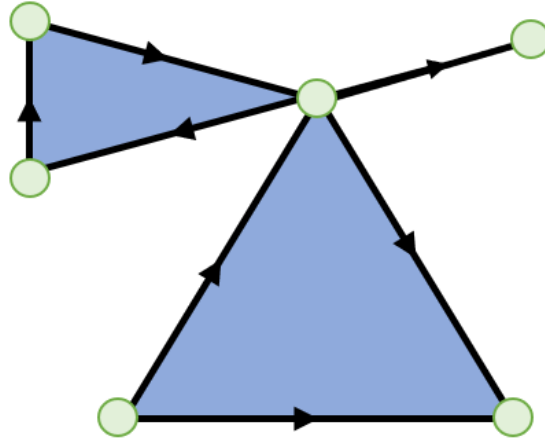


Figure 3.5 An illustration of a simplicial complex that consists of signals on nodes (green), edges (black), and triangles (blue).

The SFT is based on the EVD of the Hodge Laplacian matrix^[5]:

$$\mathbf{L}_k = \mathbf{U}_k \mathbf{\Lambda}_k \mathbf{U}_k^T. \quad (3.16)$$

The eigenvalue matrix is denoted by $\mathbf{\Lambda}_k = \text{diag}(\lambda_{k,1} \dots \lambda_{k,N_k})$. Here, the eigenvalues $\lambda_{k,i}$ are sorted in ascending order and are viewed as the frequencies in SFT. Smaller eigenvalues are seen as lower frequencies, while higher eigenvalues represent higher frequencies. Each eigenvalue in $\mathbf{\Lambda}_k$ has a corresponding eigenvector in the eigenvector matrix $\mathbf{U}_k = [\mathbf{u}_{k,1} \dots \mathbf{u}_{k,N_k}]$. Similar to GFT, the forward SFT of a spatial domain signal $\mathbf{x}_k$ transforms it from the spatial domain signal to a frequency domain signal $\mathbf{s}_k$:

$$\mathbf{s}_k = \mathbf{U}_k^T \mathbf{x}_k. \quad (3.17)$$

Naturally, the inverse SFT is the transform that transforms the frequency domain signal back to the spatial domain:

$$\mathbf{x}_k = \mathbf{U}_k \mathbf{s}_k. \quad (3.18)$$

Bear in mind that the GFT is a special case of the SFT, but the GFT is intended for signals defined only on the nodes^[5]. Since the graph Laplacian is the Hodge Laplacian when $k = 0$, there is no difference between TSP and GSP techniques for $k = 0$. The Hodge Laplacian when $k = 1$ is known as the graph Helmholtzian^[126] and could be used to define techniques to process the signals living on the graph edges.

With the establishment of SFT, we can introduce simplicial filters $\mathbf{\Sigma}$ to every frequency element in $\mathbf{L}_k$, enabling spectral domain tasks on simplicial complexes. Similar to the GSP case, in TSP a bandlimited filter in $\mathcal{X}_k$ is constructed by a diagonal matrix

$\Sigma_{\mathcal{F},k}$ of frequency set $\mathcal{F}_k$. An 1 on the i^{th} diagonal element of $\Sigma_{\mathcal{F},k}$ is an indication that $\lambda_i \in \mathcal{F}_k$ and 0 is an indication that $\lambda_i \notin \mathcal{F}_k$ [5]. A simple yet complete bandlimited TSP spectral filtering procedure of the simplicial signal $\mathbf{x}_k$ is

$$\mathbf{x}'_k = \mathbf{U}_k \Sigma_{\mathcal{F},k} \mathbf{x}_k = \mathbf{U}_k \Sigma_{\mathcal{F},k} \mathbf{U}_k^T \mathbf{x}_k. \quad (3.19)$$

If provided with the fact that the signal $\mathbf{x}_k$ is bandlimited with frequencies $\mathcal{F}_k$, then

$$\mathbf{x}_k = \mathbf{U}_k \Sigma_{\mathcal{F},k} \mathbf{U}_k^T \mathbf{x}_k. \quad (3.20)$$

It is significant to highlight that several TSP filters can be defined at once and then linearly combined. Again, to simplify notation and to promote sparsity, we will set $\mathbf{U}_{k,\mathcal{F}} = \text{support}(\mathbf{U}_k \Sigma_{\mathcal{F},k})$. Again, the filter $\Sigma_{\mathcal{F},k}$ is self-adjoint and idempotent, so in the simplified notation we have the following two properties:

$$\mathbf{U}_{k,\mathcal{F}} \mathbf{U}_{k,\mathcal{F}}^T = \mathbf{U}_k \Sigma_{\mathcal{F},k} \mathbf{U}_k^T, \quad (3.21)$$

and

$$\mathbf{U}_{k,\mathcal{F}}^T \mathbf{U}_{k,\mathcal{F}} = \mathbf{I}. \quad (3.22)$$

In case only partial observation of $\mathbf{x}_k$ is made available, we can use the sampling operation on the signal similar to the GSP situation. The sampling matrix is a diagonal matrix $\mathbf{D}_{k,S} = \text{diag}(d_{k,1} \dots d_{k,N_k})$, where $d_{k,i} = 1$ if $x_{k,i} \in S_k$ and 0 otherwise is defined from a sampling set S_k that resulted in a masking operation [5]. The sampling matrix $\mathbf{D}_{k,S}$ is idempotent and self-adjoint as well.

Here we will provide a necessary discussion on how to reconstruct a simplicial signal $\mathbf{x}_k$ when only partial observation of the signal is given. With the assumption that $\mathbf{x}_k$ is a bandlimited signal in frequencies $\mathcal{F}_k$, a reconstruction of $\mathbf{x}_k$ is possible if the below conditions are met

$$\|\bar{\mathbf{D}}_{k,S} \mathbf{U}_{k,\mathcal{F}} \mathbf{U}_{k,\mathcal{F}}^T\|_2 = \|\mathbf{U}_{k,\mathcal{F}} \mathbf{U}_{k,\mathcal{F}}^T \bar{\mathbf{D}}_{k,S}\|_2 < 1 \quad (3.23)$$

and

$$|\mathcal{F}_k| \leq |S_k|, \quad (3.24)$$

where $\bar{\mathbf{D}}_{k,S} = \mathbf{I} - \mathbf{D}_{k,S}$. As a result, the spectral reconstruction approach based on the above condition in [5] is established:

$$\mathbf{x}_k = \mathbf{H}_k \mathbf{D}_{k,S} \mathbf{x}_k, \quad (3.25)$$

where $\mathbf{H}_k = \mathbf{U}_{k,\mathcal{F}} (\mathbf{U}_{k,\mathcal{F}}^T \mathbf{D}_{k,S} \mathbf{U}_{k,\mathcal{F}})^{-1} \mathbf{U}_{k,\mathcal{F}}^T$.

Similar to the GSP case, there is a limitation of using spectral TSP where the simplicial filter Σ_{k-1} is defined by the frequency set $\mathcal{F}_{k-1}$ operating on the frequencies (eigenvalues) of regression matrix $\hat{\mathbf{L}}_{k-1}[t]$. In other words, the spectral filtering operation in TSP requires the EVD to form the matrix $\mathbf{H}_{k-1}[t]$ as well. In GSP, the EVD of the graph Laplacian suffers from high computational cost and numerical instabilities when the graph topology is large^[31]. The same limitation can be found in TSP for the Hodge Laplacian^[72]. The solution to this problem is again to approximate the spectral filters with polynomial filters defined directly based on $\mathbf{L}_k$ to avoid EVD. In TSP, similar approximation approaches can be seen in^[72] for denoising filters on static simplicial signals and for defining simplicial convolution filters^[71].

Assuming that the spatial operations are conducted in the k -simplices of the simplicial complex $\mathcal{K}$, using the Hodge Laplacian this time, the shifted Chebyshev polynomial $T_p(\mathbf{L}_k)$ is defined as

$$T_p(\mathbf{L}_k) = \begin{cases} 1, & \text{if } p = 0 \\ \frac{2\mathbf{L}_k - \lambda_{\max}}{\lambda_{\max}}, & \text{if } p = 1 \\ \frac{4\mathbf{L}_k - 2\lambda_{\max}}{\lambda_{\max}} T_{p-1}(\mathbf{L}_k) - T_{p-2}(\mathbf{L}_k), & \text{if } p \geq 2. \end{cases} \quad (3.26)$$

The approximation $\hat{\mathbf{H}}_k$ of the matrix $\mathbf{H}_k$ is

$$\mathbf{H}_k = h(\mathbf{L}_k) \approx \hat{\mathbf{H}}_k = \frac{\theta_0}{2} + \sum_{p=1}^P \theta_p T_p(\mathbf{L}_k). \quad (3.27)$$

Here in (3.27), the parameters θ_p are the coefficients of the Chebyshev polynomial, and P is the order of the polynomial. If we explicitly write out the (3.27) and rearrange the terms, we see that (3.27) becomes a spatial TSP convolution (or Simplicial Convolution^[85]) operator using $\mathbf{L}_k$:

$$\hat{\mathbf{H}}_k = \sum_{p=0}^P \hat{\theta}_p \mathbf{L}_k^p, \quad (3.28)$$

where $\hat{\theta}_p$ is the polynomial coefficient.

Now, remember that in (3.15), the Hodge Laplacian can be split into upper and lower Hodge Laplacians; this means that the Simplicial Convolution can be split as well. To do so, we can simply assign specific filter weights to the lower and upper Hodge Laplacians.

This leads to the simplicial convolution operation^[85]:

$$\mathbf{x}'_k = \sum_{j=1}^{J_l} \gamma_{k,j} \mathbf{L}_{k,l}^j \mathbf{x}_k + \sum_{j=1}^{J_u} \theta_{k,j} \mathbf{L}_{k,u}^j \mathbf{x}_k + \xi_k \mathbf{I} \mathbf{x}_k, \quad (3.29)$$

where $\gamma_{k,j}$ lower Hodge Laplacian and $\theta_{k,j}$ are the weights for the upper Hodge Laplacian. A term $\xi_k \mathbf{I} \mathbf{x}_k$ similar to the self-loop in GCN, which is added to the simplicial convolution to process the signal components that will not be directly modified by the convolution defined by the upper or lower Hodge Laplacian.

3.3 Graph and Simplicial Neural Networks

In GCN, when omitting the activation function, the graph convolution is an aggregation of graph features (signals) shown in (3.13) and (3.14), corresponding to the spectral and spatial approaches, respectively^[18-19]. Notice that self-aggregation can be achieved in (3.14) for $p = 0$. Adding the activation function σ to the spectral convolution (3.13), the l^{th} layer of spectral GCN with f filters is

$$\mathbf{X}_{l+1} = \mathbf{Z}_l = \sigma \left(\mathbf{U} \sum_{i=1}^f \boldsymbol{\Theta}_{i,l} \mathbf{U}^T \mathbf{X}_l \right). \quad (3.30)$$

In the spectral GCN, the goal is to obtain a convolution that is formed by filters localized in the spectral domain^[21]. In other words, the objective of the GCN is to train the parameters $\boldsymbol{\Theta}$ so that it eventually will resemble the structure of the underlying frequency spectrum of the ground truth data $\mathbf{X}_g$. In GSP terms, a L layer GCN trains a sequence of filters that satisfies $\mathbf{X}_g = \mathbf{Z}_L(\dots \mathbf{Z}_0(\mathbf{X}_0))$. Similarly, we can define a spatial GCN by

$$\mathbf{X}_{l+1} = \mathbf{Z}_l = \sigma \left(\sum_{p=0}^P T_p(\mathbf{L}) \mathbf{X}_l \boldsymbol{\Theta}_p = \sum_{p=0}^P \mathbf{L}^p \mathbf{X}_l \hat{\boldsymbol{\Theta}}_p \right), \quad (3.31)$$

with $\hat{\boldsymbol{\Theta}}_p$ being the trainable weights.

Using the TSP techniques seen in Section 3.2 and the same derivation logic as GCN, we can formulate the multivariate Simplicial Convolutional Neural Networks (SCNN) when there are d features or when there are d filters at layer p , which was not given in the SCNN manuscript^[85]:

$$\mathbf{Z}_{k,p+1} = \sigma \left(\sum_{j=1}^{J_l} \mathbf{L}_{k,l}^j \mathbf{Z}_{k,p} \boldsymbol{\Gamma}_{k,j} + \sum_{j=1}^{J_u} \mathbf{L}_{k,u}^j \mathbf{Z}_{k,p} \boldsymbol{\Theta}_{k,j} + \boldsymbol{\Xi}_k \mathbf{Z}_{k,p} \right). \quad (3.32)$$

In (3.32), the variables $\boldsymbol{\Gamma}_{k,j}$, $\boldsymbol{\Theta}_{k,j}$, and $\boldsymbol{\Xi}_k$ are the trainable neutral network weights. The

main difference between GCN and SCNN is in the following two aspects. First, the GCN is only defined on the nodes, while SCNN is defined on a more generalized structure: a simplicial complex. This makes the SCNN having a broader application scope and more expressive power^[127]. Second, the SCNN uses upper adjacency and lower adjacency by splitting the Hodge Laplacian matrix $\mathbf{L}_k$ into $\mathbf{L}_{k,l}$ and $\mathbf{L}_{k,u}$, while GCN only considers node adjacency. This can be reflected in (3.32) in the usage of upper and lower Hodge Laplacian matrices to conduct aggregation.

3.4 Discussion

This chapter went over the preliminary knowledge of the GSP and TSP techniques with a focus on time-invariant approaches. We delve into the derivations of spectral and spatial techniques on the graph nodes and extend beyond the graph nodes through TSP. The extension of these GSP and TSP techniques to deep learning is also being examined. In the next chapter, we will be demonstrating all the time-invariant concepts using our recently proposed Binarized Simplicial Neural Network (Bi-SCNN). The Bi-SCNN proposed by us is an extension of the SCNN, but the Bi-SCNN uses feature normalization and feature binarization, which the SCNN does not^[128].

CHAPTER 4 TIME-INVARIANT TOPOLOGICAL SIGNAL PROCESSING CASE STUDY

Let us now examine how to use the time-invariant Topological Signal Processing (TSP) techniques shown in the previous chapter in a case study. The content in this chapter is based on one of our recently completed works named the Binarized Simplicial Convolutional Neural Networks (Bi-SCNN)^[128]. The Bi-SCNN enhances the time efficiency of simplicial convolution while maintaining the effectiveness of its simplicial embedding.

4.1 Bi-SCNN Derivation

One of the many solutions to the slow training speed of neural networks is using binarization, as seen in the Binarized Graph Neural Networks (Bi-GCN)^[89], where both the trainable weights and the features are binarized, and the forward propagation is done by binary operations, bit count and XOR, to combine the binarized weights and features into a new node embedding. Another approach seen in the Graph-Sign algorithms^[9] delivers a sign-error update at each iteration that has a low runtime and a low complexity. Aside from the fact that these node-based algorithms are not suitable to be deployed on simplicial complexes, there are a few additional problems to be addressed. The Bi-GCN binarizes both the weights and features, leading to reduced performance under certain circumstances. Also, even though Bi-GCN uses binary operations to optimize the computation cost, the forward propagation is indifferent and mathematically identical compared with the original GCN. The Graph-Sign algorithm operates on a predefined bandlimited filter, which may require prior expert knowledge to obtain.

We propose a new forward propagation strategy where we combine feature normalization, feature binarization, and binary-sign propagation to form a fast and accurate Binarized Simplicial Convolution. Following the graph-based feature binarization seen in

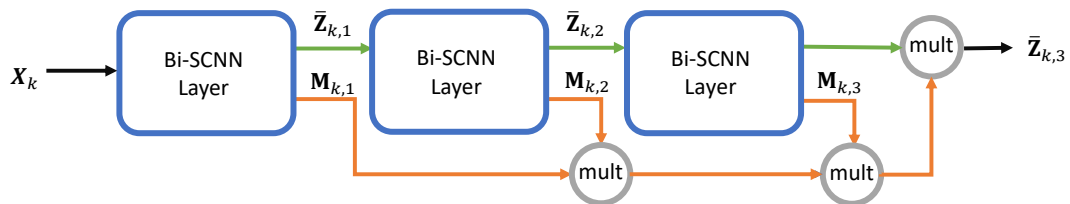


Figure 4.1 An illustration of a 3-layer Bi-SCNN network.

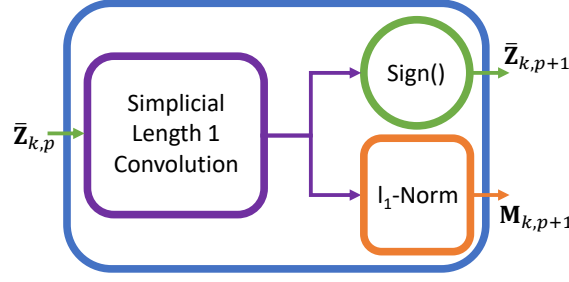


Figure 4.2 An illustration of one Bi-SCNN layer.

Bi-GCN^[89], we define the feature binarization of simplicial data $\mathbf{X}_k$ on k -simplex S_k as

$$\bar{\mathbf{X}}_k \approx \mathbf{M}_k \circ \text{Sign}(\mathbf{X}_k), \quad (4.1)$$

where $\circ$ is the Hadamard product, $\text{Sign}()$ is the Sign function and $\bar{\mathbf{X}}_k$ is the binarized simplicial features. The matrix $\mathbf{M}_k$ is formed by feature normalization, and all elements of the same row have the same value

$$m_i = \frac{\text{norm}_1(\mathbf{r}_k)}{d_k}, \quad (4.2)$$

where $\text{norm}_1(\mathbf{r}_k)$ is the l_1 -norm done on the i^{th} row of $\mathbf{X}_k$ ($\mathbf{r}_k$ in (4.2)) and d_k is the number of features in S_k . Since all elements in a single row have the same value m_i , in practice $\mathbf{M}_k$ is stored as a vector $\mathbf{m}_k$. Notice that the $\text{Sign}(\cdot)$ function only outputs binary values -1 and $+1$, meaning that the size of the binarized features $\bar{\mathbf{X}}_k$ is reduced in practice when compared to $\mathbf{X}_k$ ^[89].

Now, plugging the feature binarization (4.1) into the SCNN update in (3.32) and setting $J = 1$, we will get

$$\bar{\mathbf{Z}}_{k,p+1} = \sigma(\mathbf{L}_{k,l}\bar{\mathbf{Z}}_{k,p}\mathbf{\Gamma}_k + \mathbf{L}_{k,u}\bar{\mathbf{Z}}_{k,p}\mathbf{\Theta}_k + \bar{\mathbf{Z}}_{k,p}\mathbf{\Xi}_k), \quad (4.3)$$

where $\mathbf{\Gamma}_{k,j}$, $\mathbf{\Theta}_{k,j}$, and $\mathbf{\Xi}_k$ are again the trainable neural network weights similar to the SCNN in (3.32), Chapter 3. Looking at the expression in (4.3), aside from benefiting from the model size reduction, it still follows the same forward propagation strategy as SCNN, making it have roughly the same execution time in practice. We would like to further improve the run speed of this Binary setup by propagating only the $\text{Sign}(\cdot)$ part. Using the two properties of Hadamard products $\mathbf{A} \circ \mathbf{B} = \mathbf{B} \circ \mathbf{A}$ and $\mathbf{A} \circ (\mathbf{B} + \mathbf{C}) = \mathbf{A} \circ \mathbf{B} + \mathbf{A} \circ \mathbf{C}$, with some algebraic derivations, equation (4.3) becomes

$$\bar{\mathbf{Z}}_{k,p+1} = \mathbf{M}_{k,p} \circ (\mathbf{L}_{k,l}\text{Sign}(\bar{\mathbf{Z}}_{k,p})\mathbf{\Gamma}_k + \mathbf{L}_{k,u}\text{Sign}(\bar{\mathbf{Z}}_{k,p})\mathbf{\Theta}_k + \text{Sign}(\bar{\mathbf{Z}}_{k,p})\mathbf{\Xi}_k). \quad (4.4)$$

Notice that the l_1 -norm is nonnegative, so at each layer, $\mathbf{M}_k^p$ can be viewed as $\mathbf{M}_k^p = \text{ReLU}(\mathbf{M}_k^p)$. If the input of the $p+1^{\text{th}}$ layer in (4.4) will binarize the output of the p^{th} layer

regardless of the magnitude, we can then input only the binarized simplicial features to the $p+1^{th}$ layer when $p > 1$. In this way, the nonlinearity in both the $\text{Sign}(\cdot)$ function and the l_1 -norm can serve as activations in forward propagation. The forward propagation of a series of P concatenated Bi-SCNN is obtained by rearranging (4.4) and using the property $\mathbf{A} \circ \mathbf{B} = \mathbf{B} \circ \mathbf{A}$ again:

$$\bar{\mathbf{Z}}_{k,p} = \mathbf{M}_{k,p} \circ \mathbf{M}_{k,p-1} \circ \dots \circ \mathbf{M}_{k,1} \circ \mathbf{Q}_{k,p-1}, \quad (4.5)$$

where for the p^{th} layer

$$\mathbf{Q}_{k,p} = \text{Sign}(\mathbf{L}_{k,l} \bar{\mathbf{Z}}_{k,p-1} \mathbf{\Gamma}_k + \mathbf{L}_{k,u} \bar{\mathbf{Z}}_{k,p-1} \mathbf{\Theta}_k + \bar{\mathbf{Z}}_{k,p-1} \mathbf{\Xi}_k). \quad (4.6)$$

Even though the $\text{Sign}(\cdot)$ function has no derivative at 0 due to its discontinuity at 0 and a derivative of 0 for all other values, we approximate $\text{Sign}(\cdot)$ using a hard tanh function, allowing us to conduct forward propagation and backward propagation as usual. The way to achieve the forward propagation in (4.5) in practice is that each Bi-SCNN layer will have two outputs: the feature normalization $\mathbf{M}_{k,p}$ and feature binarization $\mathbf{Q}_{k,p}$. Notice that only $\mathbf{Q}_{k,p}$ will be forward-propagated to the next Bi-SCNN layer, meaning that $\mathbf{Q}_{k,p} = \mathbf{Z}_{k,p}$. Exceptions are the first layer, $\mathbf{Z}_{k,0} = \mathbf{X}_k$, which is the input, and the final layer all the $\mathbf{M}_{k,p}$ are multiplied together at the end to form $\bar{\mathbf{Z}}_{k,p}$. An illustration of a Bi-SCNN network with $P = 3$ is shown in Fig. 4.1. Intuitively, a single Bi-SCNN layer on a k -simplex can be understood as using the (upper and lower) Hodge Laplacians to aggregate the weighted simplicial features with the addition of the harmonic component. From the spatial TSP perspective, the weights are trained to optimize a binary-sign feature aggregation with a matrix that records the magnitude of each aggregation. From the spectral TSP perspective, the goal of training a Bi-SCNN is to obtain a filter that best fits the spectrum of the training data for the binary sign update. An illustration of one Bi-SCNN layer can be found in Fig. 4.2.

Aside from the fact that all algorithms benefit from the sparsity of $\mathbf{L}_k$, let us look into (4.5) to see the high time efficiency of Bi-SCNN. In practice, $\text{Sign}(\cdot)$ is approximated by $\text{Hardtanh}()$, meaning that a percentage r_k of the matrix $\bar{\mathbf{Z}}_{k,p}$ consists only of 1s and -1s. In this case, r_k element-wise multiplications can be omitted in the matrix multiplication, reducing $N_k^2 d_k$ multiplications to $(1 - r_k) N_k^2 d_k$ multiplications. Assuming each multiplication is 1 FLOP, each Bi-SCNN layer is at least r_k FLOPS faster than the SCNN layer for the 2^{nd} layer and beyond. Also, in the actual implementation of SCNN, to compute $\mathbf{Z}_{k,p}$, $\mathbf{L}_{k,l}$, $\mathbf{L}_{k,u}$, and $\mathbf{I}_k$ are concatenated into a $3N_k \times N_k$ matrix for each $j = 0 \dots J$; then

J matrices are stacked into a $d_p \times N_k \times J$ tensor. Trainable weights Θ_k , Γ_k , and Ξ_k are stacked into a $d_{k,p} \times d_{k,p+1} \times J$ tensor in a similar manner. Afterward, $\mathbf{Z}_{k,p+1}$ is computed by the Einstein summation between two tensors (aggregation and weight tensors) of size $d_p \times N_k \times J$ and $d_{k,p} \times d_{k,p+1} \times J$ back into a matrix of size $N_k \times d_{p+1}$. This matrix concatenation, stacking, and Einstein summation approach is less time-efficient than direct matrix multiplication, especially when $J > 1$. We should emphasize again that $\mathbf{M}_{k,p}$ is a broadcasted vector in practice, making the Bi-SCNN more efficient.

Remarks The binary-sign forward propagation strategy in the Bi-SCNN layer has 2 outputs, but the SCNN layer only has 1 output, making a neural network that contains more than 2 layers of Bi-SCNN a drastically different architecture than using purely SCNN. The Bi-GCN^[89] has an acronym similar to the Bi-SCNN, but we should emphasize that the Bi-SCNN is not simply a simplicial complex version of the Bi-GCN, and the two are drastically different from each other with the following four differences. Firstly, the Bi-GCN operates only on the graph nodes, while the Bi-SCNN operates on k -simplices. Secondly, as explained earlier, the Bi-SCNN does not follow the traditional GCN/SCNN forward propagation strategy because the feature binarization and the feature normalization propagate separately. Thirdly, the Bi-GCN binarizes both the features and the trainable weights, but Bi-SCNN only binarizes the features. Fourthly, the Bi-GCN uses binary operations, bit count and XOR, to speed up the same GCN forward propagation and uses predefined values to deal with the backpropagation of binary operations; the Bi-SCNN uses a binary-sign forward propagation, and we do not make any predefinitions for backpropagation.

4.2 Experiment Setting and Dataset Information

The Citation Complex^[84] and the Ocean Drifter Complex^[118] are chosen because both datasets are already preprocessed to have features defined in simplexes with order $k \geq 1$, making them suitable choices to test the performance of an algorithm on simplicial complexes. We would like to verify that the Bi-SCNN has a faster execution time while maintaining good accuracy when compared against existing simplicial algorithms. The experiments are conducted on one Nvidia RTX 3090 GPU with PyTorch version 1.8.0 and CUDA version 11.1. None of the two datasets provided a validation set, so the experiments are tuned based on the training performance. All the experiments were repeated 10 times; we calculated the mean and standard deviation of the metrics.

4.3 Estimation of Missing Simplicial Signals in Citation Complex

The raw data of the Citation Complex are from Semantic Scholar Open Research Corpus^[129], where the number of citations of $k + 1$ authors forms a k -simplex where $k = 0 \dots 5$. The 6 simplices naturally form a simplicial complex because they form a hierarchical structure: each k -simplex is a subset of author collaborations of the $k + 1$ -simplex. To examine the performance of our Bi-SCNN algorithm, we will follow the training and testing procedure seen in SNN^[84] and SCNN^[85]. For each k -simplex, a portion of the simplicial data will be missing at random. In this transductive learning task, the goal is to impute the missing data from the existing data. We will be testing 5 different missing rates: 10%, 20%, 30%, 40%, and 50%.

A 2-layer Bi-SCNN (Bi-SCNN-2) and a 3-layer Bi-SCNN (Bi-SCNN-3) will be compared against a 2-layer SNN (SNN-2), a 2-layer SCNN (SCNN-2), and a 3-layer SCNN (SCNN-3). The number of simplicial filters per layer is set to 30 for all algorithms. Each layer in SNN-2 and SCNN-2 will have a length $J = 1$ to keep the model size roughly the same order of magnitude as the Bi-SCNN-2. SCNN-3 will have a filter length $J = 2$, which is the best-performing setting seen in the original SCNN paper^[85]. We follow the parameter optimization seen in SCNN^[85], where all algorithms use the Adam optimizer with a learning rate of 0.001 and minimize the L1-loss on the training set for 1000 iterations. To make the comparison fair, all the tested algorithms use purely Simplicial Layers except for the activation functions. The accuracy of the imputed missing data will be considered correct if it is within $\pm 1\%$ of the ground truth value. The experiment run time is recorded as the cumulative run time of all the forward propagation and backward propagation of each experiment run. The accuracy of each algorithm setting is shown in Table 4.1. The run time of the experiments is recorded in Table 4.2. The number of trainable parameters of each algorithm can be found in Table 4.5. We included an additional visual comparison of the run time of the Bi-SCNN when compared against the baselines for the 10% missing scenario in Figure 4.3.

From Table 4.1, we can see that the 2-layer Bi-SCNN performs the best in most of the simplices under all the tested missing rates. This confirms that even though the Bi-SCNN uses a simpler propagation scheme by only having a length-1 simplicial filter, the embedding power is equivalent to that of the SNN or SCNN in the current experiment. Another interesting observation is that the 2-layer models perform slightly better than the 3-layer models in many cases. We think this is due to the fact that the 3-layer models

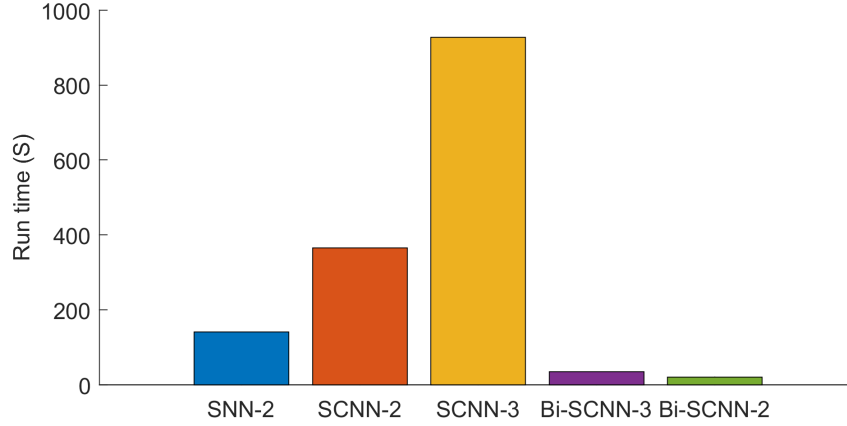


Figure 4.3 The run time of the Bi-SCNN when compared against the baselines for 10% missing rate.

have too many simplicial convolutional filters and the simplicial embedding got "over-smoothed" during the propagation process. If we look at the ratio of parameter size to runtime, we can see that the Bi-SCNN is the most efficient among all the tested algorithms. Bi-SCNN-2 has 20% more parameters than SNN-2 but takes only 1/5 the time that SNN needs to complete the experiment. We should point out that for roughly similar model sizes, the Bi-SCNN has higher accuracy in imputing the citations compared to the SNN because Bi-SCNN uses the Hodge decomposition to consider upper and lower simplicial adjacencies separately, but SNN does not, making the Bi-SCNN more expressive. The Bi-SCNN is indeed a more concise model than the SNN and the SCNN because not only does Bi-SCNN use fewer parameters but also operates faster. The reason that the Bi-SCNN is time-efficient is that the binary-sign propagation takes fewer computations, which is proven in traditional CNNs^[130] and has also been proven when data is purely on the nodes of the graph by the G-Sign^[9] and Bi-GCN^[89]. We can conclude that the Bi-SCNN can effectively and efficiently impute the missing citations compared to the SCNN and Simplicial Attention Networks (SAT).

4.4 Classification of Ocean Drifter on Simplicial Complex

The Ocean Drifter Complex is a dataset formed by using simplicial embedding to represent real-world ocean drifters around Madagascar Island^[118]. The ocean that surrounds Madagascar Island is divided into hexagonal grids, and each grid is represented by a node in the 0-simplex. If the trajectory of an ocean drifter passes through two adjacent hexagonal grids, an edge will be formed between the nodes. In the dataset, there are two types of trajectories, and the goal is to classify which type of trajectory the in-

Table 4.1 Experiment accuracy on citation complex from 10% to 50% data missing rate.

	SNN-2	SCNN-2	SCNN-3	Bi-SCNN-3	Bi-SCNN-2
10% missing					
S_0	90.60±03.47	90.51±0.34	90.57±0.38	84.52±17.95	90.65±0.32
S_1	91.00±02.45	91.02±0.25	90.52±1.43	90.77±0.24	91.03±0.24
S_2	90.94±05.59	90.60±0.67	91.22±0.16	87.81±01.91	91.22±0.16
S_3	91.45±03828	91.37±0.14	91.58±0.12	89.74±01.65	91.58±0.12
S_4	87.89±11.59	91.65±0.56	91.88±0.23	85.04±16.80	91.92±0.20
S_5	83.61±18.65	92.07±0.33	82.96±27.64	89.44±0.49	92.21±0.23
20% missing					
S_0	81.22±0.63	81.36±0.64	81.14±0.63	81.08±0.61	81.39±0.66
S_1	82.17±0.32	82.16±0.32	69.76±26.24	82.07±0.25	82.20±0.31
S_2	82.42±0.49	82.26±0.51	74.42±24.81	80.53±0.58	82.68±0.27
S_3	83.11±0.38	82.97±0.26	83.23±0.15	81.81±1.28	83.23±0.15
S_4	75.63±23.65	83.47±0.40	81.84±5.39	82.79±0.45	83.64±0.19
S_5	73.03±25.38	84.22±0.27	75.88±25.30	66.45±2.69	84.34±0.23
30% missing					
S_0	72.16±0.66	72.27±0.57	71.82±1.28	72.05±0.61	72.33±0.50
S_1	73.15±0.42	73.13±0.44	72.87±0.52	72.64±0.79	73.98±0.41
S_2	73.73±0.67	73.46±0.51	66.70±21.96	72.20±0.32	73.98±0.26
S_3	74.70±0.30	74.52±0.21	67.29±22.43	72.44±0.40	74.78±0.16
S_4	68.31±21.98	75.56±0.26	68.05±22.69	74.73±08.27	75.68±0.20
S_5	72.09±10.87	76.52±0.31	76.63±0.24	65.00±18.89	76.63±0.23
40% missing					
S_0	62.36±0.72	62.78±0.65	62.07±1.83	62.59±0.61	62.81±0.65
S_1	64.10±0.28	64.10±0.28	63.74±0.91	63.57±0.79	64.10±0.28
S_2	64.73±0.79	64.54±0.63	65.52±0.20	63.86±0.32	65.17±0.20
S_3	66.26±0.28	66.07±0.33	66.31±0.21	65.40±0.40	66.31±0.21
S_4	66.89±1.66	67.33±0.39	50.26±27.14	66.85±0.83	67.57±0.23
S_5	61.98±14.03	68.86±0.18	62.21±20.31	53.91±18.89	68.98±0.18
50% missing					
S_0	54.01±0.5457	54.03±0.06	53.98±0.58	50.28±5.35	54.18±0.52
S_1	55.05±0.3947	55.08±0.04	54.33±1.37	54.21±0.94	55.81±0.40
S_2	55.32±2.1300	55.78±0.06	56.27±0.24	52.95±4.04	56.27±0.24
S_3	57.81±0.3062	57.33±0.03	57.82±0.28	57.19±0.37	57.85±0.25
S_4	57.64±4.4258	59.05±0.04	59.54±0.29	57.98±3.21	59.54±0.29
S_5	49.87±16.5068	60.72±0.09	61.15±0.19	55.39±6.39	61.15±0.19

Table 4.2 Experiment run time (in seconds) on citation complex with $K=5$, from 10% to 50% data missing rate.

	SNN-2	SCNN-2	SCNN-3	Bi-SCNN-3	Bi-SCNN-2
10% missing	141 $\pm$ 0.37 s	365 $\pm$ 2.23 s	927 $\pm$ 11.58 s	35 $\pm$ 0.18 s	21 $\pm$0.78 s
20% missing	130 $\pm$ 0.19 s	356 $\pm$ 1.87 s	945 $\pm$ 8.42 s	36 $\pm$ 0.20 s	22 $\pm$1.73 s
30% missing	124 $\pm$ 0.61 s	358 $\pm$ 3.11 s	957 $\pm$ 9.04 s	36 $\pm$ 0.20 s	22 $\pm$1.35 s
40% missing	124 $\pm$ 0.41 s	362 $\pm$ 2.78 s	892 $\pm$ 9.86 s	35 $\pm$ 0.24 s	22 $\pm$1.12 s
50% missing	124 $\pm$ 0.22 s	357 $\pm$ 2.80 s	931 $\pm$ 5.71 s	36 $\pm$ 0.24 s	22 $\pm$1.18 s

put trajectory belongs to. We should emphasize that the formed simplex structure itself is undirected, but the data that resides on the simplex has directions. The data provided 160 training trajectories and 40 testing trajectories, all on the 1-simplex. In other words, the feature only exists on the edges. An illustration of this simplicial complex and five different Ocean Drifter Trajectories are shown in Fig 4.4.

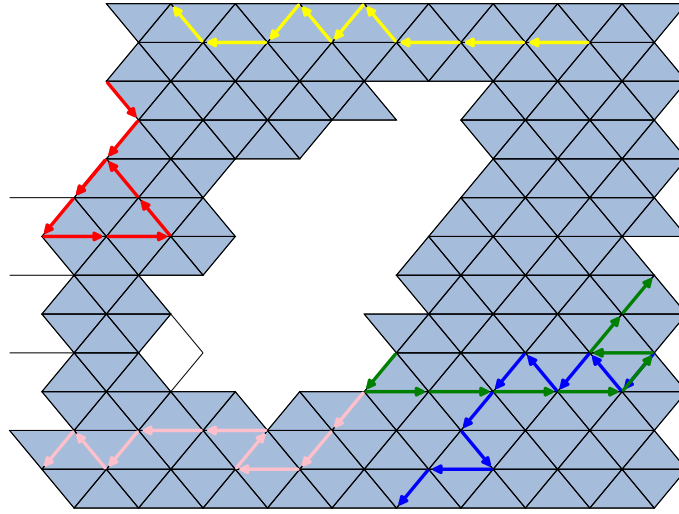


Figure 4.4 The Ocean Drifter Complex and five ocean drifters.

Our baseline algorithms are SAT^[86] and SCNN^[85]. We will follow the batch training procedure seen in SAT^[86] but with the batch size set to 40, and each experiment run consists of 5000 epochs. For all three algorithms, we will be running two different architectural settings: 2 simplicial layers and 3 simplicial layers. As suggested by^[86], the simplicial layers are followed by a mean pooling layer, then 2 MLP readout layers, and finally a softmax layer. For each architectural setting, there will be three different choices of activation function: identity (id) function, leaky ReLU (LR) function, and tanh function. The SAT suggests the choices of Id and Tanh functions because odd activa-

tion functions lead to orientation-equivariant output, which is shown to perform well on a synthetic ocean drifter dataset^[86]. The classification accuracy is shown in Table 4.3 and the time to complete the experiments is recorded in Table 4.4. The number of trainable parameters of each setting is shown in Table 4.5.

In the Ocean Drifter Dataset, we can observe again that the Bi-SCNN has similar classification accuracy to the SCNN for all three choices of action functions. The reason Bi-SCNN can consistently outperform SAT is that the Bi-SCNN utilizes the Hodge decomposition: Bi-SCNN optimizes parameters for upper and lower adjacencies separately, but SAT uses the standard Hodge Laplacian. Additionally, the processing of the harmonic component of the input simplicial features is absent in SAT, making its simplicial embedding less representative than convolution-based algorithms. One may notice that the runtime reduction of Bi-SCNN is not as prominent in the ocean drifter complex as in the citation complex. This is because when we recorded the run time, the global pooling and the 2 MLP layers also contributed to the time in addition to the time it takes to run the simplicial layers. The Bi-SCNN does not optimize the global pooling and the 2 MLP layers, so these layers will perform identically among Bi-SCNN, SAT, and SCNN. Still, Bi-SCNN is the fastest among Bi-SCNN, SAT, and SCNN when all three are compared under the same number of simplicial layers.

Table 4.3 Classification (test) accuracy on the Ocean Drifter Dataset. The number after the activation denotes the number of simplicial layers.

	SAT	SCNN	Bi-SCNN
Id, 2	60.25±6.47	64.00±3.39	71.25±4.0
Id, 3	64.75±5.30	69.00±2.78	71.00 ±4.06
LR, 2	61.75±7.24	77.00±4.00	79.50±3.67
LR, 3	62.25±2.08	81.75±2.97	77.00±4.15
Tanh, 2	64.00±6.04	74.75±6.75	78.75±5.94
Tanh, 3	66.5±5.50	75.25±6.66	73.75±3.01

4.5 Discussion

By utilizing time-invariant TSP techniques, we introduced Bi-SCNN, combining simplicial convolution with binary-sign propagation. Incorporating the upper and lower adjacencies using the Hodge Laplacian matrix, the Bi-SCNN can effectively capture higher-order structured simplicial features. Bi-SCNN outperforms previously proposed SNN

Table 4.4 Experiment run time (in seconds) on the Ocean Drifter Dataset. The number after the activation denotes the number of simplicial layers.

	SAT	SCNN	Bi-SCNN
Id, 2	1196 s	2197 s	143 s
Id, 3	1515 s	6450 s	232 s
LR, 2	1116 s	2187 s	165 s
LR, 3	1385 s	6441 s	231 s
Tanh, 2	1161 s	2196 s	150 s
Tanh, 3	1491 s	6399 s	234 s

Table 4.5 The number of trainable parameters of each network setting on all datasets.

	Citation	Ocean Drifter
SAT-2	-	5762
SAT-3	-	6722
SNN-2	906	-
SCNN-2	1986	4712
SCNN-3	29166	10142
Bi-SCNN-2	1146	3750
Bi-SCNN-3	15726	6450

algorithms on citation and ocean drifter complexes in terms of time efficiency, while maintaining comparable prediction accuracy. In the next Chapters, we will be switching gears and focusing on the cases where we have time-varying signals on graphs.

CHAPTER 5 ONLINE ESTIMATION OF TIME-VARYING GRAPH SIGNALS UNDER IMPULSIVE NOISE

5.1 Introduction

In classical signal processing, online estimation of time-varying signals is often accomplished using adaptive filters^[131]. Adaptive GSP algorithms are inspired by classical adaptive filters to perform online estimation of steady-state and time-varying graph signals through spectral methods^[35-37]. Analogous to the famous adaptive least mean squares (LMS) algorithm in classical adaptive filtering, the Graph least mean squares algorithm (GLMS) is popular due to its simplicity by using l_2 -norm optimization, which implicitly makes the Gaussian noise assumption^[35]. Other least-squares-based adaptive GSP algorithms such as the Graph normalized LMS (GNLMS)^[36] and the Graph recursive least squares (GRLS) algorithm^[37] are GSP analogs of classical adaptive filtering algorithms developed for faster convergence. Recent attention has focused on the distributed version of GSP algorithms, merging the distributed GLMS algorithm with diffusion algorithms^[38,132]. Similarly, distributed kernel GLMS was introduced in^[39], which extended the linear graph adaptive algorithm to a nonlinear version.

All these mentioned works implicitly make Gaussian noise assumptions. However, the ambient noise in various real-world applications is non-Gaussian with impulsive characteristics, which can be modeled by heavy-tailed distributions^[55,133-136]. Least-squares-based adaptive GSP algorithms, namely the GLMS, the GRLS, and the GNLMS, become unstable and diverge under impulsive noise due to large or infinite variance^[56]. In (non-graph) adaptive filtering, to effectively mitigate the influence of heavy-tailed non-Gaussian noise, the minimum absolute deviations (MAD) criterion and minimum dispersion criterion were used, which gave rise to the Sign or the least mean absolute deviation (LMAD) algorithm and the least mean p^{th} power (LMP) algorithm respectively^[56,58]. In order to avoid the drawbacks of l_2 -norm optimization when the noise distribution is impulsive and non-Gaussian in a multivariate data problem, the Graph-LMP algorithm (GLMP) was proposed, which assumes that the noise is symmetric α -stable distribution ($S\alpha S$)^[59]. This assumption leads to the minimum dispersion criterion, which can be optimized via l_p -norm minimization instead of l_2 -norm minimization, and the resulting algorithm is a direct extension of the classical LMP algorithm.

We propose two adaptive algorithms for GSP that do not experience the instability of the GLMS or GNLMS algorithms caused by the diverging variance of impulsive noise. The first algorithm is the graph normalized least mean p^{th} power (GNLMP) algorithm^[10]. Instead of a fixed step-size parameter seen in the GLMP algorithm, the GNLMP algorithm uses a time-varying convolution matrix $\mathbf{M}[t]$ that significantly reduces the number of iterations to converge to a stable value. The second algorithm we proposed is the Graph-Sign algorithm (G-Sign), with a lower computational cost than GLMP and GNLMP^[9]. The G-Sign algorithm is the GSP analogy of the classical adaptive sign error (or the LMAD) algorithm for multivariate signals^[58,131]. Our proposed G-Sign algorithm is derived based on the minimum absolute deviations criterion, which is a special case of the minimum least l_p -norm optimization for $p = 1$. This leads to l_1 -norm optimization, which removes the need for prior knowledge of any noise parameters or noise model assumptions^[137]. This allows the G-Sign algorithm to avoid the instability seen in the least-squares-based algorithms when estimating graph signals under impulsive noise. The G-Sign algorithm further reduces computational complexity to estimate a steady-state graph signal compared to the GLMP and the GLMS algorithms, making the G-Sign algorithm time-efficient. The G-Sign algorithm is robust when estimating time-varying graph signals under impulsive noise, making it capable of performing online graph signal estimation.

5.2 Impulsive Distributions

Algorithms based on least-squares estimation are based on the Gaussian noise assumption, which has proven to be an oversimplification of noise in various real-world applications. In this dissertation, the SaS , Cauchy, Student's t , and Laplace distributions are considered as additive noise to reflect the impulsiveness of real-life noises. In underwater communications, the noise can be modeled using a Cauchy-Gaussian mixture^[133] or using the Student's t distribution^[134]. In powerline communications and sensing applications such as source localization, the noise is modeled using α -stable distributions^[55,59]. In carbon nanotube detectors, the electrical noise follows a Cauchy distribution^[135]. The matrix factorization problems of several computer vision tasks are assumed to follow a mixture of Laplace distributions^[136]. The Probability Density Functions (PDFs) of these non-Gaussian distributions are illustrated in Fig. 5.1 and are discussed in this section. As shown in Fig. 5.1, in certain parameter sets of these distortions, heavy-tailed behavior is observed, which generates outliers in the data that can be considered as additive

noise. Least squares-based algorithms show unstable behavior under impulsive noise due to large or infinite variance; least l_p -norm based methods were proposed, which provide more stable estimators^[53,58].

The S α S distribution satisfies the generalized central limit theorem and is governed by the characteristic exponent α , the location parameter μ , and the scale parameter γ . The mean of S α S is only defined when $\alpha > 1$: with μ being the mean when $1 < \alpha \leq 2$, and the median when $\alpha < 1$. The variance of S α S is defined only when $\alpha = 2$, and in the other cases the scale parameter is given by the dispersion^[58]. Gaussian distribution is obtained by setting $\alpha = 2$, and a Cauchy distribution when $\alpha = 1$. From Fig. 5.1(a) we can observe that the tail of the S α S distribution is algebraic and hence heavy except for $\alpha = 2$. The S α S has no analytic PDF for other α values but has the characteristic function

$$\phi(t) = \exp \{ j\mu t - \gamma |t|^\alpha \}. \quad (5.1)$$

The Student's t distribution is a heavy-tailed distribution for estimating the mean of a Gaussian distribution under unknown variance and small sample size. The Student's t distribution is parameterized by degrees of freedom ν and can be defined as the distribution of the location of the sample mean relative to the true mean, divided by the sample standard deviation. The distribution has infinite variance when $1 < \nu \leq 2$, and undefined variance when $\nu \leq 1$. The Student's t distribution becomes the Gaussian distribution when $\nu = \infty$. Observing Fig. 5.1(b), we can see that the Student's t distribution is heavy-tailed compared to the Gaussian case. The PDF of the Student's t distribution is given by

$$\mathbf{f}(t) = \frac{\Gamma\left(\frac{\nu+1}{2}\right)}{\sqrt{\nu\pi}\Gamma\left(\frac{\nu}{2}\right)} \left(1 + \frac{t^2}{\nu}\right)^{-\frac{\nu+1}{2}}, \quad (5.2)$$

where Γ is the gamma function.

The Cauchy distribution is heavy-tailed with mean and variance undefined and is a special case of the Student's t distribution with $\nu = 1$ and of the S α S at $\alpha = 1$:

$$\mathbf{f}(t, \gamma) = \frac{1}{\pi\gamma \left[1 + \left(\frac{t-\mu_\alpha}{\gamma}\right)^2\right]}. \quad (5.3)$$

In Fig. 5.1(c), it is clear that the Cauchy distribution is heavy-tailed when compared to the Gaussian distribution.

Finally, the Laplace distribution, or the double exponential distribution, is a special case of the well-known generalized Gaussian distribution and is characterized by the location parameter μ and the scale parameter b . The PDF of the Laplace distribution can be

expressed as

$$\mathbf{f}(t, \mu, b) = \frac{1}{2b} \exp\left(-\frac{|t - \mu|}{b}\right). \quad (5.4)$$

Looking at Fig. 5.1(d), we can see that the tail of the Laplace distribution gets heavier when b increases.

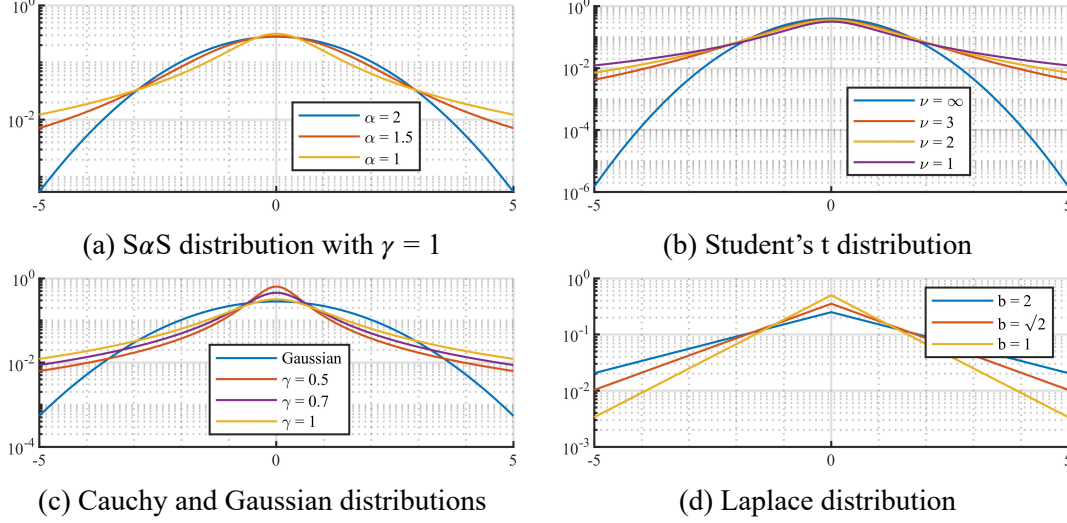


Figure 5.1 The PDF of various non-Gaussian distributions all with $\mu = 0$ in logarithmic scale.

5.3 Existing Adaptive GSP Algorithms

In Chapter 2, we went over several previously proposed adaptive GSP algorithms, namely the GLMS algorithm^[35], the GNLMS algorithm^[36], and the GLMP algorithm^[59]. In this section, we will conduct a theoretical analysis of the above algorithms based on the GSP techniques shown in Chapter 3. For the details of GSP preliminaries, please refer to Chapter 3.

Let us consider a bandlimited graph signal $\mathbf{x}_{bl}[t] \subseteq \mathbb{R}^N$ and its noisy observation $\mathbf{y}[t] = \mathbf{D}_S (\mathbf{x}_{bl}[t] + \boldsymbol{\eta}[t])$, where partial observations are modeled using a sampling matrix $\mathbf{D}_S$, and t represents the t^{th} discrete time step or iteration ranging from 0 to t_{max} . The noise $\boldsymbol{\eta}[t]$ is a zero-mean (or zero-median when the mean is undefined) random variable and serves as additive noise that is i.i.d. among N different nodes and across different t_{max} time steps. As mentioned in Chapter 3, the computation $\mathbf{U}\boldsymbol{\Sigma}\mathbf{U}^T$ can be reduced by defining $\mathbf{U}_{\mathcal{F}} = \mathbf{U}\boldsymbol{\Sigma}$ then dropping the all zeros columns, resulting in $\mathbf{U}_{\mathcal{F}}\mathbf{U}_{\mathcal{F}}^T\mathbf{x} = \mathbf{H}\mathbf{x}$ ^[37]. For a perfectly bandlimited graph signal $\mathbf{x}_{bl}[t]$ with frequency bands $\mathcal{F}$, $\mathbf{x}_{bl}[t] = \mathbf{H}\mathbf{x}_{bl}[t]$ ^[37]. In GLMS, the cost function is a convex optimization problem that aims to minimize the

error between $\mathbf{y}[t]$ and $\mathbf{D}_S \mathbf{H} \hat{\mathbf{x}}[t]$, where $\hat{\mathbf{x}}[t]$ is the current step estimate of $\mathbf{x}_{bl}[t]$:

$$J(\hat{\mathbf{x}}[t]) = \mathbb{E} \|\mathbf{y}[t] - \mathbf{D}_S \mathbf{H} \hat{\mathbf{x}}[t]\|_2^2. \quad (5.5)$$

In order to make one-step-ahead prediction, the spatial-domain update can be derived by stochastic gradient descent:

$$\hat{\mathbf{x}}[t+1] = \hat{\mathbf{x}}[t] + \mu_{lms} \mathbf{H} \mathbf{D}_S (\mathbf{y}[t] - \hat{\mathbf{x}}[t]), \quad (5.6)$$

where μ_{lms} is the step size.

In classical adaptive filtering, one possible solution to increase the convergence speed of the LMS algorithm is the normalization operation^[138]. Although in GSP the analog of the classical NLMS is the GNLMS algorithm, instead of just having a fixed step size, the GNLMS algorithm includes a symmetric convolution matrix as normalization^[36]. The update function of the GNLMS algorithm could be expressed as

$$\hat{\mathbf{x}}[t+1] = \hat{\mathbf{x}}[t] + \mu_n \mathbf{U}_F \mathbf{M}_n \mathbf{U}_F^T \mathbf{D}_S (\mathbf{y}[t] - \hat{\mathbf{x}}[t]), \quad (5.7)$$

where

$$\mathbf{M}_n = (\mathbf{U}_F^T \mathbf{D}_S \mathbf{U}_F)^{-1}, \quad (5.8)$$

and μ_n is the step size parameter. The inclusion of $\mathbf{M}_n$, μ_n and μ_{lms} will affect the update at each iteration differently.

The sensitivity of least squares to outliers makes least-squares-based algorithms such as the GLMS and the GNLMS algorithms unstable under impulsive noise^[53]. In order to overcome this limitation, the GLMP algorithm was introduced in^[139] as an improvement of the GLMS algorithm, modeling the noise using the $S\alpha S$. Instead of using least squares estimation, the cost function (5.5) is modified based on the MD criterion to obtain a stable estimation under the presence of $S\alpha S$ noise:

$$J(\hat{\mathbf{x}}[t]) = \mathbb{E} \|\mathbf{y}[t] - \mathbf{D}_S \mathbf{H} \hat{\mathbf{x}}[t]\|_p^p. \quad (5.9)$$

The parameter p is chosen such that $1 < p < 2$ to make the cost function differentiable^[139]. The update function of the GLMP algorithm is derived using stochastic gradient approaches similar to (5.6):

$$\hat{\mathbf{x}}[t+1] = \hat{\mathbf{x}}[t] + \mu_{lmp} \mathbf{H} \mathbf{D}_S (|\mathbf{y}[t] - \hat{\mathbf{x}}[t]|^{p-1}) \circ \text{Sign}(\mathbf{y}[t] - \hat{\mathbf{x}}[t]). \quad (5.10)$$

5.4 The Adaptive GNLMP Algorithm

Even though the GLMP algorithm is able to estimate the graph signal corrupted by S α S, the GLMP algorithm still does not solve the problem of slow convergence speed compared to the GLMS algorithm. We propose the GNLMP algorithm, which also has the advantage of normalization and has been shown for classical adaptive algorithms to speed up convergence and to make the estimates more stable.

We first transform (5.10) into the spectral-domain using GFT and replace μ_{lmp} with a time-varying convolution matrix $\mathbf{M}[t]$, leading to the spectral-domain update of the GNLMP algorithm:

$$\hat{\mathbf{s}}[t+1] = \hat{\mathbf{s}}[t] + \mathbf{M}[t] \mathbf{U}_{\mathcal{F}}^T (|\mathbf{e}[t]|^{p-1} \circ \text{Sign}(\mathbf{e}[t])), \quad (5.11)$$

where $\mathbf{e}[t] = \mathbf{D}_{\mathcal{S}}(\mathbf{y}[t] - \hat{\mathbf{x}}[t])$ is the current step spatial-domain estimation error.

In order to find $\mathbf{M}[t]$, following the derivations in^[36], we define the *a posteriori error* $\boldsymbol{\varepsilon}[t] = \mathbf{D}_{\mathcal{S}}(\hat{\mathbf{x}}[t] - \mathbf{U}_{\mathcal{F}}\hat{\mathbf{s}}[t+1])$ to measure the error between current step prediction and the next step prediction. We also define $\Delta\tilde{e}^p = \|\boldsymbol{\varepsilon}[t]\|_p^p - \|\mathbf{e}[t]\|_p^p$ to be a measure the closeness of $\boldsymbol{\varepsilon}[t]$ to $\mathbf{e}[t]$. In GNLMP, in addition to the optimization problem in (5.9), which can be seen as minimizing $\mathbf{e}[t]$, we want $\Delta\tilde{e}^p$ to be minimized as well. This minimization of $\Delta\tilde{e}^p$ with respect to $\mathbf{M}[t]$ could be interpreted as using the spectral domain difference between $\hat{\mathbf{s}}[t+1]$ and $\hat{\mathbf{s}}[t]$ as spectral domain normalization of the update term^[36], while keeping $\mathbf{e}[t]$ small based on (5.5). Taking the derivative of $\Delta\tilde{e}^p$ with respect to $\mathbf{M}[t]$, we have:

$$0 = \mathbf{O}^T (\mathbf{H}|\mathbf{e}[t]|^{p-1} \circ \text{Sign}(\mathbf{e}[t])) + (|\mathbf{e}[t]|^{p-1} \circ \text{Sign}(\mathbf{e}[t])\mathbf{H}) \mathbf{O}, \quad (5.12)$$

where

$$\mathbf{O} = \mathbf{D}_{\mathcal{S}} \mathbf{U}_{\mathcal{F}} \mathbf{M}[t] \mathbf{U}_{\mathcal{F}}^T (|\mathbf{e}[t]|^{p-1} \circ \text{Sign}(\mathbf{e}[t])) - \mathbf{e}[t]. \quad (5.13)$$

Notice in (5.12) that the two sides of the addition are transposes of each other. Therefore, we can solve (5.12) by setting $\mathbf{O}$ to zero. Utilizing the property $\mathbf{I} = \mathbf{U}_{\mathcal{F}}^T \mathbf{U}_{\mathcal{F}}$ and noticing that $\mathbf{D}_{\mathcal{S}}$ is idempotent and self-adjoint, the expression for $\mathbf{M}[t]$ is

$$\mathbf{M}[t] = (\mathbf{U}_{\mathcal{F}}^T \mathbf{D}_{\mathcal{S}} \text{diag}(|\mathbf{y}[t] - \hat{\mathbf{x}}[t]|^{p-2}) \mathbf{U}_{\mathcal{F}})^{-1}. \quad (5.14)$$

Following the convention of classical adaptive filtering, we add a step size parameter μ_{nlmp} to balance the convergence speed of the algorithm and the effectiveness of the update at each step. The spatial-domain update step of the adaptive GNLMP algorithm can be

formalized as

$$\hat{\mathbf{x}}[t+1] = \hat{\mathbf{x}}[t] + \mu_{nlmp} \mathbf{U}_{\mathcal{F}} \mathbf{M}[t] \mathbf{U}_{\mathcal{F}}^T (|\mathbf{e}[t]|^{p-1} \circ \text{Sign}(\mathbf{e}[t])), \quad (5.15)$$

where $\mathbf{M}[t]$ is shown in (5.14). Notice that this update function not only adaptively updates based on the error, but also has a time-varying parameter $\mathbf{M}[t]$, which is different from the GLMS, the GNLMS, and the GLMP algorithms, as they only adaptively update the error. It is worth mentioning that when we set $p = 2$ in (5.15), $\mathbf{M}[t]$ will reduce to $\mathbf{M}_n$, and $|\mathbf{e}[t]|^{p-1} \circ \text{Sign}(\mathbf{e}[t])$ reduces to just $\mathbf{e}[t]$, which is exactly the update function of GNLMS in (5.7).

Compared to the GLMP algorithm, the extra computations for calculating $\mathbf{M}[t]$ increase the run time of the GNLMP algorithm. Before that, we would like to simplify our algorithm without sacrificing much of the performance. Even though we do not have an analytical PDF for the S α S, we can use the FLOM to approximate the noise statistics. Also, the sampling strategy we adopted from^[36] does not change as the algorithm progresses. Using these facts, we can approximate $\mathbf{M}[t]$ to be $(\mathbf{U}_{\mathcal{F}}^T \mathbf{D}_{\mathcal{S}} \mathbf{R} \mathbf{U}_{\mathcal{F}})^{-1}$, with $\mathbf{R} = (E|\boldsymbol{\eta}[t]|^p)^{p-2} \mathbf{I}$. Using this approximation, we can combine all the time-independent terms of the GNLMP algorithm to form a matrix to perform the spectral domain filtering and normalization: $\mathbf{H}_n = \mathbf{U}_{\mathcal{F}} (\mathbf{U}_{\mathcal{F}}^T \mathbf{D}_{\mathcal{S}} \mathbf{R} \mathbf{U}_{\mathcal{F}})^{-1} \mathbf{U}_{\mathcal{F}}^T$. Now the update function of GNLMP in (5.15) is approximated to be

$$\hat{\mathbf{x}}[t+1] = \hat{\mathbf{x}}[t] + \mu \mathbf{H}_n \mathbf{D}_{\mathcal{S}} (|\mathbf{e}[t]|^{p-1} \circ \text{Sign}(\mathbf{e}[t])). \quad (5.16)$$

This formulation significantly reduces the number of operations done by our proposed GNLMP algorithm because, by predefining $\mathbf{B}_n$, now $\mathbf{H}_n$ is calculated only once. However, (5.16) loses the time-variability of $\mathbf{M}[t]$ that can be seen in (5.15). To maintain the adaptiveness gained from the time-varying $\mathbf{M}[t]$ in (5.15) as well as maintaining the efficiency of (5.16), we use a threshold-based switching between (5.15) and (5.16). At step t , if the total amount of update magnitude at all sampled nodes is smaller than a certain threshold, we switch from (5.15) to (5.16). Otherwise, the update is (5.15). Earlier in this section, we assumed that after a few iterations, $\mathbf{e}[t]$ is dominated by $\boldsymbol{\eta}[t]$; we set the threshold to be $threshold = |S| \text{FLOM}(p-1, \alpha, \gamma)$, where $|S|$ is the cardinality of S . The choice of threshold is not a strict requirement; it can be changed to other values to suit the needs of the target application. The resulting GNLMP algorithm is shown in Algorithm 5.1.

Algorithm 5.1 Threshold based GNLMP

```

 $\mathbf{H}_n = \mathbf{U}_F (\mathbf{U}_F^T \mathbf{D}_S \mathbf{R} \mathbf{U}_F)^{-1} \mathbf{U}_F^T$ 
 $threshold = |S| \text{FLOM}(p-1, \alpha, \gamma)$ 
while within the iteration limit do
    if  $\text{sum}(|\mathbf{e}[t]|^{p-1}) < threshold$  then
        update  $\hat{\mathbf{x}}[t+1]$  based on (5.16)
    else
        update  $\hat{\mathbf{x}}[t+1]$  based on (5.15)
    end if
end while

```

5.5 The Adaptive Graph Sign Algorithm

Even though the GLMS algorithm is simple, it is not time-efficient and suffers from instability. The GLMP algorithm^[59] and the GNLMP^[10] are extensions of the GLMS algorithm and have stable estimation performance compared to GLMS when estimating a graph signal under $S\alpha S$ noise but have additional computational complexity. In classical adaptive filtering, the LMS algorithm is used extensively due to its simplicity of implementation, and the Sign-Error algorithm or the LMAD algorithm is an extension of the LMS algorithm to further increase the run-speed and decrease the algorithm complexity, with additional robustness gained by the l_1 -norm cost function. To improve the time-efficiency and robustness of adaptive GSP algorithms under impulsive noise, we use a special case of the minimum dispersion criterion to form the cost function, reducing it to an l_1 -norm optimization problem as in the LMAD algorithm in one-dimensional adaptive filtering^[58]:

$$J(\hat{\mathbf{x}}[t]) = \mathbb{E} \|\mathbf{y}[t] - \mathbf{D}_S \mathbf{H} \hat{\mathbf{x}}[t]\|_1. \quad (5.17)$$

l_1 -norm error function provides a very plausible choice when the distribution is $S\alpha S$, Cauchy, Laplace, or Student's t, with unknown density parameters^[53]. The cost function (5.17) can be viewed as recovering the mean $\mathbf{x}_{bl}[t]$ from distribution $\mathbf{y}[t]$ and is LMAD-sense optimal for $S\alpha S$ and Cauchy noise. Equation (5.17) is the optimal Maximum Likelihood Estimator for parameter estimation in the Laplace distribution. Using the bandlimitedness property $\mathbf{H} \hat{\mathbf{x}}[t] = \hat{\mathbf{x}}[t]$, the update function of the G-Sign algorithm is obtained by stochastic gradient as in (5.6)

$$\begin{aligned} \hat{\mathbf{x}}[t+1] &= \hat{\mathbf{x}}[t] - \mu_s \frac{\partial \|\mathbf{y}[t] - \mathbf{D}_S \mathbf{H} \hat{\mathbf{x}}[t]\|_1}{\partial \hat{\mathbf{x}}[t]} \\ &= \hat{\mathbf{x}}[t] + \mu_s \mathbf{H} \mathbf{D}_S \text{Sign}(\mathbf{D}_S (\mathbf{y}[t] - \hat{\mathbf{x}}[t])). \end{aligned} \quad (5.18)$$

The $\text{Sign}(\cdot)$ function in the update equation results from taking the derivative of the

l_1 -norm cost function, with consideration of the point of discontinuity of the derivative at 0. This makes the update function resemble the form seen in the weight update of classical LMAD or Sign-Error algorithm^[58,131]. A step size parameter μ_s is added by following the classical adaptive filtering convention to control the magnitude of the update. The i^{th} 0 in the diagonal of $\mathbf{D}_S$ corresponds to a 0 in the i^{th} element of $\text{sign}(\mathbf{D}_S (\mathbf{y}[t] - \hat{\mathbf{x}}[t]))$. So, we can safely refactor (5.18) into

$$\hat{\mathbf{x}}[t+1] = \hat{\mathbf{x}}[t] + \mu_s \mathbf{H} \text{Sign}(\mathbf{D}_S (\mathbf{y}[t] - \hat{\mathbf{x}}[t])). \quad (5.19)$$

This fixed amount of update from the minimum dispersion criterion is unaffected by impulsive noise because $\text{Sign}(\cdot)$ is only defined between -1 and 1, which clips the amplitude of extreme noise outliers^[58,140].

Equation (5.19) reduces the number of operations by $2(N - |S|)$ because the zeros in $\mathbf{D}_S$ make $\text{Sign}(\mathbf{D}_S (\mathbf{y}[t] - \hat{\mathbf{x}}[t]))$ sparse. The $\text{Sign}(\cdot)$ operation essentially compares the non-zero elements in S , where in the worst case $\mathbf{y}[t] = \hat{\mathbf{x}}[t]$ it compares all the digits of $\hat{\mathbf{x}}[t]$ and $\mathbf{y}[t]$ without performing any algebraic operation. Since $\hat{\mathbf{x}}[t]$ is a noisy estimation of an observation $\mathbf{y}[t]$, this worst case $\hat{\mathbf{x}}[t] = \mathbf{y}[t]$ is unlikely to happen. By inspecting (5.15), it is not difficult to see that the GNLMP algorithm takes more computations than the G-Sign algorithm. The computational complexity of the G-Sign algorithm in (5.19), the GNLMP algorithm in (5.15), the approximated GNLMP algorithm in (5.16), the GLMS algorithm^[35], and the GLMP algorithm^[59] is in Table 5.1.

Table 5.1 Computational Complexity Analysis

	GLMS ^[35]	GLMP ^[59]	GNLMP (5.15)	GNLMP (5.16)	G-Sign (5.19)
Addition	$N^2 + N$	$N^2 + 2N$	$N^3 + 2N$	$N^2 + 2N$	$N^2 + S $
Multiplication	$N^2 + 2N$	$N^2 + 3N$	$N^3 + 2N^2 + 4N$	$N^2 + 3N$	$N^2 + N + S $
p^{th} power	0	N	$2N$	N	0
$\text{Sign}(\cdot)$	0	N	N	N	$ S $

Remarks Note that the GSP-based adaptive algorithms update the graph-signal estimates instead of the weights for the data samples as done by classical adaptive algorithms. The desired parallelism of the inference in predicting the outcomes of typically hundreds of nodes is made possible by the graph version of the signal processing algorithms. Dimensionality reduction is achieved by localizing the graph signal both spatially and spectrally. However, introducing a large amount of weights that need to be jointly optimized is mainly avoided by preferring a data-centered approach in graph-based gradient methods.

5.6 Convergence Analysis of G-Sign Under Steady-State Estimation

To estimate the steady-state performance of the G-Sign algorithm, we provide an analysis of the update function (5.19) using the Mean Squared Deviations (MSD). We follow the definition seen in^[36], where if the algorithm's MSD behavior does not change after a few iterations under steady-state estimation, then we say the algorithm converges. Based on a second-order analysis of the G-Sign algorithm, the MSD at step t is calculated by:

$$\text{MSD}[t] = \mathbb{E} \left\| \hat{\mathbf{x}}[t] - \mathbf{x}_{bl}[t] \right\|^2. \quad (5.20)$$

Note that the $\text{Sign}(\cdot)$ function will clip the actual estimation output $\hat{\mathbf{x}}[t+1]$ in (5.19), so an MSD analysis is possible on the actual output when executing the algorithm.

Generally, the MSD analysis is conducted for impulsive noise distributions that possess finite variance, such as the generalized Gaussian distribution or, in some cases, the Student's t distribution. In situations where the $S\alpha S$ is undefined and assuming that the noise statistics are known for theoretical analysis only, it is shown that the normalized second-order and fourth-order moments converge in probability to a finite value^[141-142]. We can conduct the MSD analysis using the fractional lower-order moment (FLOM), so a finite statistical moment is approximated. Let the error of estimating $\mathbf{x}_{bl}[t]$ at step t be $\tilde{\mathbf{x}}[t] = \hat{\mathbf{x}}[t] - \mathbf{x}_{bl}[t]$, then the error of the update (5.19) is

$$\tilde{\mathbf{x}}[t+1] = \tilde{\mathbf{x}}[t] + \mu_s \mathbf{H} \text{Sign}(\mathbf{D}_S(\boldsymbol{\eta}[t] - \tilde{\mathbf{x}}[t])). \quad (5.21)$$

The term $\boldsymbol{\eta}[t] - \tilde{\mathbf{x}}[t]$ is unlikely to be zero due to the estimation inaccuracy and the noise, then $\text{Sign}(\boldsymbol{\eta}[t] - \tilde{\mathbf{x}}[t]) = c(\boldsymbol{\eta}[t] - \tilde{\mathbf{x}}[t]) = (\boldsymbol{\eta}[t] - \tilde{\mathbf{x}}[t]) / |\boldsymbol{\eta}[t] - \tilde{\mathbf{x}}[t]|$ for $\boldsymbol{\eta}[t] - \tilde{\mathbf{x}}[t] \neq 0$; here the division is the element-wise division. For spectral analysis, (5.21) is transformed to the spectral domain using GFT by multiplying $\mathbf{U}_{\mathcal{F}}^T$ on both sides of (5.21):

$$\tilde{\mathbf{s}}[t+1] = \tilde{\mathbf{s}}[t] + \mu_s \mathbf{U}_{\mathcal{F}}^T \mathbf{D}_S c(\boldsymbol{\eta}[t] - \tilde{\mathbf{x}}[t]), \quad (5.22)$$

where $\tilde{\mathbf{s}}[t]$ is the GFT of $\tilde{\mathbf{x}}[t]$.

The error, or the deviation from the ground-truth value in the mean-square sense, can be obtained by taking the expectation of l_2 -norm squared (5.22), leading to

$$\mathbb{E} \|\tilde{\mathbf{s}}[t+1]\|^2 = \mathbb{E} \|\tilde{\mathbf{s}}[t]\|^2 + \mathbb{E} \|\mu_s \mathbf{U}_{\mathcal{F}}^T \mathbf{D}_S c(\boldsymbol{\eta}[t] - \tilde{\mathbf{x}}[t])\|^2. \quad (5.23)$$

Considering that $\boldsymbol{\eta}[t] \gg \tilde{\mathbf{x}}[t]$ when t is large, we use the first-order Taylor series approximation of $c(\boldsymbol{\eta}[t] - \tilde{\mathbf{x}}[t])$ around $\tilde{\mathbf{x}}[t] = 0$ to split the RHS of (5.23) into the error

caused by noise and error caused by algorithm estimation, resulting in $c(\boldsymbol{\eta}[t] - \tilde{\mathbf{x}}[t]) \approx c(\boldsymbol{\eta}[t]) - \tilde{\mathbf{x}}[t]c'(\boldsymbol{\eta}[t])$ which can be seen as a sigmoid function approximation to the signing algorithm, making it differentiable. Using the inverse GFT, $\tilde{\mathbf{x}}[t]$ can be written as $\mathbf{U}_F \tilde{\mathbf{s}}[t]$ and (5.23) becomes

$$\begin{aligned} \mathbb{E} \|\tilde{\mathbf{s}}[t+1]\|^2 &= \mathbb{E} \|\tilde{\mathbf{s}}[t]\|^2 + \mathbb{E} \|\mu_s \mathbf{U}_F^T \mathbf{D}_S (c(\boldsymbol{\eta}[t]) - \mathbf{U}_F \tilde{\mathbf{s}}[t] c'(\boldsymbol{\eta}[t]))\|^2 \\ &= \mathbb{E} \|\tilde{\mathbf{s}}[t]\|_{\Phi}^2 + \mu_s^2 \mathbb{E} \|\mathbf{U}_F^T \mathbf{D}_S \mathbf{R}_w \boldsymbol{\eta}[t]\|^2, \end{aligned} \quad (5.24)$$

where $\|\tilde{\mathbf{s}}[t]\|_{\Phi}^2$ is the weighted Euclidean norm $\tilde{\mathbf{s}}^T[t] \Phi \tilde{\mathbf{s}}[t]$ and Φ can be expressed as

$$\Phi = (\mathbf{I} - \mu_s \mathbf{U}_F^T \mathbf{D}_S \mathbf{R}_p \mathbf{U}_F)^T (\mathbf{I} - \mu_s \mathbf{U}_F^T \mathbf{D}_S \mathbf{R}_p \mathbf{U}_F). \quad (5.25)$$

The expectation of the element-wise division $1/|\boldsymbol{\eta}[t]|$ in $c(\boldsymbol{\eta}[t])$ can be approximated using the FLOM $\mathbb{E}|\boldsymbol{\eta}[t]|^{-p}$ as seen in the GLMP algorithm^[59], where p is modified from 1 to 0.99 which gives us $\mathbf{R}_p = (1-p)\mathbb{E}|\boldsymbol{\eta}[t]|^{-p}\mathbf{I}$, and $\mathbf{R}_w = \mathbb{E}|\boldsymbol{\eta}[t]|^{-p}\mathbf{I}$. Equation(5.24) can be factorized using the trace trick $\mathbb{E}\{\mathbf{X}^T \mathbf{Y} \mathbf{X}\} = \text{Tr}(\mathbb{E}\{\mathbf{X} \mathbf{X}^T \mathbf{Y}\})$:

$$\mathbb{E} \|\tilde{\mathbf{s}}[t+1]\|^2 = \mathbb{E} \|\tilde{\mathbf{s}}[t]\|_{\Phi}^2 + \mu_s^2 \text{Tr}(\mathbf{U}_F^T \mathbf{D}_S \mathbf{C} \mathbf{D}_S \mathbf{U}_F), \quad (5.26)$$

where $\mathbf{C} = \mathbb{E} \|\mathbf{R}_w \boldsymbol{\eta}[t]\|^2$ has a similar structure to the correlation matrix of $\mathbf{R}_w \boldsymbol{\eta}[t]$. In Assumption 5.3 we assumed that the noises across different nodes are i.i.d., so the correlation of the noise between two different nodes is zero, leading to $\mathbf{C} = \mathbf{I}$. Combining with the idempotent and self-adjoint property of the sampling matrix $\mathbf{D}_S$, (5.26) can be simplified to

$$\mathbb{E} \|\tilde{\mathbf{s}}[t+1]\|^2 = \mathbb{E} \|\tilde{\mathbf{s}}[t]\|_{\Phi}^2 + \mu_s^2 \text{Tr}(\mathbf{U}_F^T \mathbf{D}_S \mathbf{U}_F). \quad (5.27)$$

Using the properties $\text{Tr}\{\mathbf{Y} \mathbf{X}\} = \text{vec}(\mathbf{X}^T)^T \text{vec}(\mathbf{Y})$ and $\text{vec}(\mathbf{X} \mathbf{Y} \mathbf{Z}) = (\mathbf{Z}^T \otimes \mathbf{X}) \text{vec}(\mathbf{Y})$, (5.27) can be written into a recursive relationship starting from time step 0:

$$\begin{aligned} \mathbb{E} \|\tilde{\mathbf{s}}[t+1]\|^2 &= \mathbb{E} \|\tilde{\mathbf{s}}[0]\|_{\Phi}^2 + \mu_s^2 \sum_{i=0}^t \text{Tr}(\Phi^i \mathbf{G}) \\ &= \mathbb{E} \|\tilde{\mathbf{s}}[0]\|_{\Phi}^2 + \mu_s^2 \text{vec}(\mathbf{G})^T \sum_{i=0}^t \mathbf{Q}^i \text{vec}(\mathbf{I}), \end{aligned} \quad (5.28)$$

where $\tilde{\mathbf{s}}[0]$ is the error at $k = 0$, $\mathbf{G} = \mathbf{U}_F^T \mathbf{D}_S \mathbf{U}_F$, and $\mathbf{Q} = (\mathbf{I} - \mu_s \mathbf{U}_F^T \mathbf{D}_S \mathbf{R}_p \mathbf{U}_F)^T \otimes (\mathbf{I} - \mu_s \mathbf{U}_F^T \mathbf{D}_S \mathbf{R}_p \mathbf{U}_F)$. From (5.28), we see that $\mathbb{E} \|\tilde{\mathbf{s}}[t+1]\|^2$ converges to a steady value if the RHS of (5.28) converges.

Considering (5.25), when the initial error $\tilde{\mathbf{s}}[0]$ is bounded, we demand that the weighted Euclidean norm $\mathbb{E} \|\tilde{\mathbf{s}}[0]\|_{\Phi}^2$ converges to zero at large t , requiring $\|(\mathbf{I} -$

$\mu_s \mathbf{U}_F^T \mathbf{D}_S \mathbf{R}_p \mathbf{U}_F$) $\| < 1$. Following this condition, the summation term on the RHS of (5.28) will be a geometric series that converges to a constant value. For a diagonalizable matrix $\mathbf{X}$ of size $N \times N$ and a vector $\mathbf{z}$ of size N , the property $\|\mathbf{X}\mathbf{z}\|^2 = \sum \lambda_i z_i^2$ results in the inequality $\|\mathbf{X}\| \leq |\lambda_{\max}|$. Here $\{\lambda_i, i = 1 \dots N\}$ are the eigenvalues of $\mathbf{X}$ and $\lambda_{\max}$ is the largest eigenvalue of $\mathbf{X}$. Let $\mathbf{X} = \mathbf{U}_F^T \mathbf{D}_S \mathbf{R}_p \mathbf{U}_F$ and use the above property; satisfying the condition $\|(\mathbf{I} - \mu_s \mathbf{U}_F^T \mathbf{D}_S \mathbf{R}_p \mathbf{U}_F)\| < 1$ is equivalent to satisfying the condition $0 < |1 - \mu_s \lambda_{\max}| < 1$. Since the parameter μ_s is the step size, it should always have a positive value. Knowing that $\lambda_{\max}$ is the maximum eigenvalue of $\mathbf{U}_F^T \mathbf{D}_S \mathbf{R}_p \mathbf{U}_F$, $\lambda_{\max}$ is positive because the FLOM calculated in $\mathbf{R}_p$ is positive. As a result, the error of the G-Sign algorithm will converge to a steady value by choosing the only free parameter μ_s to satisfy the inequality $0 < |1 - \mu_s \lambda_{\max}| < 1$, leading to the condition

$$0 < \mu_s < \frac{2}{\lambda_{\max}}. \quad (5.29)$$

Even though the condition (5.29) has a structure similar to the convergence condition seen in GLMS, the bound $2/\lambda_{\max}$ is obtained using a different expression than the one for GLMS. This bound is derived based on the $\text{Sign}(\cdot)$ update of (5.19) that originated from the non-Gaussian noise assumption and the l_1 -norm optimization problem (5.17). Under condition (5.29), the G-Sign algorithm has a stable MSD behavior.

As $k \rightarrow \infty$ and μ_s satisfies (5.29), the difference between $\tilde{\mathbf{s}}[t]$ and $\tilde{\mathbf{s}}[t]$: $\lim_{k \rightarrow \infty} \tilde{\mathbf{s}}[t] = \lim_{k \rightarrow \infty} \tilde{\mathbf{s}}[t+1]$. Using the energy conservation property of GFT seen in the GLMS in^[35], which is a direct extension from the energy conservation of the classical LMS proven in^[143], the relationship between the spatial domain MSD and the spectral domain MSD is $\lim_{k \rightarrow \infty} \mathbb{E} \|\tilde{\mathbf{x}}[t]\|^2 = \lim_{k \rightarrow \infty} \mathbb{E} \|\tilde{\mathbf{s}}[t]\|^2$. In GFT, this energy-preserving property (a special case of Parseval's relation), can be easily proven using the property of orthonormal basis $\mathbf{U}^T = \mathbf{U}^{-1}$, where $\|\tilde{\mathbf{s}}[t]\|^2 = (\mathbf{U}^T \tilde{\mathbf{x}}[t])^T \mathbf{U}^T \tilde{\mathbf{x}}[t] = \|\tilde{\mathbf{x}}[t]\|^2$ ^[144]. Re-visiting (5.28), after selecting a μ_s that satisfies (5.29), the theoretical steady-state MSD can be calculated:

$$\begin{aligned} \text{MSD}[t] &= \lim_{k \rightarrow \infty} \mathbb{E} \|\tilde{\mathbf{x}}[t]\|^2 = \lim_{k \rightarrow \infty} \mathbb{E} \|\tilde{\mathbf{s}}[t]\|^2 \\ &= \mu_s^2 \text{vec}(\mathbf{G})^T (\mathbf{I} - \mathbf{Q})^{-1} \text{vec}(\mathbf{I}). \end{aligned} \quad (5.30)$$

5.7 Graph-Sign-Diffusion Algorithm

Directly applying the spectral approach shown in Section 5.5 requires the eigendecomposition, which takes $O(n^3)$ computational complexity and can be unstable when the

graph size is large^[31]. This problem limits the applicability of GSP algorithms when the target data has dynamic topology, as each change in topology requires a new eigendecomposition to obtain GFT. As a solution, one can use a shifted Chebyshev polynomial to approximate the filters, as shown in Chapter 3, equation 3.11, to avoid the eigendecomposition and convert spectral algorithms into spatial algorithms. It can be shown that $\mathbf{U}h_l(\mathbf{\Lambda})\mathbf{U}^T = h_l(\mathbf{L})$. To approximate $h_l(\mathbf{L})$ by a series of shifted Chebyshev polynomials, let us revisit (3.11), where we define the Chebyshev polynomial $T_p(\mathbf{L})$ over the interval $[0, \lambda_N]$:

$$T_p(\mathbf{L}) = \begin{cases} 1, & \text{if } p = 0 \\ \frac{2\mathbf{L} - \lambda_N}{\lambda_N}, & \text{if } p = 1 \\ \frac{4\mathbf{L} - 2\lambda_N}{\lambda_N} T_{p-1}(\mathbf{L}) - T_{p-2}(\mathbf{L}), & \text{if } p \geq 2. \end{cases} \quad (5.31)$$

Let us also revisit the approximation of the matrix $h_l(\mathbf{L})$ shown in Chapter 3:

$$h_l(\mathbf{L}) \approx \theta_0 + \sum_{p=1}^P \theta_p T_p(\mathbf{L}) = \sum_{p=0}^P \hat{\theta}_p \mathbf{L}^p, \quad (5.32)$$

where P is the number of polynomials, θ_p and $\hat{\theta}_p$ are the weight of the p^{th} polynomial. Now, using this approximation and following the optimization steps in Section 5.5, a graph-diffusion-based spatial domain update function of the G-Sign algorithm can be derived as:

$$\hat{\mathbf{x}}[t+1] = \hat{\mathbf{x}}[t] - \mu_s \frac{\partial \|J(\hat{\mathbf{x}}[t])\|_1}{\partial \hat{\mathbf{x}}[t]} = \hat{\mathbf{x}}[t] + \mu_s \sum_{p=0}^P \hat{\theta}_p \mathbf{L}^p \mathbf{D}_S \text{Sign}(\mathbf{y}[t] - \hat{\mathbf{x}}[t]). \quad (5.33)$$

The p^{th} power of $\mathbf{L}$ aggregates the signal from the p -hop neighboring nodes. The update in (5.33) aggregates the p -hop signed error with magnitude μ_s , but in the opposite direction of the error. Similar to the spectral case, the $\text{Sign}(\cdot)$ part in (5.18) is the optimization result from the minimum dispersion criterion, which has a fixed magnitude and is unaffected by impulsive noise.

5.8 Experimental Results

We would like to test the performance of the G-Sign algorithm in estimating graph signals under impulsive noise. Steady-state experiments are conducted using the random sensor graph generated by the Python package PyGSP shown in Fig. 5.2 with $N = 50$, bandlimited frequencies $|\mathcal{F}| = 20$, and a greedy sampling strategy in^[36] with $|\mathcal{S}| = 30$.

A real-time-varying graph signal with the topology shown in Fig. 5.3 is estimated in Sec-

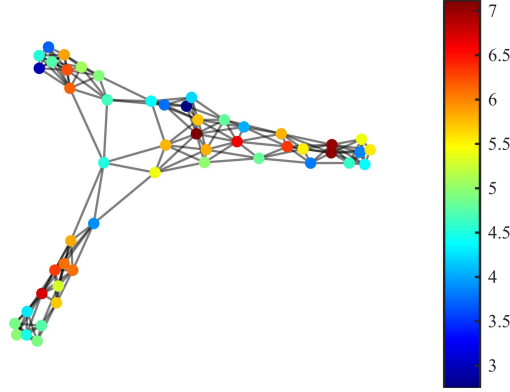


Figure 5.2 The graph signal of a sensor graph and its topology.

tion 5.8.3. The graph signal in Fig. 5.3 represents hourly temperature recorded across the U.S. at 205 different locations^[125].

For time-varying experiments, we used a real-world dataset of weather recordings. The graph topology is defined using the geological locations of the weather stations, where each station is a node. We follow an 8-nearest-neighbor approach to construct the graph seen in^[36] to form the topology shown in Fig. 5.3 with $N = 205$. In the experiment, the sampling technique is the same greedy strategy as in^[36] with $|S| = 130$ and $|F| = 125$. All the experiments are averaged over 100 independent runs. The experiments were conducted in MATLAB 2020b on a computer with an AMD Ryzen 5 3600 CPU and 32GB of RAM.

5.8.1 The Effect of the Step Size Parameter

In the G-Sign algorithm, the step size μ_s is the only user-defined parameter. We want to observe the effect of changing the value of μ_s . The experiment is conducted using the graph signal shown in Fig. 5.2, and the graph signal is corrupted by S α S noise with $\alpha = 1.06$ and $\gamma = 0.1$. We tested four different values of μ_s and ran the G-Sign algorithm for $t_{max} = 3000$ iterations. The performance of the G-Sign algorithm is measured by comparing the output graph signal estimation with the clean bandlimited graph signal using the MSD defined in (5.20).

Both the MAD and the MSD of this experiment are shown in Fig. 5.4. We can see from Fig. 5.4 that as μ_s decreases, the G-Sign algorithm will get more accurate but will also require more iterations to converge to a steady value.

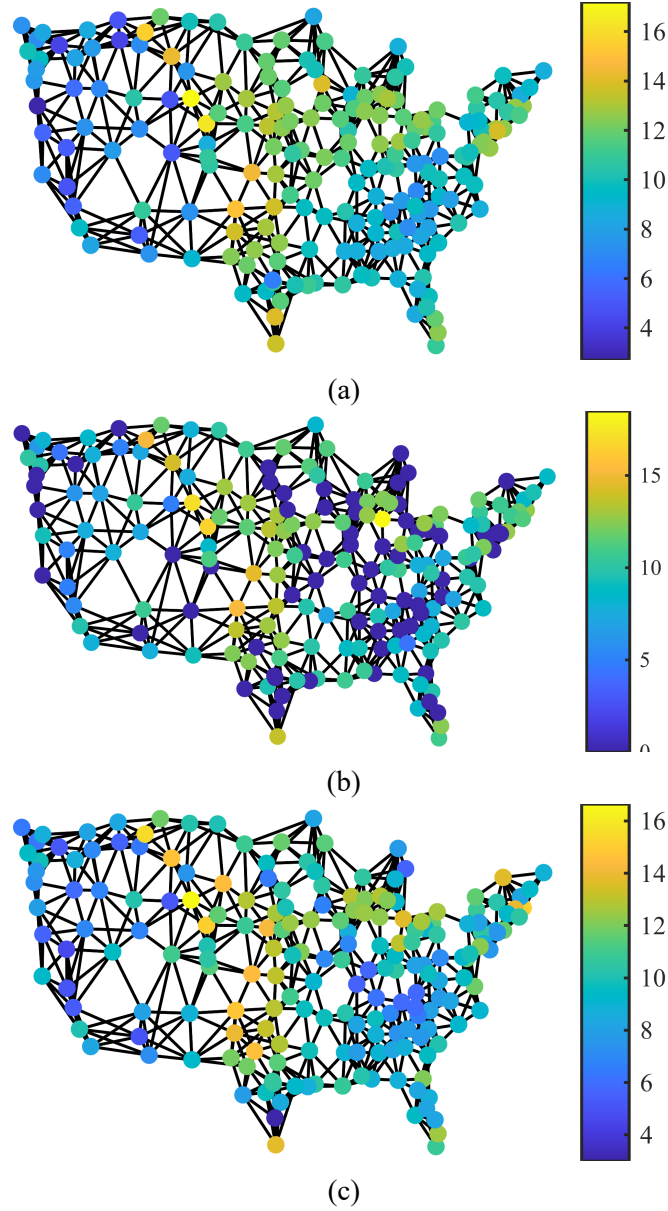


Figure 5.3 The first time instance of time-varying temperatures as a graph signal and its graph topology. (a) Ground truth graph signal. (b) Noisy observation ($S\alpha S$ noise) with missing values (missing values are treated as 0 in the figure). (c) Recovered graph signal by G-Sign.

5.8.2 Steady-State MSD Under Non-Gaussian Noises

The G-Sign algorithm and the GNLMP algorithm are compared with the GLMS algorithm and the GLMP algorithm for estimating a partially observed steady-state graph signal under $S\alpha S$, Cauchy, Student's t , and Laplace noises. The aim is to compare the stability of estimation, the iterations until convergence, and the run-time. In order to fairly compare the algorithms under each noise scenario, the step sizes are tuned so the algorithms behave similarly in MSD when there is a stable estimation. Notice that $S\alpha S$ becomes Cauchy when $\alpha = 1$, and the p parameter of the GLMP algorithm is defined only

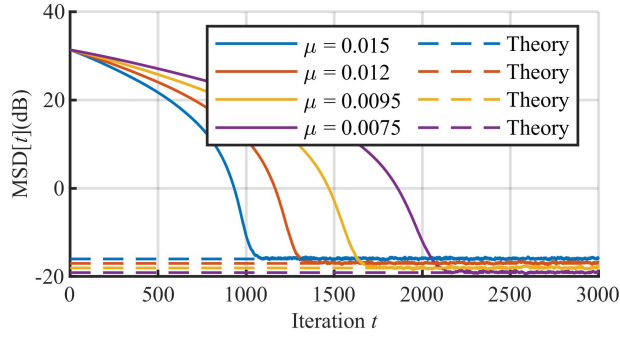


Figure 5.4 MSD performance for estimating a steady-state graph signal under $S\alpha S$ noise using different μ_s .

for $1 < p < 2$ with $p < \alpha$, so we test the GLMP algorithm under a near Cauchy $S\alpha S$ noise with $\alpha = 1.06$ with $p = \alpha - 0.05$ (as suggested in^[59]) for the Cauchy experiment. For the experiment under $S\alpha S$ noise, $p = \alpha - 0.05$ for the GLMP algorithm, and for other noise distributions we set $p = 1.5$ for the GLMP algorithm. The MSD of the experiments is in Fig. 5.5 with the theoretical MSD calculated using (5.30). The run time for running $t_{max} = 2400$ iterations of each algorithm for different experiments is in Table 5.2.

Table 5.2 Run Time Comparison of Steady-State Experiments

	$S\alpha S$	Cauchy	Student's t	Laplace
GLMS	0.0175(s)	0.0183(s)	0.0164(s)	0.0179(s)
GLMP	0.0313(s)	0.0232(s)	0.0599(s)	0.0235(s)
GNLMP	0.0351(s)	0.0488(s)	0.0659(s)	0.0470(s)
G-Sign	0.0029(s)	0.0028(s)	0.0028(s)	0.0026(s)

From Fig. 5.5, we can see that the GLMS algorithm is unstable when estimating the graph signal under $S\alpha S$, Cauchy, and Student's t noise. This instability is introduced by the heavy-tailed behavior of the noises^[53]. In Fig. 5.5(d), the GLMS algorithm is stable under Laplace noise but requires about 5 times the run-time that the G-Sign algorithm needs to converge. The G-Sign algorithm behaves similarly to the GLMP algorithm under Cauchy/near Cauchy noise in Fig. 5.5(b) because both algorithms are derived based on the minimum dispersion criterion. Under $S\alpha S$ noise, even though the iterations to reach the same steady-state MSD for the G-Sign algorithm and GLMP algorithm in Fig. 5.5(b) are 1200 iterations and 650 iterations, respectively, when looking at Table 5.2 we see that the G-Sign algorithm only takes 1/8 the time that the GLMP algorithm needs to complete the experiment. We can make similar observations when comparing GNLMP and G-Sign:

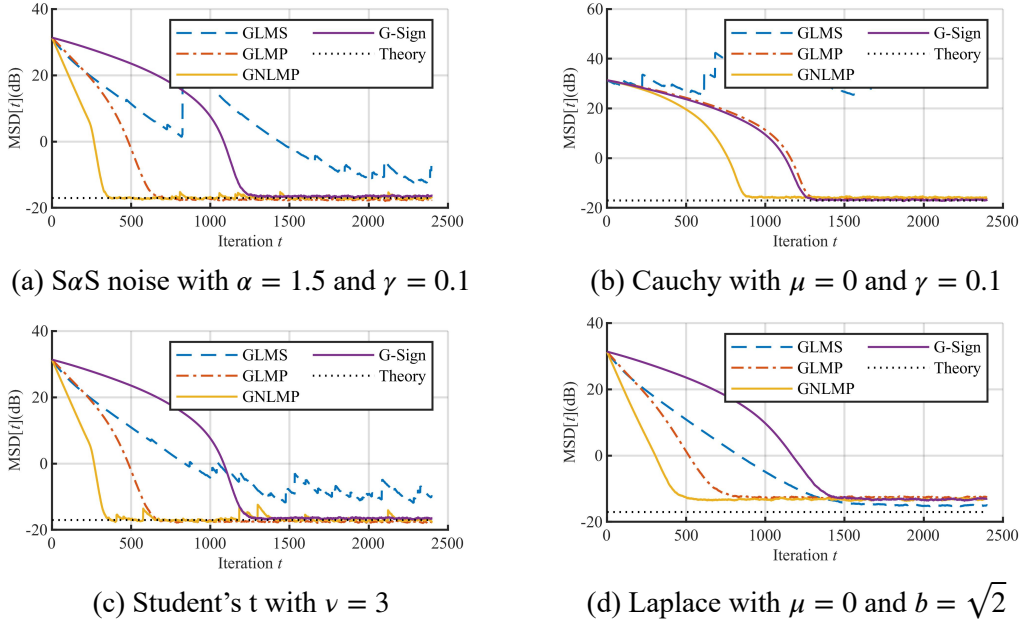


Figure 5.5 MSD of the experiments and the theoretical MSD for estimating steady-state graph signal under different noises.

even though the GNLMP takes roughly 1/3 of the iterations to converge to the same MSD, the G-Sign only takes roughly 1/10 of the time to complete the experiment. We should emphasize that this experiment only aims to look at the steady-state convergence behavior of the algorithms. The MSD performance of the G-Sign algorithm matches the theoretical results under the four impulsive noises. In Table 5.2, we see that the G-Sign algorithm has the fastest run-time under all scenarios, which is consistent with the analysis in Table 5.1 that the G-Sign algorithm has the lowest computational complexity. Combining with Fig. 5.5, we conclude that for steady-state graph signal estimation with missing node values under non-Gaussian noise, the proposed G-Sign algorithm is able to make a stable estimation and has a faster run-time compared to the GLMS algorithm and the GLMP algorithm.

5.8.3 Time-varying Estimation Under Impulsive Noise

In this section, the G-Sign and the GNLMP algorithms are used to estimate a time-varying graph signal corrupted by noise modeled by SαS, Cauchy, Student's t, and Laplace distributions. The G-Sign algorithm is compared to the GLMP and GLMS algorithms. The duration of this time-varying graph signal is 95 hours, making $t_{max} = 95$. At each iteration, each adaptive GSP algorithm outputs an online estimate of the graph signal. Again, for the Cauchy experiment, we run the GLMP algorithm under a near-Cauchy SαS noise with $\alpha = 1.06$ and $p = \alpha - 0.05$. An illustration of one time step of the

graph signal is shown in Fig. 5.3. To make a fair comparison, the step sizes are $\mu_s = 1.5$ for all the algorithms. Fig. 5.6 illustrates the estimation of one selected node with

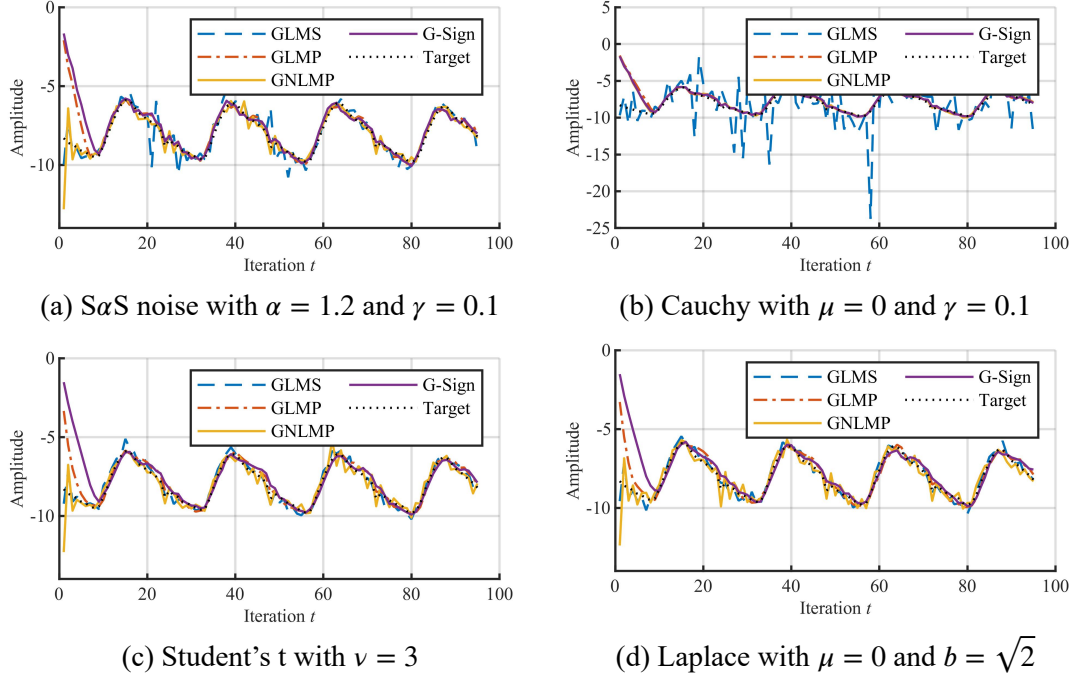


Figure 5.6 Estimation of one selected node from a time-varying graph signal under different noises.

a time-varying graph signal. Notice that the GLMS algorithm is again unstable under SαS, Cauchy, and Student's t noises, whereas the G-Sign algorithm is not influenced by any of these impulsive noises. The run times for GLMS, GLMP, GNLMP, and G-Sign algorithms to finish this experiment are shown in Table 5.3. From Table 5.3, we can see that the G-Sign algorithm remains the fastest among all compared algorithms under the time-varying setting. From Fig. 5.6 and the run-time comparisons, we conclude that the G-Sign algorithm is able to track a time-varying graph signal under non-Gaussian noise in a time-efficient manner.

Table 5.3 Run Time Comparison of Time-varying Temperature Predictions

	SαS	Cauchy	Student's t	Laplace
GLMS	0.0122(s)	0.0125(s)	0.0125(s)	0.0127(s)
GLMP	0.0146	0.0137(s)	0.0145(s)	0.0147(s)
GNLMP	0.0368	0.0148(s)	0.0357(s)	0.0354(s)
G-Sign	0.0033(s)	0.0035(s)	0.0035(s)	0.0027(s)

5.8.4 GSD and G-Sign Comparison

The GSD algorithm is compared to the G-Sign to investigate the differences between the two algorithms. We also included GLMS^[35], GNLMP, and the Graph Diffusion LMS (GDLMS)^[30] in the experiments. The dataset used in this experiment is also the hourly meteorological data recorded across the U.S. at 205 weather stations, but instead of temperature, we are forecasting the wind speed^[125]. Doing so will also test the capability of the adaptive GSP algorithms in different applications. We measure the NMSD over time to observe the time-varying behaviors of the tested algorithms:

$$\text{NMSD}[t] = \frac{1}{N} \sum_{i=1}^N \frac{(\hat{\mathbf{x}}[t] - \mathbf{x}[t])_i^2}{(\mathbf{x}[t])_i^2}. \quad (5.34)$$

The wind speed data is under $S\alpha S$ noise with $\alpha = 1.3$ and $\gamma = 0.1$. An illustration of the output of the tested algorithms is shown in Fig. 5.7(a), and the NMSD[t] is shown in Fig. 5.7(b). The run time of the experiments is shown in Table 5.4.

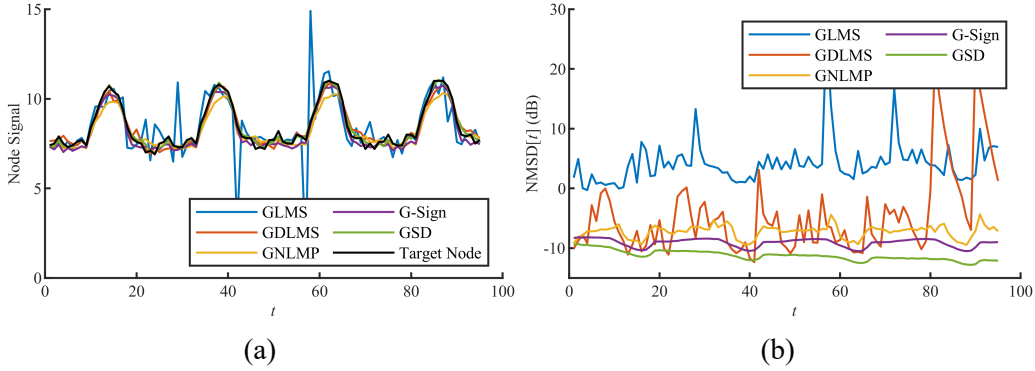


Figure 5.7 Wind speed prediction using adaptive graph algorithms. (a) Reconstruction of one node. (b) The NMSD over time.

From Fig. 5.7, we can see that the GLMS and GDLMS have poor prediction results, reflected by the unstable NMSD. This is because the least squares are unstable under impulsive noise^[58]. Both G-Sign and GSD wind speed predictions are stable because of the l_1 -norm optimization. The GSD and G-Sign perform similarly, which can be observed from Fig. 5.7. This means that even though GSD has a spatial diffusion update strategy, it is effective at accomplishing the tasks designed for the spectral G-Sign. In fact, GSD performs slightly better than the G-Sign if we look at the NMSD. This is because real-world signals are not perfectly bandlimited, but the G-Sign assumes the signals are bandlimited. Additionally, two weather stations that are close to each other are likely to have similar recordings, which means that diffusing known values to missing nodes is

expected to diffuse a value that is close to the ground truth. Further analyzing the runtime of the experiment, we see that both G-Sign and GSD are faster than the other algorithms, which confirms that the Sign update has low computational complexity. GSD has lower runtime than G-Sign, indicating the lower computational complexity of GSD.

Table 5.4 Run Time Comparison of Wind Speed Experiments

GLMS	GDLMS	GNLMP	G-Sign	GSD
0.0194(s)	0.0189(s)	0.0341(s)	0.0052(s)	0.0036(s)

5.9 Discussion and Future Work

In this Chapter, we analyzed the GNLMP algorithm and the G-Sign algorithm for online estimation of partially observed steady-state and time-varying graph signals under impulsive noise. Both algorithms are derived using the minimum dispersion criterion, which is stable and robust under impulsive noise. Experiments conducted on a real-world graph of time-varying temperature using the G-Sign algorithm confirm that it has low computational complexity, good time efficiency, and high robustness under impulsive noise. Here, in Table 5.5, a summary of the graph adaptive filters discussed in this dissertation is presented, listing the characteristics and advantages of each algorithm. Currently,

Table 5.5 A List of Adaptive Filters on Graphs

	Computational Complexity	Robustness	Spatial or Spectral
GLMS ^[35]	Low	Low	Spectral
GLMP ^[59]	High	High	Spectral
GNLMS ^[36]	Low	Low	Spectral
GNLMP	High	High	Spectral
GRLS ^[132]	High	Low	Spectral
G-Sign	Low	High	Spectral
GDLMS ^[30]	Low	Low	Spatial
GSD	Low	High	Spatial

we are expecting three extensions of this line of work. The first extension is to add non-linearity to the adaptive GSP algorithms to derive the Neural Network version of these two algorithms. To give a preview, our current approach is to use the same l_p -norm optimization problem in (5.9) as a generalized algorithm, and specify $p = 1, 1 < p < 2$,

and 2 as Sign-GNN, LMP-GNN, and LMS-GNN. Instead of predefining the filter, we use neural networks to train the filter. The idea is to obtain a filter that has frequency bands more closely aligned with the ground truth data than a predefined filter while keeping the online update and denoising abilities of the adaptive algorithms. We are also in the process of generalizing the G-Sign and GNLMP algorithms using the concept of Message Passing. The third extension of this work is to use Topological Signal Processing (TSP) tools shown in 3 to define G-Sign and GNLMS algorithms on higher-order structures.

CHAPTER 6 ADAPTIVE ESTIMATION OF TIME-VARYING INTERACTING SIMPLICIAL SIGNALS

In Chapter 1, we mentioned that there are four major challenges that impede the effectiveness of classical signal processing algorithms on multivariate time-varying signals that reside on topological structures: the representation of topological irregularity, the estimation of unknown data based on known data, the removal of noise present in the data, and the extraction of the cross-space-time time-variation. In this chapter, we will be discussing one of our recently completed works, which is a framework named Joint Online Estimation (JOE), as a strategy aimed at addressing these four challenges all at once^[145]. We will also discuss a less effective workaround proposed by us that uses GSP to process the signals on the graph edges, named Line Graph Least Mean Squares (LGLMS). The LGLMS will be compared against JOE in the experiment section, and we will be discussing the pros and cons of using each approach.

6.1 Introduction

Despite the success of GSP, data in GSP is solely on the graph nodes, meaning that higher-order signals, such as signals on graph edges, are not being represented. Examples of signals existing on the graph but not only on the graph nodes can be found in the following literature: traffic data^[63,65], gene interaction^[64], multi-agent systems^[65]. In existing GSP algorithms that process time-varying node signals, the interaction among the nodes is modeled using fixed edge weights at all time instances. Failing to capture the time-varying interaction would lead to an inaccurate estimation. Therefore, we need a joint online estimation framework addressing time-varying, multi-order, and multi-variate data to bridge multi-order space-time gaps.

The recently emerged Topological Signal Processing (TSP) has established an alternative point of view from GSP, overcoming the limitation of GSP by generalizing graphs to simplicial complexes. The signals on the nodes of a graph are considered as signals of the 0^{th} order simplex, while higher-order simplices could be used to represent signals on the edges and beyond^[5-6]. Semi-supervised learning for data on the graph edges is done using the Hodge Laplacians and flow conservation^[63]. An analogy of spectral wavelets on simplicial complexes called Hodgelets was proposed^[67]. The topological Slepians

are the signals concentrated on the topological domain and localized in the frequency domain^[68]. Concepts like finite impulse response filters and the convolution operation for signals on a simplicial complex are also introduced through TSP^[69,71,146]. A Chebyshev polynomial approximation of filters on simplicial complexes can be provided^[71-72]. Recently, discussions on online water flow imputation on graph edges were attempted strictly using flow conservation^[73]; the simplicial vector autoregressive (SVAR) model was proposed on the same water flow imputation task^[74].

The above-mentioned Hodge Laplacian-based algorithms demonstrated good performance at processing signals on graph edges, but we may encounter additional challenges in more complicated situations. First and foremost, prior TSP and GSP literature primarily focused on signals in a single simplicial order. The consideration of processing signals that reside in multiple orders of simplices needs to gain more attention because, in reality, signals may appear and interact in multiple orders of simplices. Additionally, a deeper discussion should be conducted on time-varying signal models using the Hodge Laplacian or TSP due to the fact that numerous real situations require online time-varying data models; static algorithms may not perform optimally to capture the dynamic change in the time-varying data. On a side note, data gathered from the real world is often noisy, meaning that the denoising of simplicial data should be considered when formulating simplicial representations. These challenges comprise addressing topological irregularities, revealing data interactions across multiple dimensions, eliminating data noise, and capturing time-varying patterns. Therefore, it is a pressing necessity for an online algorithm that focuses on time-varying multi-order and multi-variate data on simplicial complexes to bridge the cross-space-time signal gaps. Our recently proposed JOE framework is a strategy aimed at addressing the above-mentioned focus.

6.2 Joint Online Estimation of Two Interacting Simplicial Signals

We propose a joint estimation strategy for a simplicial diffusion data model on a simplicial complex in which the time-varying $\mathcal{X}_{k-1}$ signal is influenced by the time-varying $\mathcal{X}_k$ signal. For instance, we can consider a time-varying regression graph for gene expressions formed in^[64], where a series of graphs is formed with dynamic edges representing the interactions of the time-varying gene expression levels on the nodes. In a multi-agent system, agent interactions could be formulated using a graph with time-varying weights^[147]. Four TSP-based aggregation terms are provided along with an algorithm

named the Hodge-NLMS algorithm for online simplicial signal estimation. An illustrative analysis of the JOE framework is carried out using a specific example of interacting time-varying node and edge signals on a graph $\mathcal{G}[t]$ where JOE is accomplished by an alternating estimation strategy between $\mathcal{X}_1$ and $\mathcal{X}_0$.

6.2.1 Time-varying Simplicial Regression

In the JOE framework, a TSP approach is taken to represent data interactions that occur between two simplices of order k and $k - 1$. We view the influence of $\mathcal{X}_k$ onto $\mathcal{X}_{k-1}$ as a time-varying regression in the form of the function value (k -simplicial signal) on topologies. To begin, we will formulate a time-varying regression on $\mathcal{X}_{k-1}$ with the regression parameters represented by time-varying k -simplicial signal $\mathbf{x}_k[t]$:

$$\mathbf{x}_{k-1}[t+1] = \hat{\mathbf{L}}_{k-1}[t]\mathbf{x}_{k-1}[t] = \hat{\mathbf{B}}_{k-1}^T \hat{\mathbf{B}}_{k-1} \mathbf{x}_{k-1}[t] + \hat{\mathbf{B}}_k \hat{\mathbf{B}}_k^T \mathbf{x}_{k-1}[t], \quad (6.1)$$

where $\hat{\mathbf{L}}_{k-1}[t]$ is a time-varying regression matrix in the form of a topology containing a simplicial signal. The term $\hat{\mathbf{B}}_{k-1}$ is given as

$$\hat{\mathbf{B}}_{k-1} = (\mathbf{B}_{k-1} \text{diag}(\text{sign}(\mathbf{x}_k[t]) \circ \sqrt{|\mathbf{x}_k[t]|})), \quad (6.2)$$

where $\text{diag}(\mathbf{x}_k[t])$ forms a diagonal matrix from the vector $\mathbf{x}_k[t]$ and $\text{sign}(\cdot)$ computes the elementwise sign. The term $\hat{\mathbf{B}}_k$ can be defined in a similar manner. Notice that when representing signals in the graph nodes, $(k - 1) = 0$, so there are only upper adjacencies and $\mathbf{B}_0$ is undefined. Consequently, the term $\hat{\mathbf{B}}_{k-1}^T \hat{\mathbf{B}}_{k-1} \mathbf{x}_{k-1}[t]$ should be dropped in (6.1). Similarly, for $k - 1 = K$, $\hat{\mathbf{B}}_k \hat{\mathbf{B}}_k^T \mathbf{x}_{k-1}[t]$ is dropped. The regression matrix $\hat{\mathbf{L}}_{k-1}[t]$ should not be perceived as a weighted version of the Laplacian matrix $\mathbf{L}_{k-1}$. We result in a time-varying influence of $\mathbf{x}_k[t]$ on $\mathbf{x}_{k-1}[t]$ in the form of time-varying regression matrix $\hat{\mathbf{L}}_{k-1}[t]$ that models the change of $\mathbf{x}_{k-1}[t]$.

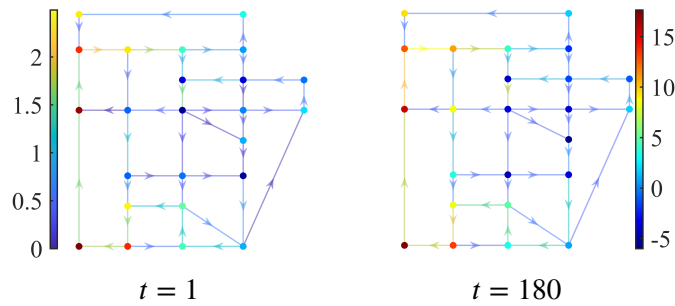


Figure 6.1 A graph with time-varying signals on both the nodes and the edges. (left color bar: edges, right color bar: nodes)

A diffusion process on a simplex is when simplicial signals propagate through the

adjacency relationships to their locally connected neighbors^[148]. If we break the matrix multiplication in (6.1) down, we will see that

$$x_i[t + 1] = \sum_{j=1}^N \hat{l}_{ij}[t] x_j[t], \quad (6.3)$$

where $\hat{l}_{ij}$ is the i^{th} row and j^{th} column of $\hat{\mathbf{L}}_k[t]$, and $x_i[t]$ is the i^{th} element of $\mathbf{x}_k[t]$. Realize that in $\hat{\mathbf{L}}_k[t]$, $\hat{l}_{ij}$ is non-zero only when there are two adjacent elements. So, each summation in equation (6.3) essentially calculates the signal at x_j minus all the signals from all the upper and lower adjacent neighbors that are 1-hop away. If we repeat (6.3) for all N_k elements of $\mathcal{X}_k$, each element will be aggregated with its 1-hop adjacent neighbors. If we look at (6.3) from another aspect, it essentially means that all the elements will be diffused to their connected neighbors, meaning that the aggregation done in (6.3) is equivalent to a simplicial diffusion. Following this logic, the p^{th} power of $\hat{\mathbf{L}}_{k-1}$ will be a diffusion of the p -hop neighbors, which gives us a p -hop version of the time-varying regression in (6.1):

$$\hat{\mathbf{H}}_{k-1}[t] = \sum_{p=0}^P \hat{\theta}_p \hat{\mathbf{L}}_{k-1}^p[t], \quad (6.4)$$

where $\hat{\theta}_p$ is the polynomial coefficient. The diffusive nature of this formulation means that missing data on the simplicial complex can be imputed from neighborhood connections as the diffusion progresses. Moreover, when the weights in (6.4) are defined based on a properly designed simplicial filter, simplicial denoising can be achieved with this formulation; a spectral analysis will be conducted in the next subsection as a guide for how to define the weights. Plugging (6.4) into (6.1), the following signal model is then established:

$$\mathbf{x}_{k-1}[t + 1] = \hat{\mathbf{H}}_{k-1}[t] \mathbf{x}_{k-1}[t] + \epsilon_{k-1}[t], \quad (6.5)$$

where $\mathbf{H}_{k-1}[t]$ is the time-varying p -hop diffusion of the $k-1$ -simplicial signals influenced by the k -simplicial signals.

Here in (6.5), $\epsilon_{k-1}[t]$ is the noise term modeled using zero-mean i.i.d. Gaussian noise in the spatial topology elements and time snapshots. For simplicity, we assume that the topology (the connection between nodes) is known, but the signals on the simplices are noisy or unknown at the current time t . Preferably, the topology does not change over time. We will also assume that there are no unsampled simplicial signals in simplicial order $k-1$; the discussion on TSP sampling techniques can be found in^[5]. An example

of synthetic data based on a real-world traffic network of the above data model is shown in Fig. 6.1, where time-varying signals are generated on both the graph nodes and the edges.

6.2.2 Spectral Analysis of the Time-varying Simplicial Regression

For $\mathcal{X}_{k-1}$ signals, the TSP spectral techniques enable us to tune the algorithm based on a specific frequency band $\mathcal{F}_{k-1}$. In addition, assuming that the designed filter is close to the actual spectrum of the data, using this filter will lead to good denoising performance^[5].

One approach to form the regression shown in (6.1) from the spectral simplicial filters is to use the SFT in (3.16) to apply the simplicial filter Σ_{k-1} to the time-varying regression $\hat{\mathbf{L}}_{k-1}[t]$ as shown in (3.19). Let us represent the simplicial filter Σ_{k-1} using a function $h(\Lambda_{k-1}[t]) = \Sigma_{k-1}$. Then, the regression in (6.1) in terms of the simplicial spectral filtering in (3.19) is

$$\hat{\mathbf{U}}_{k-1}[t]h(\Lambda_{k-1}[t])\hat{\mathbf{U}}_{k-1}^T[t]\mathbf{x}_{k-1}[t] = h(\hat{\mathbf{L}}_{k-1}[t])\mathbf{x}_{k-1}[t]. \quad (6.6)$$

Slightly abusing the notation by omitting the subscripts $k-1$, to achieve a reconstruction similar to (3.25), after some algebraic manipulation, the resulting matrix $h(\hat{\mathbf{L}}_{k-1}[t])$ has expression

$$\hat{\mathbf{H}}_{k-1}[t] = \hat{\mathbf{U}}_{\mathcal{F}}[t](\hat{\mathbf{U}}_{\mathcal{F}}[t]^T \mathbf{D}_{\mathcal{S}}[t] \hat{\mathbf{U}}_{\mathcal{F}}[t])^{-1} \mathbf{U}_{\mathcal{F}}[t]^T, \quad (6.7)$$

where the matrix $\hat{\mathbf{U}}_{\mathcal{F}}[t]$ is the band-limited eigenvector matrix of $\hat{\mathbf{L}}_{k-1}[t]$ with frequency band defined in $\mathcal{F}_{k-1}$.

Now, to use (6.7) to express (6.4) in the spectral domain, we approximate the p -hop regression $\hat{\mathbf{H}}_{k-1}[t]$ in (6.4) using a shifted Chebyshev polynomial $T_p(\mathbf{L}_{k-1}[t])$ defined over the interval $[0, \lambda_N]$, where λ_N is the largest eigenvalue of $\hat{\mathbf{L}}_{k-1}[t]$. Assuming that the spatial operations are conducted in the k -simplices of simplicial complex $\mathcal{K}$, the shifted Chebyshev polynomial $T_p(\hat{\mathbf{L}}_k[t])$ is defined as

$$T_p(\hat{\mathbf{L}}_k[t]) = \begin{cases} 1, & p = 0 \\ \frac{2\hat{\mathbf{L}}_k[t] - \lambda_N}{\lambda_N}, & p = 1 \\ \frac{4\hat{\mathbf{L}}_k[t] - 2\lambda_N}{\lambda_N} T_{p-1}(\hat{\mathbf{L}}_k[t]) - T_{p-2}(\hat{\mathbf{L}}_k[t]), & p \geq 2. \end{cases} \quad (6.8)$$

The approximation $h(\hat{\mathbf{L}}_{k-1}[t])$ in (6.6) is

$$h(\hat{\mathbf{L}}_{k-1}[t]) \approx \hat{\mathbf{H}}_{k-1}[t] = \frac{\theta_0}{2} + \sum_{p=1}^P \theta_p T_p(\hat{\mathbf{L}}_{k-1}[t]). \quad (6.9)$$

In (6.9), the constant P is the order of the polynomial, and we use θ_p to denote the coef-

ficients of the Chebyshev polynomial. If we explicitly write out (6.9) and rearrange the terms, we see that (6.9) becomes (6.4).

There is a limitation to using spectral methods in practice. The simplicial filter Σ_{k-1} is defined by the frequency set $\mathcal{F}_{k-1}$ operating on the frequencies (eigenvalues) of regression matrix $\hat{\mathbf{L}}_{k-1}[t]$. This means that the spectral filtering operation in (6.7) requires the eigendecomposition at every time instance when the signals $\mathbf{x}_k[t]$ change due to the time-varying formulation in (6.1). In GSP, the eigendecomposition of the graph Laplacian suffers from high computational cost and numerical instabilities when the graph topology is large^[31]. The same limitation can be found in TSP for the Hodge-Laplacian^[72]. Thus, the spatial formulations are preferred in practice, while the spectral formulations provide a signal processing analytical perspective. Similar approximation approaches can be seen in^[72] for denoising filters on static simplicial signals and for defining simplicial convolution filters^[71], but they are approximating the simplicial filtering (3.19), while we are approximating the time-varying p -hop regression matrix (6.4).

6.2.3 The Hodge NLMS Algorithm

Even though the model in (6.5) provides us with diffusion-based time-varying simplicial regressions under the influence of simplicial signals, the formulations are derived purely using non-adaptive TSP, which is not optimized to capture the time-varying dynamics of the signal. In classical signal processing, a simple yet efficient adaptive model can be used to represent the time-varying dynamics of signals:

$$\mathbf{x}_k[t+1] = \mathbf{x}_k[t] + \Delta_k[t], \quad (6.10)$$

where $\Delta_k[t]$ is the change that lead $\mathbf{x}_k[t]$ to $\mathbf{x}_k[t+1]$. Thus, it would be more advantageous for the JOE to operate based on an algorithm that is optimized in the time dimension as well.

Prior to completing the JOE framework, we would like to propose a novel online adaptive filtering algorithm named the Hodge-NLMS algorithm for online time-varying simplicial signal prediction on the k -simplices. Let us first consider a simplicial complex $\mathcal{K}$ with time-varying signal $\mathbf{x}_k[t]$, where $t = 1, \dots, T$ is the time index. Suppose we have another signal $\mathbf{y}_k[t]$, which is the observation of $\mathbf{x}_k[t]$ that is noisy and has missing elements. This signal $\mathbf{y}_k[t]$ is modeled using a sampling operation on $\mathbf{D}_{k,S}$ and an i.i.d. additive Gaussian noise $\boldsymbol{\eta}_k[t]$:

$$\mathbf{y}_k[t] = \mathbf{D}_{k,S}(\mathbf{x}_k[t] + \boldsymbol{\eta}_k[t]). \quad (6.11)$$

We could set up a cost function $\mathbf{J}(\hat{\mathbf{x}}_k[t])$ that minimizes the squared error between $\mathbf{y}_k[t]$ to recover $\mathbf{x}_k[t]$ due to the fact that an l_2 -norm optimization problem yields the optimal solution under Gaussian noise assumption:

$$\mathbf{J}(\hat{\mathbf{x}}_k[t]) = \mathbb{E} \|\mathbf{y}_k[t] - \mathbf{D}_{k,S} \mathbf{U}_{k,\mathcal{F}} \mathbf{U}_{k,\mathcal{F}}^T \hat{\mathbf{x}}_k[t]\|_2^2 + g(\hat{\mathbf{x}}_k[t]), \quad (6.12)$$

where $\hat{\mathbf{x}}_k[t]$ is the estimated bandlimited signal with the same frequency bands as $\mathbf{x}_k[t]$, and $g(\hat{\mathbf{x}}_k[t])$ is a convex cost function of a task-specific simplicial aggregation. Combining bandlimitedness of $\hat{\mathbf{x}}_k[t]$, the idempotent property of $\mathbf{D}_{k,S}$, and the TSP reconstruction of sampled signals in (3.25), the term $\mathbf{y}_k[t] - \mathbf{D}_{k,S} \mathbf{U}_{k,\mathcal{F}} \mathbf{U}_{k,\mathcal{F}}^T \hat{\mathbf{x}}_k[t]$ in (6.12) can be written as $\mathbf{D}_{k,S}(\mathbf{y}_k[t] - \mathbf{H}_k \mathbf{D}_{k,S} \hat{\mathbf{x}}_k[t])$. We can formulate the cost function (6.12) as a convex least-squares optimization problem

$$\min_{\hat{\mathbf{x}}_k[t]} \mathbf{J}(\hat{\mathbf{x}}_k[t]), \text{ s.t. } \mathbf{U}_{k,\mathcal{F}} \mathbf{U}_{k,\mathcal{F}}^T \mathbf{x}_k[t] = \mathbf{x}_k[t]. \quad (6.13)$$

The solution of (6.13) could be found by stochastic gradient approaches seen in classical and graph adaptive filtering^[34,36,59].

In JOE, an essential step in online estimation is to effectively represent change $\Delta_k[t]$ on each simplex using TSP. Looking at (6.13), we can represent $\Delta_k[t]$ as the inverse direction that causes the largest error, which is obtained by the (negative) gradient of $J(\hat{\mathbf{x}}_k[t])$. By letting $\Delta_k[t] = -\frac{\partial \mathbf{J}(\hat{\mathbf{x}}_k[t])}{\partial \hat{\mathbf{x}}_k[t]}$ in (6.10), an adaptive update function based on (6.10) for online simplicial signal estimation on order k using the Hodge Laplacian can be derived as

$$\hat{\mathbf{x}}_k[t+1] = \hat{\mathbf{x}}_k[t] - \frac{\partial \mathbf{J}(\hat{\mathbf{x}}_k[t])}{\partial \hat{\mathbf{x}}_k[t]} = \hat{\mathbf{x}}_k[t] + 2(\mathbf{H}_k \mathbf{e}_k[t] - \mathbf{R}_k \hat{\mathbf{x}}_k[t]), \quad (6.14)$$

where $\mathbf{e}_k[t] = \mathbf{D}_{k,S}(\mathbf{y}_k[t] - \hat{\mathbf{x}}_k[t])$ is the estimation error of all sampled part of the signal, $\mathbf{H}_k$ is the same as in (3.25), and $\mathbf{R}_k \hat{\mathbf{x}}_k[t] = \frac{\partial g(\hat{\mathbf{x}}_k[t])}{\partial \hat{\mathbf{x}}_k[t]}$. A step-size parameter is assigned to the update term to control the magnitude of the update, as conventionally seen in classical adaptive filters. The **spectral** update function of our Hodge-NLMS algorithm is derived as

$$\hat{\mathbf{x}}_k[t+1] = \hat{\mathbf{x}}_k[t] + \mu_k \mathbf{H}_k \mathbf{e}_k[t] + r_k \mathbf{R}_k \hat{\mathbf{x}}_k[t], \quad (6.15)$$

where μ_k is the step size parameter and r_k is the weight parameter for the aggregation.

It can be shown that the Hodge-NLMS algorithm in (6.15) converges. For ease of analysis, we will not be using the polynomial approximation of $\mathbf{H}_k$ shown in (6.9) in this analysis but rather use the spectral definition $\mathbf{H}_k = \mathbf{U}_{k,\mathcal{F}}(\mathbf{U}_{k,\mathcal{F}}^T \mathbf{D}_{k,S} \mathbf{U}_{k,\mathcal{F}})^{-1} \mathbf{U}_{k,\mathcal{F}}^T$. Suppose that we have a clean simplicial signal $\mathbf{x}_k$, the update error $\tilde{\mathbf{x}}_k[t]$ at time t is the difference

between the current estimation $\hat{\mathbf{x}}_k[t]$ and $\mathbf{x}_k$: $\tilde{\mathbf{x}}_k[t] = \mathbf{x}_k - \hat{\mathbf{x}}_k[t]$. The Hodge-NLMS algorithm is stable if the update error converges under steady-state estimation. To begin the proof, we use the SFT to transfer the update function (6.15) in the spectral domain:

$$\tilde{\mathbf{s}}_k[t+1] = \tilde{\mathbf{s}}_k[t] + \mu_k \mathbf{U}_{k,\mathcal{F}}^T \mathbf{H}_k \mathbf{e}_k[t] + r \mathbf{U}_{k,\mathcal{F}}^T \mathbf{R}_k \mathbf{U}_{k,\mathcal{F}} \hat{\mathbf{s}}_k[t], \quad (6.16)$$

Since the aggregation term is recursive based on $\hat{\mathbf{s}}_k[t]$, we will first prove that the update error $\tilde{\mathbf{s}}_k[t]$ converges independently of the aggregation term. The estimation error $\mathbf{e}_k[t]$ can be written in terms of $\tilde{\mathbf{s}}_k[t]$ and $\mathbf{s}_k$ in the spectral domain as $\mathbf{e}_k[t] = \mathbf{D}_S \mathbf{U}_{k,\mathcal{F}} (\mathbf{s}_k + \boldsymbol{\eta}_k[t] - \hat{\mathbf{s}}_k[t])$. Then, dropping the aggregation, (6.16) is rearranged into

$$\tilde{\mathbf{s}}_k[t+1] = \boldsymbol{\Xi} \tilde{\mathbf{s}}_k[t] + \mu_k \mathbf{U}_{k,\mathcal{F}}^T \mathbf{H}_k \mathbf{D}_{k,S} \boldsymbol{\eta}_k[t], \quad (6.17)$$

where $\boldsymbol{\Xi} = (\mathbf{I} - \mu_k \mathbf{U}_{k,\mathcal{F}}^T \mathbf{H}_k \mathbf{D}_{k,S} \mathbf{U}_{k,\mathcal{F}})$. Equation (6.17) can be expressed as a recursive update starting from $t = 0$:

$$\tilde{\mathbf{s}}_k[t+1] = \boldsymbol{\Xi}^t \tilde{\mathbf{s}}_k[0] + \mu_k \sum_{i=1}^t \boldsymbol{\Xi}^{i-1} \mathbf{U}_{k,\mathcal{F}}^T \mathbf{H}_k \mathbf{D}_{k,S} \boldsymbol{\eta}_k[t]. \quad (6.18)$$

Now, we can find the update error in the mean-squared sense by calculating the variance of (6.18):

$$\mathbb{E} \|\tilde{\mathbf{s}}_k[t+1]\|^2 = \mathbb{E} \|\tilde{\mathbf{s}}_k[0]\|_{\boldsymbol{\Phi}'}^2 + \mu_k^2 \mathbb{E} \sum_{i=0}^{t-1} (\mathbf{U}_{k,\mathcal{F}}^T \mathbf{H}_k \mathbf{D}_{k,S} \boldsymbol{\eta}_k[t])^T \boldsymbol{\Phi}^i (\mathbf{U}_{k,\mathcal{F}}^T \mathbf{H}_k \mathbf{D}_{k,S} \boldsymbol{\eta}_k[t])^2. \quad (6.19)$$

For notation simplicity, we set $\boldsymbol{\Phi} = \boldsymbol{\Xi}^T \boldsymbol{\Xi}$ and use the the weighted Euclidean norm $\|\tilde{\mathbf{s}}_k[0]\|_{\boldsymbol{\Phi}'} = \|\tilde{\mathbf{s}}_k[0]\|^T \boldsymbol{\Phi}' \tilde{\mathbf{s}}_k[0]\|$ in (6.19). Then, we rearrange some terms by using the Trace trick $\mathbb{E} (\mathbf{X}^T \mathbf{Y} \mathbf{X}) = \text{Tr}(\mathbb{E} (\mathbf{X} \mathbf{X}^T \mathbf{Y}))$, leading to

$$\mathbb{E} \|\tilde{\mathbf{s}}_k[t+1]\|^2 = \mathbb{E} \|\tilde{\mathbf{s}}_k[0]\|_{\boldsymbol{\Phi}'}^2 + \mu_k^2 \text{Tr} \left(\sum_{i=0}^{t-1} \boldsymbol{\Psi} \boldsymbol{\Phi}^i \right), \quad (6.20)$$

where $\boldsymbol{\Psi} = \mathbf{U}_{k,\mathcal{F}}^T \mathbf{H}_k \mathbf{D}_{k,S} \mathbf{C}_k \mathbf{H}_k^T \mathbf{U}_{k,\mathcal{F}}$ and $\mathbf{C} = \text{cov}(\boldsymbol{\eta}, \boldsymbol{\eta})$. With some further derivations using two properties, $\text{Tr}(\mathbf{Y} \mathbf{X}) = \text{vec}(\mathbf{X}^T) \text{vec}(\mathbf{Y})$ and $\text{vec}(\mathbf{X} \mathbf{Y} \mathbf{Z}) = (\mathbf{Z}^T \otimes \mathbf{X}) \text{vec}(\mathbf{Y})$, we can factor out the terms independent of the summation index i from (6.20). The summation in (6.20) becomes a geometric series when the condition $\|\boldsymbol{\Xi}\| < 1$ is satisfied, then

$$\mathbb{E} \|\tilde{\mathbf{s}}_k[t+1]\|^2 = \mathbb{E} \|\tilde{\mathbf{s}}_k[0]\|_{\boldsymbol{\Phi}'}^2 + \mu_k^2 \text{vec}(\boldsymbol{\Psi})^T \sum_{i=0}^{t-1} \mathbf{Q}^i \text{vec}(\mathbf{I}), \quad (6.21)$$

where $\mathbf{Q} = \boldsymbol{\Xi}^T \otimes \boldsymbol{\Xi}$. Taking the limit with $\lim_{t \rightarrow \infty} \mathbb{E} \|\tilde{\mathbf{s}}_k[t+1]\|^2$ and combining the condition of $\|\boldsymbol{\Xi}\| < 1$, we recognize the the right side of (6.21) converges to a constant.

The only user-defined parameter in the Hodge-NLMS algorithm without aggregation is a positive step size μ_k . Using the properties of l_2 -norm, we can have an inequality $\|\Xi\| < \|\mathbf{I}\| + \mu_k \|(\mathbf{U}_{k,\mathcal{F}}^T \mathbf{H}_k \mathbf{D}_{k,S} \mathbf{U}_{k,\mathcal{F}})\| < 1$. Now, because the sampling matrix $\mathbf{D}_{k,S}$ is idempotent and self-adjoint, the matrix $\mathbf{U}_{k,\mathcal{F}}^T \mathbf{H}_k \mathbf{D}_{k,S} \mathbf{U}_{k,\mathcal{F}}$ is symmetric, and its l_2 -norm will be less or equal to its largest eigenvalue. This means that the Hodge-NLMS algorithm has a bounded update error, and the algorithm converges if μ_k is chosen under the condition

$$0 < \mu_k < \frac{2}{\lambda_{\max}(\mathbf{U}_{k,\mathcal{F}}^T \mathbf{H}_k \mathbf{D}_{k,S} \mathbf{U}_{k,\mathcal{F}})}, \quad (6.22)$$

where $\lambda_{\max}(\cdot)$ calculates the maximum eigenvalue of a matrix. Now, with the selection condition of μ_k provided, let us look back at (6.21). If we choose a smaller value for μ_k , the term Ξ will be large, so the squared error will converge to a smaller value but will take more iterations to converge. On the other hand, if we choose a larger value for μ_k , Ξ is small, and the algorithm converges faster but with higher error.

The Hodge-NLMS algorithm converges with the aggregation term. To see this, an expression similar to (6.17) but with the aggregation can be formulated as

$$\tilde{\mathbf{s}}_k[t+1] = \Xi_r \tilde{\mathbf{s}}_k[t] + \mu_k \mathbf{U}_{k,\mathcal{F}}^T \mathbf{H}_k \mathbf{D}_{k,S} \boldsymbol{\eta}_k[t] + r \mathbf{U}_{k,\mathcal{F}}^T \mathbf{R}_k \mathbf{U}_{k,\mathcal{F}} \mathbf{s}_k, \quad (6.23)$$

where $\mathbf{s}_k = \mathbf{U}_{k,\mathcal{F}}^T \mathbf{x}_k$ and $\Xi_r = (\Xi + r \mathbf{U}_{k,\mathcal{F}}^T \mathbf{R}_k \mathbf{U}_{k,\mathcal{F}})$. Let $\Phi_r = \Xi_r^T \Xi_r$ and $\mathbf{Q}_r = \Xi_r^T \otimes \Xi_r$. Then, taking the same derivation steps as before, the update error in the mean-squared sense is

$$\mathbb{E} \|\tilde{\mathbf{s}}_k[t+1]\|^2 = \mathbb{E} \|\tilde{\mathbf{s}}_k[0]\|_{\Phi_r}^2 + \mathbb{E} \|r \mathbf{U}_{k,\mathcal{F}}^T \mathbf{R}_k \mathbf{U}_{k,\mathcal{F}} \mathbf{s}_k\|^2 + \mu_k^2 \text{vec}(\Psi)^T \sum_{i=0}^{t-1} \mathbf{Q}_r^i \text{vec}(\mathbf{I}). \quad (6.24)$$

The condition for (6.24) to converge is again $\|\Xi_r\| < 1$. Now, using the property of the l_2 -norm, we have the inequality $\|\Xi_r\| < \|\Xi\| + \|r \mathbf{U}_{k,\mathcal{F}}^T \mathbf{R}_k \mathbf{U}_{k,\mathcal{F}}\|$. To be on the safe side, and assuming that $r > 0$, the Hodge-NLMS algorithm converges if (6.22) is met and if we have $\|\Xi\| + r \|\mathbf{U}_{k,\mathcal{F}}^T \mathbf{R}_k \mathbf{U}_{k,\mathcal{F}}\| < 1$. As a result, the range of choice for selecting parameter r is

$$0 < r < \frac{1 - \Xi}{\lambda_{\max}(\mathbf{U}_{k,\mathcal{F}}^T \mathbf{R}_k \mathbf{U}_{k,\mathcal{F}})}. \quad (6.25)$$

6.2.4 Joint Online Estimation Algorithm

In Section 6.2.1, we discussed that our JOE aims to conduct online estimation when the time-varying $\mathcal{X}_k$ signals influence the time-varying $\mathcal{X}_{k-1}$ signals. For simplicity, we will assume the $\mathcal{X}_k$ signals are time-varying, but the dynamics are modeled by time-

invariant aggregation $\hat{\mathbf{H}}_k$ instead of the time-varying regression seen in the $\mathcal{X}_{k-1}$ signals. By setting $\hat{\mathbf{H}}_k = \sum_{p=0}^P \hat{\theta}_{k,p} \mathbf{L}_k^p$ using Chebyshev approximation again, the **spatial** update Hodge-NLMS can then be obtained as:

$$\hat{\mathbf{x}}_k[t+1] = \hat{\mathbf{x}}_k[t] + \mu_k \sum_{p=0}^P \hat{\theta}_{k,p} \mathbf{L}_k^p \mathbf{e}_k[t] + r_k \mathbf{R}_k \hat{\mathbf{x}}_k[t]. \quad (6.26)$$

After obtaining an estimation strategy for the k -simplicial signal $\hat{\mathbf{x}}_k[t]$, we will feed $\hat{\mathbf{x}}_k[t]$ into the regression defined in (6.1) to form the diffusion $\mathbf{H}_k$ defined in (6.4) to define the JOE expression in $\mathcal{X}_{k-1}$. Following the time-varying regression data model in (6.5), we shall replace the matrix $\mathbf{H}_k$ in (6.15) with a time-varying regression $\hat{\mathbf{H}}_{k-1}[t] = \sum_{p=0}^P \hat{\theta}_{k-1,p} (\hat{\mathbf{L}}_{k-1}[t])^p$ to incorporate the influence of the k -simplices onto the $(k-1)$ -simplices. Then, to conduct an estimation on $\mathbf{x}_{k-1}[t]$ when it is under the influence of $\mathcal{X}_k$ signal $\mathbf{x}_k[t]$, the update strategy based on Hodge-NLMS is

$$\hat{\mathbf{x}}_{k-1}[t+1] = \hat{\mathbf{x}}_{k-1}[t] + \mu_{k-1} \sum_{p=0}^P \hat{\theta}_{k-1,p} (\mathbf{L}_{k-1}[t])^p \mathbf{e}_{k-1}[t] + r_{k-1} \mathbf{R}_{k-1} \hat{\mathbf{x}}_{k-1}[t], \quad (6.27)$$

where $\hat{\mathbf{x}}_{k-1}[t]$ is the estimation of signals in $\mathcal{X}_{k-1}$ at time t , and the residual $\mathbf{e}_{k-1}[t] = \mathbf{y}_{k-1}[t] - \mathbf{x}_{k-1}[t]$ is the estimation error.

In JOE, the joint estimation of simplicial signals in $\mathcal{X}_{k-1}$ and $\mathcal{X}_k$ can be done by combining (6.26) and (6.27) into a single update. We define a joint shift operator $\mathbf{H}_J[t]$ to incorporate the adjacent relationship within one simplex order together with the incidence relationship between different orders of simplicity. To formulate the proper JOE frameworks on simplicial complexes that are higher-order compared with graph nodes and edges, we will have matrices defined using TSP techniques:

$$\begin{aligned} \mathbf{H}_J[t] &= \text{blkdiag}(\mu_{k-1} \hat{\mathbf{H}}_{k-1}[t], \mu_k \mathbf{H}_k), \\ \mathbf{R}_{J,S} &= \text{blkdiag}(r_{k-1,S} \mathbf{R}_{k-1,S}, r_{k,S} \mathbf{R}_{k,S}) \text{ and} \\ \mathbf{R}_{J,\bar{S}} &= \text{blkdiag}(r_{k-1,\bar{S}} \mathbf{R}_{k-1,\bar{S}}, r_{k,\bar{S}} \mathbf{R}_{k,\bar{S}}), \end{aligned} \quad (6.28)$$

The other joint terms are defined as

$$\begin{aligned} \hat{\mathbf{x}}_J[t] &= \text{vec}(\hat{\mathbf{x}}_{k-1}[t], \hat{\mathbf{x}}_k[t]), \\ \mathbf{y}_J[t] &= \text{vec}(\mathbf{y}_{k-1}[t], \mathbf{y}_k[t]), \text{ and} \\ \mathbf{e}_J[t] &= \text{vec}(\mathbf{e}_{k-1}[t], \mathbf{e}_k[t]), \end{aligned} \quad (6.29)$$

where $\hat{\mathbf{x}}_J[t]$ is the joint time-varying $\mathcal{X}_k$ and $\mathcal{X}_{k+1}$ signals, $\mathbf{y}_J[t]$ is the joint observations,

and $\mathbf{e}_J[t]$ is the joint estimation error. The update scheme of JOE is constructed as

$$\hat{\mathbf{x}}_J[t+1] = \hat{\mathbf{x}}_J[t] + \mathbf{H}_J[T]\mathbf{e}_J[t] + \mathbf{R}_{J,S}\hat{\mathbf{x}}_J[t] + \mathbf{R}_{J,\bar{S}}\hat{\mathbf{x}}_J[t]. \quad (6.30)$$

The individual node signal and edge signal can be obtained by mapping the corresponding elements in $\mathbf{x}_J[t+1]$ back into $\mathbf{x}_0[t+1]$ and $\mathbf{x}_1[t+1]$. We summarize the above procedures of JOE in Algorithm 6.1. In the next section, we will give a detailed analysis of JOE and discuss how to effectively define the aggregation term $r_{k-1}\mathbf{R}_{k-1}\hat{\mathbf{x}}_{k-1}[t]$ and $\mu_{k,R}\mathbf{R}_k\hat{\mathbf{x}}_k[t]$.

Algorithm 6.1 Joint estimation of signals in $\mathcal{X}_{k-1}$ and $\mathcal{X}_k$

Given: (unweighted) graph topology $\mathcal{G}[t]|_{t=0}$, spectral filters $\mathbf{\Sigma}_{k-1}$ and $\mathbf{\Sigma}_k$
 Initialize $\hat{\mathbf{x}}_{k-1}[0]$ and $\hat{\mathbf{x}}_k[0]$
 Construct $\mathbf{H}_k$ and $\hat{\mathbf{H}}_{k-1}[t]$
 Construct $\mathbf{R}_{k-1}$ and $\mathbf{R}_k$ based on prior knowledge using (6.31), (6.32), (6.34), or (6.35)
 Construct $\mathbf{H}_J[t]$ and $\mathbf{R}_J[t]$ using (6.28)
while there are new observations $\mathbf{y}_{k-1}[t]$ and $\mathbf{y}_k[t]$ **do**
 Define the changed components in $\mathbf{H}_J[t]$ and $\mathbf{R}_J$ again
 Output joint signal $\hat{\mathbf{x}}_J[t+1]$ from $\mathbf{y}_J[t]$ using (6.30)
end while

6.2.5 Simplicial Aggregation

Notice that the JOE update strategy shown in (6.26) and (6.27) are based on the errors $\mathbf{e}_k[t]$ and $\mathbf{e}_{k-1}[t]$, but not on the signals $\mathbf{x}_k[t]$ and $\mathbf{x}_{k-1}[t]$ themselves if ignoring the $r\mathbf{R}_k\hat{\mathbf{x}}_k[t]$ term. Thus, it is advantageous to define aggregation terms based on the underlying signal to acquire the desired restriction effect. For example, if we want to emphasize the property $\mathbf{B}_k^T\mathbf{B}_k\hat{\mathbf{x}}_k[t] = 0$ on the lower adjacency of $\mathcal{X}_k$, then we can define an aggregation term $g(\hat{\mathbf{x}}_k[t]) = \|\mathbf{B}_k\hat{\mathbf{x}}_k[t]\|_2^2$ in the cost function (6.12). Following the optimization steps in (6.14), the resulting aggregation $\mathbf{R}_{k,l}\hat{\mathbf{x}}_k[t]$ in the update equation (6.26) is

$$r_k\mathbf{R}_{k,l}\hat{\mathbf{x}}_k[t] = \frac{\partial g(\hat{\mathbf{x}}_k[t])}{\partial \hat{\mathbf{x}}_k[t]} = 2\nu_k\mathbf{L}_{k,l}\hat{\mathbf{x}}_k[t], \quad (6.31)$$

where $\mu_{k,R} = 2\nu_k$ is the weight for the aggregation. Similarly, we can define the upper adjacent aggregation $\mathbf{R}_{k,u}\hat{\mathbf{x}}_k[t]$ using the same approach by setting $h(\hat{\mathbf{x}}_k[t]) = \|\mathbf{B}_{k+1}^T\hat{\mathbf{x}}_k[t]\|_2^2$:

$$\mathbf{R}_{k,u}\hat{\mathbf{x}}_k[t] = \mathbf{L}_{k,u}\hat{\mathbf{x}}_k[t], \quad (6.32)$$

Let us bring in the concept of Hodge decomposition. A $\mathcal{X}_k$ simplicial signal can be decomposed as follows^[5]:

$$\mathbf{x}_k[t] = \mathbf{x}_{k,H}[t] + \mathbf{B}_k^T\mathbf{x}_{k-1}[t] + \mathbf{B}_{k+1}\mathbf{x}_{k+1}[t]. \quad (6.33)$$

The terms $\mathbf{B}_k^T \mathbf{x}_{k-1}[t]$ and $\mathbf{B}_{k+1} \mathbf{x}_{k+1}[t]$ are the signal components that can be induced from $\mathcal{X}_{k-1}$ and $\mathcal{X}_{k+1}$ signals respectively. The term $\mathbf{x}_{k,H}[t]$ in (6.33) is the harmonic component corresponding to the portion of the signal that cannot be induced from $\mathbf{x}_{k-1}$ or $\mathbf{x}_{k+1}$. It is worth mentioning that these three components are orthogonal^[5]. Inspired by the Hodge decomposition, if we revisit the regression/diffusion defined in (6.5), we can see that at each time instance t , we can also define aggregation based on the boundary and coboundary. Remember that the upper adjacency, lower adjacency, boundary, and coboundary of $\mathcal{X}_k$ can be defined using $\mathbf{B}_k$ and $\mathbf{B}_{k+1}$. For example, if we want to restrict the induced components from $\mathcal{X}_{k+1}$ onto $\mathcal{X}_k$, we can set $g(\hat{\mathbf{x}}_{k+1}[t]) = \|\mathbf{B}_k \hat{\mathbf{x}}_{k+1}[t]\|_1^1$. The resulting boundary aggregation is

$$\mathbf{R}_{k,b} \hat{\mathbf{x}}_k[t] = \mathbf{B}_{k+1} \hat{\mathbf{x}}_{k+1}[t]. \quad (6.34)$$

The coboundary aggregation can be defined in a similar manner:

$$\mathbf{R}_{k,c} \hat{\mathbf{x}}_k[t] = \mathbf{B}_k^T \hat{\mathbf{x}}_{k-1}[t]. \quad (6.35)$$

Note that the above four aggregations can be linearly combined into a single term $\mathbf{R}_k \hat{\mathbf{x}}_k[t]$ in the JOE to achieve the desired effects.

Paying attention to (6.12) and (6.15), we see that the estimation error $\mathbf{e}_k[t]$ is only calculated using the sampled signals. To compensate for this in the aggregation, the aggregation parameters are split as $\mathbf{R}_k = \mathbf{R}_{k,S} + \mathbf{R}_{k,\bar{S}} = \mathbf{D}_{k,S} \mathbf{R}_k + \bar{\mathbf{D}}_{k,S} \mathbf{R}_k$. Then, we assign different weights to $\mathbf{R}_{k,S}$ and $\mathbf{R}_{k,\bar{S}}$, resulting in

$$r_k \mathbf{R}_k \hat{\mathbf{x}}_k[t] = r_{k,S} \mathbf{R}_{k,S} \hat{\mathbf{x}}_k[t] + r_{k,\bar{S}} \mathbf{R}_{k,\bar{S}} \hat{\mathbf{x}}_k[t]. \quad (6.36)$$

The logic behind this setup is that different weights were assigned to the sampled and unsampled signals.

6.2.6 TSP Spectral Analysis of the JOE

Breaking (6.15) down, we see that the update is contributed by the filtering $\mathbf{H}_k$ and aggregation $\mathbf{R}_k$. In order to analyze $\mathbf{H}_k$, we need to rewrite $\mathbf{H}_k$ by using (3.19) and (3.25) into

$$\mathbf{H}_k = \mathbf{U}_k \boldsymbol{\Sigma}_k \mathbf{U}_k^T, \quad (6.37)$$

where $\boldsymbol{\Sigma}_k = \boldsymbol{\Sigma}_{\mathcal{F},k} ((\mathbf{U}_k \boldsymbol{\Sigma}_{\mathcal{F},k})^T \mathbf{D}_{k,S} \mathbf{U}_k \boldsymbol{\Sigma}_{\mathcal{F},k})^{-1} \boldsymbol{\Sigma}_{\mathcal{F},k}$ is the spectral domain filter on the k -simplices we aim to design. There are two strategies we can take from this step: we could either operate on the frequencies components by defining a frequency set $\mathcal{F}$ with a filter

$\Sigma_{F,k}$, or we could consider Σ_k as a whole and directly design the filter Σ_k . Either way, in a classical signal processing perspective, we would apply filter operations by using the frequencies in Λ_k defined by the SFT in (3.16). The frequencies are defined as larger eigenvalues corresponding to high frequencies, so we can define low-pass, high-pass, or band-limited filters similar to how these types of filters are designed in classical signal processing. Each update term $\mu_k \mathbf{H}_k \mathbf{e}_k[t]$ is applying the filter Σ_k to the SFT of the error component $\mathbf{e}_k[t]$. As a result, at each time instance t , the term $\mathbf{H}_k \mathbf{e}_k[t]$ in (6.15) ensures that the update of the Hodge-NLMS algorithm is time-varying and is adaptively generated in the direction opposite to the error.

Even though the aggregation $\mathbf{R}_k$ is task-specifically defined from the properties of the signal of interest, we can still gain some insights by looking at them in the spectral domain. Generally speaking, by the Hodge decomposition in (6.33), the nonzero eigenvectors-eigenvalue pairs in $\mathbf{B}_k \mathbf{B}_k^T$ are orthogonal to the nonzero eigenvectors-eigenvalue pairs in $\mathbf{B}_{k+1}^T \mathbf{B}_{k+1}$; all the nonzero eigenvalue-eigenvector pairs in $\mathbf{L}_k$ are either from $\mathbf{B}_k \mathbf{B}_k^T$ or $\mathbf{B}_{k+1}^T \mathbf{B}_{k+1}$ [5]. This leads to another interesting property $\mathbf{B}_k \mathbf{B}_{k+1} = 0$. Remember that in the Hodge decomposition (6.33), the harmonic component of a simplicial $\mathcal{X}_k$ signal cannot be induced from $\mathcal{X}_{k+1}$ or $\mathcal{X}_{k-1}$; from the orthogonality, the harmonic components are associated with the zero eigenvalue-eigenvector pairs of $\mathbf{L}_k$ [5].

Remarks For a graph with only node signals, setting $k = 0$ reduces TSP to GSP. In the Hodge-NLMS algorithm, the term $(\mathbf{U}_{k,F}^T \mathbf{D}_{k,S} \mathbf{U}_{k,F})^{-1}$ is obtained from (3.25) through the TSP sampling theory. When operating the Hodge-NLMS algorithm on the graph nodes, the term $(\mathbf{U}_{k,F}^T \mathbf{D}_{k,S} \mathbf{U}_{k,F})^{-1}$ in $\mathbf{H}_k$ of the update (6.15) resembles the structure of the normalization seen in the GSP normalized LMS (GNLMS) algorithm in [36] and the GSP normalized least mean p^{th} algorithm when $p = 2$ in [10]. However, the normalization in GSP comes from stochastic gradient optimization between node signal estimations at time t and time $t + 1$. This provides a different interpretation of the normalization term in the case of GSP.

6.3 Specifying JOE Onto Graph Nodes and Edges

In this section, the JOE is analyzed given a specific scenario on a graph $\mathcal{G}$, where the time-varying signals on the edges $\mathbf{x}_1[t]$ influence the time-varying signals on the nodes $\mathbf{x}_0[t]$. Following the convention of previous works in [5,71], we will set the order $K = 2$ for TSP on graphs.

6.3.1 Joint Prediction of Edge and Node Signals

To begin the JOE, we use the Hodge-NLMS algorithm in (6.26) to conduct an online estimation of edge signals. Given the topology, the next-step prediction is achieved by setting $k = 1$ when a noisy and partial observation of the edge signal $\mathbf{y}_1[t]$ arrives:

$$\hat{\mathbf{x}}_1[t+1] = \hat{\mathbf{x}}_1[t] + \mu_1 \hat{\mathbf{H}}_1 \mathbf{e}_1[t] + r_1 \mathbf{R}_1 \hat{\mathbf{x}}_1[t], \quad (6.38)$$

where $\mathbf{e}_1[t] = \mathbf{D}_{S,1}(\mathbf{y}_1[t] - \hat{\mathbf{x}}_1[t])$. We proceed with JOE by constructing the node diffusion $\hat{\mathbf{H}}_0[t]$ following (6.1) and (6.4).

For node signal prediction given a noisy observation $\mathbf{y}_0[t]$, we recover the underlying node signal $\mathbf{x}_0[t]$ by replacing $\hat{\mathbf{H}}_{k-1}$ in (6.27) with $\hat{\mathbf{H}}_0[t]$. The online estimation on $\mathbf{x}_0[t]$ becomes

$$\hat{\mathbf{x}}_0[t+1] = \hat{\mathbf{x}}_0[t] + \mu_0 \hat{\mathbf{H}}_0[t] \mathbf{e}_0[t] + r_0 \mathbf{R}_0 \hat{\mathbf{x}}_0[t], \quad (6.39)$$

where $\hat{\mathbf{x}}_0[t]$ is the estimation at time t and $\mathbf{e}_0[t] = \mathbf{y}_0[t] - \mathbf{x}_0[t]$ is the estimation error. We can formulate the joint terms on the nodes and edges again as

$$\begin{aligned} \mathbf{H}_J[t] &= \text{blkdiag}(\mu_0 \hat{\mathbf{H}}_0[t], \mu_1 \mathbf{H}_1), \\ \mathbf{R}_{J,S} &= \text{blkdiag}(r_{0,S} \mathbf{R}_{0,S}, r_{1,S} \mathbf{R}_{1,S}) \text{ and} \\ \mathbf{R}_{J,\bar{S}} &= \text{blkdiag}(r_{0,\bar{S}} \mathbf{R}_{0,\bar{S}}, r_{1,\bar{S}} \mathbf{R}_{1,\bar{S}}). \end{aligned} \quad (6.40)$$

The joint observation, signal, and error terms on the nodes and edges are

$$\begin{aligned} \hat{\mathbf{x}}_J[t] &= \text{vec}(\hat{\mathbf{x}}_0[t], \hat{\mathbf{x}}_1[t]), \\ \mathbf{y}_J[t] &= \text{vec}(\mathbf{y}_0[t], \mathbf{y}_1[t]), \text{ and} \\ \mathbf{e}_J[t] &= \text{vec}(\mathbf{e}_0[t], \mathbf{e}_1[t]). \end{aligned} \quad (6.41)$$

These joint terms are plugged back into the JOE update shown in (6.30).

The JOE operates in an online fashion where the time-varying edge signals and the time-varying node signal are predicted simultaneously using (6.38) and (6.39) at each time instance. The complete procedure is summarized in Algorithm 6.2. It should be emphasized that the JOE discussed here is not simply estimating edge weights from graph node signals nor a link prediction task; the graph edges are regarded as the 1-simplex with signals existing on the edges regardless of the existence of node signals.

6.3.2 TSP Analysis for JOE on Edges and Nodes

When specifying a graph $\mathcal{G}$ as an example of a simplicial complex of order $K = 2$, there are two incidence matrices we are particularly interested in, namely the node-to-edge

Algorithm 6.2 Adaptive joint node-edge estimation

Given the (unweighted) graph topology $\mathcal{G}[t]|_{t=0}$, spectral filters Σ_0 and Σ_1
 Initialize $\hat{\mathbf{x}}_0[0]$ and $\hat{\mathbf{x}}_1[0]$
 Construct $\mathbf{H}_1$ and $\hat{\mathbf{H}}_0[t]$
 Construct $\mathbf{R}_1$ and $\mathbf{R}_0$ based on prior knowledge using (6.31) by specifying them on $k = 1$ or $k = 0$, (6.32), (6.34), or (6.35)
 Construct $\mathbf{H}_J[t]$ and $\mathbf{R}_J[t]$ using (6.40)
while there are new observations $\mathbf{y}_0[t]$ and $\mathbf{y}_1[t]$ **do**
 Define the changed components in $\mathbf{H}_J[t]$ and $\mathbf{R}_J$ again
 Output joint signal $\hat{\mathbf{x}}_J[t + 1]$ from $\mathbf{y}_J[t]$ using (6.30)
end while

incidence matrix $\mathbf{B}_1$ and the edge-to-triangle incidence matrix $\mathbf{B}_2$. The matrix $\mathbf{B}_1$ is a divergence operator because $\mathbf{B}_1 \mathbf{x}_1[t] \in \mathbb{R}^{N_0}$ calculates the difference between the outgoing edge signals and the incoming edge signals at a node. An edge signal $\mathbf{x}_1[t]$ is considered divergence-free if $\mathbf{B}_1 \mathbf{x}_1[t] = 0$ ^[63,69]. The gradient operator $\mathbf{B}_1^T$ on the node signal $\mathbf{x}_0$ calculates the difference between the endpoints in each edge along the edge orientation, inducing a gradient flow $\mathbf{x}_G[t] = \mathbf{B}_1^T \mathbf{x}_0[t] \in \mathbb{R}^{N_1}$ on the edges. The matrix $\mathbf{B}_2^T$ is a curl operator since $\mathbf{B}_2 \mathbf{x}_1[t] \in \mathbb{R}^{N_1}$ computes the net flow of the edges along each triangle. If $\mathbf{B}_2^T \mathbf{x}_1 = 0$ holds true, then the edge signal $\mathbf{x}_1[t]$ is curl-free^[71]. The curl adjoint operator $\mathbf{B}_2$ induces the curl flow onto the edges from the triangles by $\mathbf{x}_C[t] = \mathbf{B}_2 \mathbf{x}_2[t]$. From the Hodge decomposition, aggregations that utilize the four abovementioned operators can be defined using (6.31), (6.32), (6.34). and (6.35).

In the spectral domain, the Hodge decomposition in (6.33) implies that we can modify the signal error $\mathbf{x}_1[t]$ by modifying the harmonic component, the gradient component, and the curl component separately. Based on the Hodge decomposition, the eigenvector matrix $\mathbf{U}_1$ and the corresponding eigenvalue matrix Λ_1 of $\mathbf{L}_1$ in the SFT in (3.16) can be rearranged as shown below^[71]:

$$\begin{aligned} \mathbf{U}_1 &= [\mathbf{U}_H \ \mathbf{U}_G \ \mathbf{U}_C], \\ \Lambda_1 &= \text{blkdiag}(\Lambda_H, \Lambda_G, \Lambda_C), \end{aligned} \tag{6.42}$$

where $\text{blkdiag}(\cdot)$ is the block diagonal concatenation. The matrices $\mathbf{U}_H$, $\mathbf{U}_G$, and $\mathbf{U}_C$ in (6.42) are the eigenvector matrices of the harmonic component, the gradient component, and the curl component, respectively. The diagonal matrices Λ_H , Λ_G , and Λ_C contains the eigenvalues corresponding to the eigenvectors in $\mathbf{U}_H$, $\mathbf{U}_G$, and $\mathbf{U}_C$. Because $\mathbf{L}_1$ can be represented as the sum of $\mathbf{L}_{1,l}$ and $\mathbf{L}_{1,u}$, the following expressions can be derived using

the logic in (6.42)^[71]:

$$\begin{aligned}\mathbf{L}_{1,l} &= \mathbf{U}_1 \text{blkdiag}(\mathbf{0}, \mathbf{\Lambda}_G, \mathbf{0}) \mathbf{U}_1^T, \\ \mathbf{L}_{1,u} &= \mathbf{U}_1 \text{blkdiag}(\mathbf{0}, \mathbf{0}, \mathbf{\Lambda}_C) \mathbf{U}_1^T.\end{aligned}\tag{6.43}$$

Now, by looking at (6.42) and (6.43), it is apparent that by carefully designing the filter $\mathbf{\Sigma}_1$ in (6.37) in the spectral domain, we could achieve the direct modification of the harmonic component, the gradient component, and the curl component of the update $\mathbf{e}_1[t]$ as well as the estimated signal $\hat{\mathbf{x}}_1[t]$.

Generally speaking, in node signal estimation tasks, the aggregation requires a smooth (low-frequency) signal assumption. If $\hat{\mathbf{H}}_0[t]$ is defined based on an approximation of a low-pass filter $\mathbf{\Sigma}_{F,0}$, then it enforces the estimated signal to be smooth^[122]. As a result, the aggregation term $\mathbf{R}_0 \mathbf{x}_0[t]$ can be dropped in (6.39) if a divergence-free assumption also holds for the edge signals.

6.4 Edge Signal Estimation via Line Graph Transformation

Given a graph $\mathcal{G}$, the edge-to-vertex dual of $\mathcal{G}$ is known as the Line Graph of $\mathcal{G}$. We denote the Line Graph as $\mathcal{G}_{LG}$. The adjacency matrix of the Line Graph $\mathcal{G}_{LG}$ can be constructed using the (oriented) node-to-edge incidence matrix $\mathbf{B}_1$:

$$\mathbf{A}_{LG} = \text{abs}(\mathbf{B}_1^T \mathbf{B}_1) - 2\mathbf{I},\tag{6.44}$$

where $\mathbf{I}$ is the identity matrix. In other words, to connect the nodes in the Line Graph $\mathcal{G}_{LG}$, an edge is placed between two nodes of $\mathcal{G}_{LG}$ if their corresponding edges in the original graph $\mathcal{G}$ are connected to the same node. Using the definition in Chapter 3, we can form the Laplacian matrix $\mathbf{L}_{LG}$ for the Line Graph. The edge signals $\mathbf{x}_1[t]$ in the original graph $\mathcal{G}$ is now projected to the node signals $\mathbf{x}_n[t]$ of the line graph $\mathcal{G}_{LG}$. Now, assuming the edge signals are noisy, we could model the noise on the Line Graph nodes too, because it can also be projected onto the Line Graph. Missing observations can be modeled again using a masking or sampling matrix $\mathbf{D}_{S,1}$, where the i^{th} diagonal is an indicator of whether the i^{th} node is missing or not. The signal on the nodes of $\mathcal{G}_{LG}$ has missing value and noise at time t is then $\mathbf{y}_n[t] = \mathbf{D}_{S,1}(\mathbf{x}_n[t] + \boldsymbol{\eta}_0[t])$. Notice that the missing nodes on $\mathcal{G}_{LG}$ mimicked via sampling also follow the GSP node sampling idea to maximize certain desired properties of the signal or to enforce sparsity^[37].

A similar noisy and missing edge signal model $\mathbf{y}_1[t] = \mathbf{D}_{S,1}(\mathbf{x}_1[t] + \boldsymbol{\eta}_1[t])$ is used in LGLMS. In most cases, a graph shift operator is required in GSP for node tasks. Popular

choices are either some variants of the adjacency matrix or some variants of the graph Laplacian matrix^[3]. Choosing $\mathbf{L}_{LG}$ as the graph shift, spectral operations on the edges are defined again on $\mathcal{G}_{LG}$ using the GFT:

$$\mathbf{L}_{LG} = \mathbf{U}_{LG} \mathbf{\Lambda}_{LG} \mathbf{U}_{LG}^T. \quad (6.45)$$

Afterward, we will process the time-varying signals using the following model:

$$\mathbf{x}_1[t+1] = \mathbf{x}_1[t] + \Delta_1[t], \quad (6.46)$$

where $\Delta_1[t]$ represents the change in the edge signal that would lead $\mathbf{x}_1[t]$ to $\mathbf{x}_1[t+1]$. Then, a spectral domain filter $\mathbf{\Sigma}_{LG}$ based on the edge signal smoothness or edge signal bandlimitedness is defined. To obtain $\Delta_1[t]$, we rely on minimizing an l_2 -norm optimization problem similar to what is seen in previous chapters:

$$J(\hat{\mathbf{x}}_1[t]) = \mathbb{E} \left\| \mathbf{y}_1[t] - \mathbf{D}_{S,1} \mathbf{U}_{LG} \mathbf{\Sigma}_{LG} \mathbf{U}_{LG}^T \hat{\mathbf{x}}_1[t] \right\|_2^2. \quad (6.47)$$

Based on the bandlimitedness of the edge signal, we can obtain the solution

$$\Delta_1[t] = \frac{\partial f(\hat{\mathbf{x}}_1[t])}{\partial \hat{\mathbf{x}}_1[t]} = -2 \mathbf{U}_{LG} \mathbf{\Sigma}_{LG} \mathbf{U}_{LG}^T \mathbf{D}_{S,1} (\mathbf{y}_1[t] - \hat{\mathbf{x}}_1[t]) \quad (6.48)$$

Plugging (6.48) into (6.46), the update function of the LGLMS algorithm will produce the next-step edge signal estimation $\hat{\mathbf{x}}_1[t+1]$:

$$\hat{\mathbf{x}}_1[t+1] = \hat{\mathbf{x}}_1[t] + \delta \mathbf{U}_{LG} \mathbf{\Sigma}_{LG} \mathbf{U}_{LG}^T \mathbf{D}_{S,1} (\mathbf{y}_1[t] - \hat{\mathbf{x}}_1[t]), \quad (6.49)$$

where δ is the step size. For simplicity, we can assume the graph $\mathcal{G}$ is fixed. It is worth pointing out that the change in the graph structure will not change the definitions of any GSP techniques; thus, the Line Graph transformation techniques remain valid.

Here is an intuitive explanation of the complete procedure for using the LGLMS to process edge signals. Given a graph $\mathcal{G}$, begin the LGLMS by constructing the Line Graph $\mathcal{G}_{LG}$ as seen in (6.44). Then, in order to conduct spectral domain operations, we define the necessary GFT components not using the original graph $\mathcal{G}$ but using the Line Graph $\mathcal{G}_{LG}$ using (6.45) and define a spectral domain filter $\mathbf{\Sigma}_{LG}$ (preferably a bandlimited close to the ground truth). While there are noisy and missing edge signal observations $\mathbf{y}_1[t]$, iteratively execute the LGLMS algorithm shown in (6.49). The pseudocode for implementing this procedure for online estimation of edge signals using LGLMS is illustrated in Algorithm 6.3.

We should emphasize that in LGLMS we are operating on the edge signals that are projected onto the nodes of the Line Graph, and all operations are defined using the graph

Algorithm 6.3 Estimating edge signals using LGLMS

- 1: Given $\mathcal{G}$, construct its Line Graph $\mathcal{G}_{LG}$ using (6.44)
 - 2: Define the GFT of $\mathcal{G}_{LG}$ and the bandlimited filter Σ_{LG} using (6.45)
 - 3: **while** There is new edge signal observation $\mathbf{y}_1[t]$ **do**
 - 4: Obtain the current edge signal observation $\mathbf{y}_1[t]$
 - 5: Execute the LGLMS algorithm shown in (6.49)
 - 6: **end while**
-

Laplacian of the Line graph $\mathbf{L}_{LG}$; this is fundamentally different from the original GLMS that defines all operations based on the original graph Laplacian $\mathbf{L}$. This is our main advantage in LGLMS: we can utilize well-developed GSP tools to process signals on the graph edges even though these GSP techniques were defined originally on the graph nodes. It is also worth mentioning that the Line Graph transform is not constrained to using only the GLMS. The choice of setting up an l_2 -norm optimization problem is attributed to the straightforward solution and implementation of forming adaptive algorithms for the online estimation of time-varying signals^[34]. The LGLMS is the parallelization of the classical adaptive LMS algorithm on the graph edges. Additionally, the adaptive GLMS algorithm defined on the graph nodes was proved to be stable and convergent under Gaussian noise, meaning that the LGLMS algorithm can inherit this behavior on the line graph nodes but process the edge signals instead.

Remarks One drawback of this approach is that the projected signals may not share the same GSP properties, which hinders the performance of this workaround^[75-76]. Later in the experiments, we will show that even though LGLMS is effective, JOE is able to outperform LGLMS due to having a more effective representation. Moreover, JOE can be generalized to higher-order structures and is being derived for joint estimations, but LGLMS only works for edge signals.

6.5 Experiment Results and Discussion

6.5.1 Experiment Set Up

The JOE proposed in Algorithm 6.2 will be examined for its ability to estimate time-varying node and edge signals under the conditions that the node signal is influenced by the edge signal. Since the online estimation in JOE utilized the Hodge-NLMS as the simplicial signal prediction algorithm, we will conduct preliminary experiments on the Hodge-NLMS for both steady-state and time-varying simplicial signals in the k -simplices prior to the JOE experiments. The datasets that we are going to use are the Sioux Falls

transportation network, the Anaheim transportation network, and the England COVID-19 dataset. The edge signal missing rates for the Sioux Falls network and the Anaheim network are 26% and 36%, respectively. The England COVID-19 dataset is a dynamic network that changes over time, so every newly appeared edge in the topology at time t when compared to the previous topology at $t - 1$ will be treated as an edge with missing signals. All the experiments are conducted in MATLAB 2022a and are repeated $N = 100$ times unless specified otherwise. In the experiment, all the tested algorithms will use low-pass filters. All algorithms running in a given simplex will have the same passband to ensure the fairness of comparison. The performance of all the experiments is measured using the normalized mean squared error (NMSE):

$$\text{NMSE}_k[t] = \frac{1}{N} \sum_{i=1}^N \frac{(\hat{\mathbf{x}}_k[t] - \mathbf{x}_k[t])_i^2}{(\mathbf{x}_k[t])_i^2}, \quad (6.50)$$

where $(\hat{\mathbf{x}}_k[t] - \mathbf{x}_k[t])_i^2$ is the squared error on the i^{th} run.

The topology for the Sioux Falls transportation network and the Anaheim transportation network are two transportation networks gathered from the real world^[15]. The Sioux Falls network consists of $N_0 = 24$ nodes and $N_1 = 38$ edges and the Anaheim network has $N_0 = 406$ nodes and $N_1 = 624$ edges. These two transportation networks have provided static traffic flows on the edges for traffic under equilibrium^[15]. As for the England COVID-19 dataset, it is a series of 61 dynamic graphs from^[149]. All 61 graphs have $N_0 = 129$ nodes, with a different number of edges $N_1[t]$ on each day. The nodes represent regions in England, and the edges represent population mobility between regions; self-loops represent the mobility within the region. The dataset contains a series of time-varying joint simplicial signals on the 0-simplex and the 1-simplex. Each of the 61 graphs records the population mobility on the graph edges and the number of COVID-19 cases on the graph nodes. Figure 6.7 displays the England COVID-19 dataset at three different time points. Even though there is no data on the 2-simplex in any of the datasets, triangles can be represented in $\mathbf{L}_{1,\mu}$ because the topological structures still exist.

6.5.2 Hodge-NLMS Analysis

Since we are using the Hodge-NLMS algorithm as the online estimation algorithm for the JOE framework, we will first conduct a few relatively simple experiments to assess the performance of our Hodge-NLMS algorithm in (6.15). In this experiment, we will be

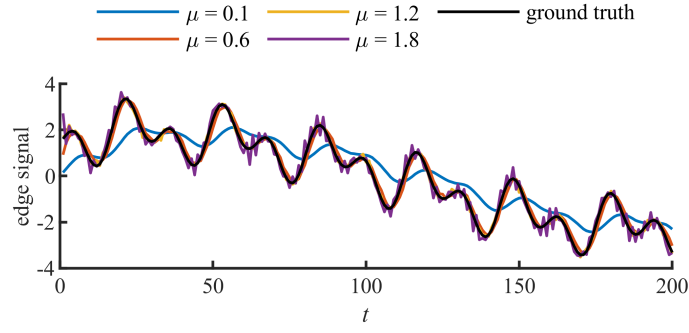
estimating an edge signal in an online procedure on the Sioux Falls network by setting k to 1 for the Hodge-NLMS. We will be testing the Hodge-NLMS algorithm under the following two experimental settings. First, fix the noise level and run the Hodge-NLMS algorithm for 4 different step sizes. Second, fix the step size and run the Hodge-NLMS algorithm for 4 different noise levels. The edge signals are generated synthetically on the Sioux Falls network by combining several sinusoidal functions to make the static signal time-varying:

$$\mathbf{x}_1[t] = \sum_{p=1}^P \mathbf{a}_p f_p[t], \quad (6.51)$$

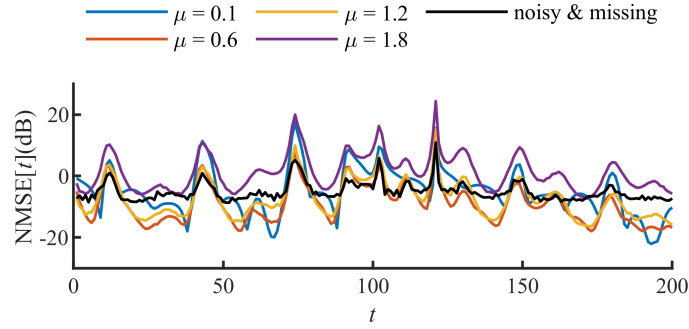
The signals $\mathbf{a}_p$ are static divergence-free edge signals generated using the method in^[63], and $f_p[t]$ is a sinusoidal function. The missing edge signals in the Sioux Falls network are modeled using the spectral sampling strategy discussed in^[36]. We will make online estimations for 200 iterations, where at each iteration we have a new noisy and missing observation of the edge signal. The low-pass filter in 1-simplices has a passband of 50% lowest frequencies. The experiments in this section are repeated 10 times; the averaged edge signal estimations of a few selected edges will be demonstrated along with the NMSE for the entire topology.

For the fixed noise experiment, the noise is a Gaussian distribution with mean = 0 and variance = 0.1; four different step sizes are $\mu_1 = 0.1, 0.6, 1.2$, and 1.8. The signal reconstruction of the signal on one edge and the NMSE on the entire topology are shown in Figure. 6.2(a) and Figure. 6.2(b), respectively. From the edge signal estimation in Figure. 6.2(a), we can see that the step size parameter μ_1 indeed controls the magnitude of the update. For $\mu_1 = 0.1$, the magnitude of the update is too small, which is reflected by visually inspecting Figure. 6.2(a): the estimation does not catch up with the time-varying change in the signal. As for the case when μ_1 is too large ($\mu_1 = 1.8$), the large magnitude causes the estimation to fluctuate and start to become unstable, which can be observed from Figure. 6.2(a) as well. This confirms that μ_1 controls the amount of update of the Hodge-NLMS. Inspecting the NMSE in Figure. 6.2(b), we can gain some insights into the design choice of μ_1 . When comparing Hodge-NLMS to the missing unfiltered signal, the estimation error will be large when μ_1 is either too large or too small.

Now, we will be fixing the step size $\mu_1 = 0.6$ and conducting an online estimation of a time-varying edge signal under four different noise settings. The Gaussian noises all have zero mean but four different variances: $\sigma = 0.1, 0.3, 0.5$, and 0.7. The signal recon-



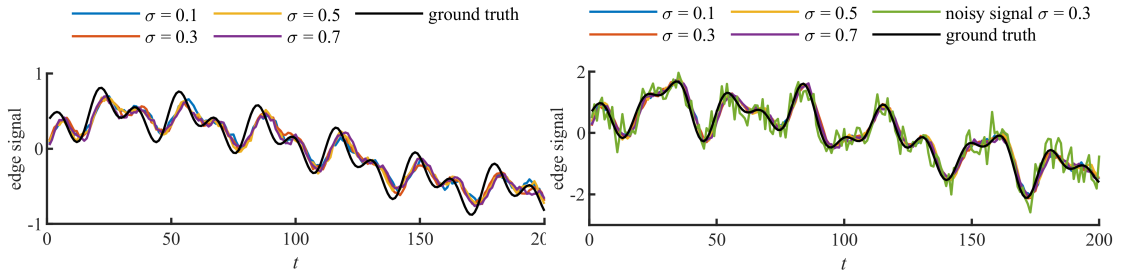
(a) The online estimation of one selected edge.



(b) The total NMSE at each time point for all edge signals.

Figure 6.2 Performance of the Hodge-NLMS under fixed noise and different step sizes.

struction of one missing edge is shown in Fig 6.3(a), and the denoising of one observed edge signal is shown in Fig 6.3(b). Upon examination of Figure. 6.3, we verified that the



(a) Reconstruction of a missing edge signal. (b) Denoising of an observed noisy signal.

Figure 6.3 Performance of the Hodge-NLMS under fixed step size and different noise levels.

Hodge-NLMS can effectively conduct online estimations of time-varying signals on the graph edges under Gaussian noise with various noise levels.

6.5.3 JOE on Transportation Network

In this section, we will be jointly estimating time-varying edge signals and time-varying node signals on the Sioux Falls network and the Anaheim network. The previous two experiments have shown that the Hodge-NLMS algorithm could be applied to online

simplicial estimation tasks. Returning to the question of joint estimation, we will be testing the JOE framework based on Algorithm 6.2 for jointly estimating time-varying node and edge signals by combining JOE with the Hodge-NLMS algorithm.

The edge signal generation and missing data masking follow exactly the same procedure as in 6.5.2. The ground truth node signals are synthetically generated using (6.5) with additive Gaussian noise of mean = 0 and variance = 0.1 on both the Sioux Falls network and the Anaheim network. We should point out that even though the node signal is synthetic, which seems a bit abstract and may not have physical meaning, it is built using properties extracted from the underlying real-world topology. In the context of a transportation network, one may perceive the node signal as a measurement of congestion at an intersection or a representation of the crowd at a place of interest. Since this dissertation is not focused on transportation network models, we will not discuss further the meaning of the node signal and use it purely as the target signal to be estimated. Figure. 6.1 illustrates an example of such a time-varying node signal under the influence of time-varying edge signals on the Sioux Falls network.

To highlight the advantages of using a TSP approach (the Hodge-NLMS) in JOE, instead of using the Hodge-NLMS as in JOE, we will compare Hodge-NLMS against LGLMS shown in Section 6.4. For edge signal estimation baselines, in addition to the LGLMS algorithm baseline, we also included an online non-adaptive TSP baseline using a basic low-pass simplicial filter $\Sigma_{1,lp}$ similar to (3.19):

$$\hat{\mathbf{x}}_1[t+1] = \mathbf{U}_1 \Sigma_{1,lp} \mathbf{U}_1^T \hat{\mathbf{x}}_1[t]. \quad (6.52)$$

The node estimation results of JOE will be compared with the following three GSP baseline algorithms in addition to the LGLMS: the GLMS algorithm^[35], the Graph Adaptive Least Mean p^{th} algorithm (GLMP)^[59], and the Graph adaptive-Sign (G-Sign) algorithm^[9]. Note that, by the algorithm definitions in their original literature, the GSP methods use a fixed Graph Laplacian, so among all the baselines, only the LGLMS algorithm is aware of the change of the edge signals influencing the nodes. The step sizes and the aggregation weights are tuned using grid search; the final parameter selections for LGLMS and JOE are shown in Table 6.2 and Table 6.1.

The average NMSE $[t]$ of the signal estimation results over the 100 runs is recorded in Fig. 6.5 for the Sioux Falls network and in Fig. 6.6 for the Anaheim network. Additionally, an illustration of the estimation results on one of the missing edges and one of the nodes in the Sioux Falls network is shown in Fig. 6.4.

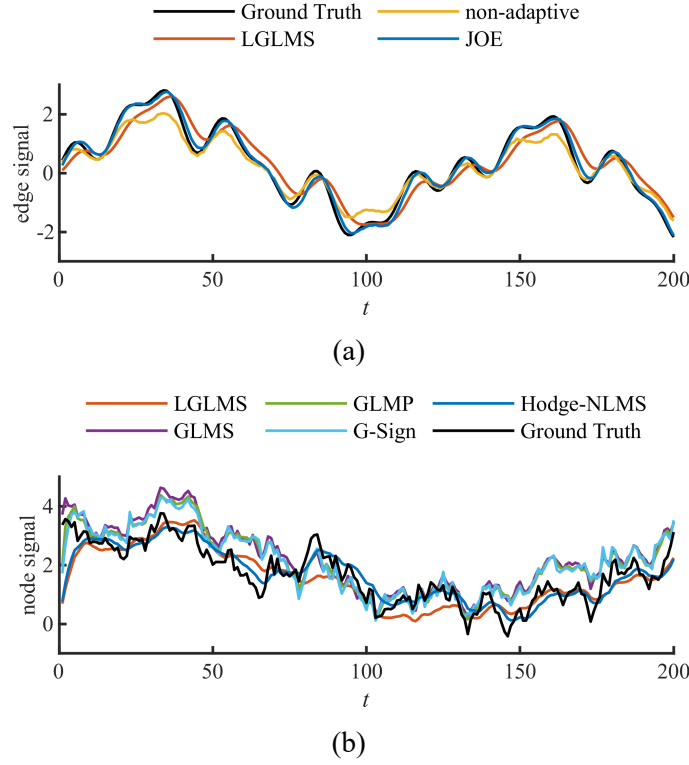


Figure 6.4 JOE compared with separate estimation on one of the edges (a) and one of the nodes (b) of the Sioux Falls network.

Table 6.1 Parameters for LGLMS and JOE in nodes

	μ_0	$\mathcal{F}_0$
Sioux Falls	0.2	$[0, 0.4\lambda_{0,max}]$
Anaheim	0.2	$[0, 0.4\lambda_{0,max}]$

From Fig. 6.5 and Fig. 6.6, JOE has lower NMSE than the two baselines for the majority of the time points. From the low NMSE, we can confirm that the combination of JOE with the Hodge-NLMS algorithm is accurate at tracking the changes of time-varying edge signals. The JOE framework outperforms the LGLMS algorithm because the LGLMS does not operate directly on the edge signal; the LGLMS algorithm treats the original edge signal as node signals on a line graph projection. However, the projected signal in LGLMS is not guaranteed to be smooth on the 0-simplices, so using the GSP nodal op-

Table 6.2 Parameters for LGLMS and JOE in edges

	μ_1	$r_{1,S}$	$r_{\bar{S}}$	$\mathcal{F}_1$
Sioux Falls	0.2	0.02	0.42	$[0, 0.58\lambda_{1,max}]$
Anaheim	1.2	0.1	0.2	$[0, 0.2\lambda_{1,max}]$

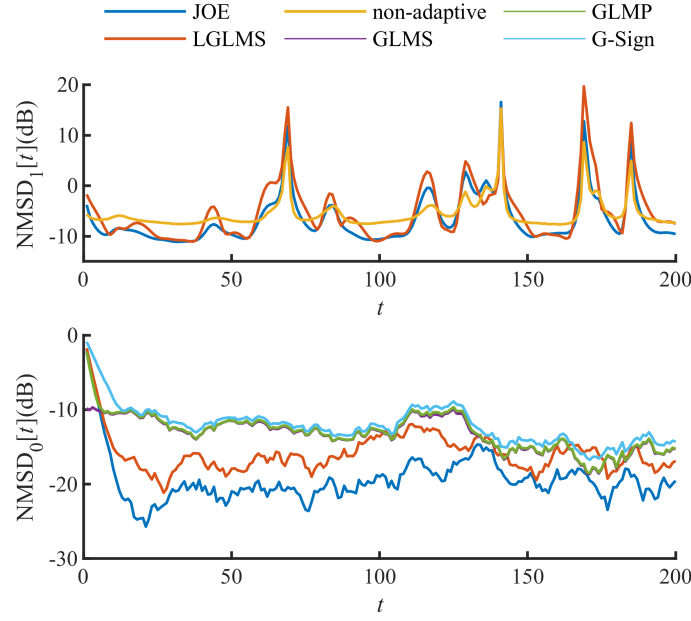


Figure 6.5 NMSE of the JOE compared against separate estimation in the Sioux Falls network (top: edge, bottom: node).

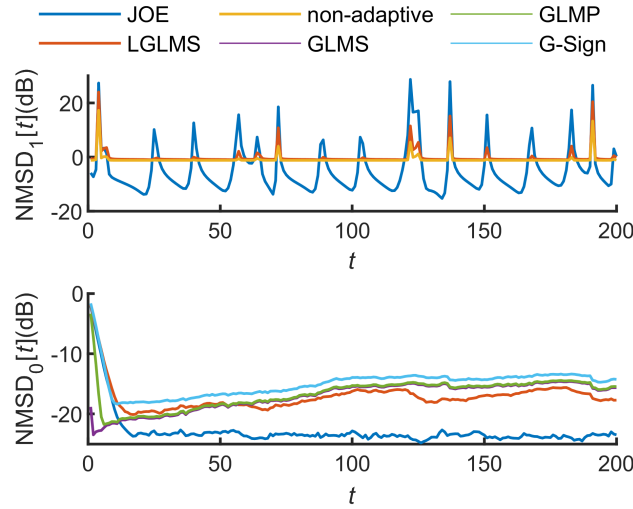


Figure 6.6 NMSE of the JOE compared against separate estimation in the Anaheim network (top: edge, bottom: node).

erations to predict edge signals based on a smooth graph assumption on the line graph projection does not perform well^[76]. As for the TSP non-adaptive method, the filtering operation $\mathbf{U}_1 \Sigma_{1,l,p} \mathbf{U}_1^T \mathbf{y}_1[t]$ is not optimized to reduce time-dependent errors compared to the adaptive update $\hat{\mathbf{H}}_1 \mathbf{e}_1[t]$ in (6.38), which lead to the less compelling results compared to JOE.

The JOE and Hodge-NLMS combination has the lowest estimation error again for the node signal estimation when inspecting Fig. 6.5 and Fig. 6.6. The GSP approaches do

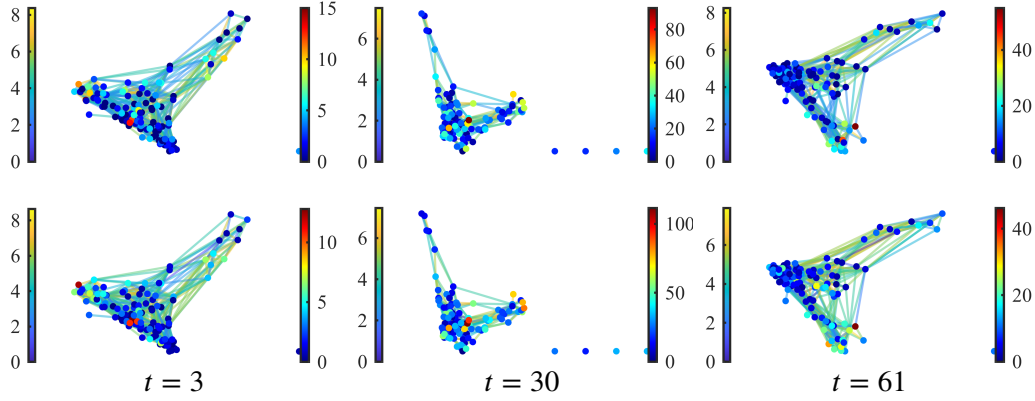


Figure 6.7 Dynamic graphs that record COVID-19 cases on the nodes and population mobility on the edges (self-loops are omitted). Upper row: ground truth. Lower row: predicted signals. Left color bar: edges (log scale). Right color bar: nodes.

not quite match the performance of JOE in accurately predicting the time-varying node signals because the GSP methods used a fixed graph Laplacian to process the signals on the nodes. On the other hand, JOE utilizes the edge signal to represent a time-varying diffusion to model the influence of the edge signals upon the node signals; then estimation is made in the form of a time-varying node regression. Our proposed JOE framework in Algorithm 6.2 has demonstrated the ability to effectively capture the time-varying edge signals jointly with the time-varying node signals when combined with the proposed Hodge-NLMS algorithm. When LGLMS takes a similar approach as JOE to use edge signals to represent the time-varying regression, the relatively inaccurate edge signal estimation causes the LGLMS to have inaccurate node signal estimation as well.

6.5.4 Covid-19 Case Prediction in Population Mobility Network

In this last experiment, we will use the JOE framework combined with Hodge-NLMS to jointly estimate real-world time-varying node and edge signals in the England COVID-19 dataset using the procedure in Algorithm 6.2. The goal of this real-world dataset is to predict the confirmed number of COVID-19 cases in the future using the currently given information. It has been shown that in the England COVID-19 dataset, the number of confirmed cases (node signals) is influenced by population mobility (edge signals), which matches the signal model in (6.5)^[149]. This means that our JOE framework is suitable to be applied to conduct online prediction of the next-day cases, as a diffusion modeled by a low-pass filter matches the diffusive nature of the data. Instead of the offline approaches proposed in the paper that introduced this dataset^[149], we will take an online approach to this problem and make one-day-ahead predictions of the number of cases.

For the England COVID-19 dataset, we will assume that the underlying topology is known when a new time instance arrives, but the simplicial signals on both the nodes and edges remain unknown and waiting to be predicted. During preprocessing, we combined the multi-edges between nodes into a single edge and calculated the absolute sum of the population mobility as the signal on the combined edge. The edge signals are scaled to 1/1000 of the original value in the preprocessing and are scaled back to the original scale after the prediction. We added Gaussian noise to both the ground truth node and edge signals. The dynamic nature of this dataset is modeled by a dynamic sampling matrix in the edges $\mathbf{D}_{k,S}[t]$ where we compare the graph topology of the graph at the current time point $\mathcal{G}[t]$ with newly obtained graph topology $\mathcal{G}[t + 1]$. If an edge is present in both $\mathcal{G}[t]$ and $\mathcal{G}[t + 1]$, then the corresponding diagonal element in $\mathbf{D}_{k,S}[t + 1]$ will be 1. If an edge is present in $\mathcal{G}[t]$ but not in $\mathcal{G}[t + 1]$, then the corresponding diagonal element in $\mathbf{D}_{k,S}[t + 1]$ will be 0. Both aggregations (6.31) and (6.32) are adopted in the experiment. Our proposed joint estimation strategy will be compared against three online GSP algorithms: GLMS^[35], GLMP^[59], and G-Sign^[9]. All these GSP algorithms are modified to be aware of the topology change, but they are not provided with the time-varying population mobility on the graph edges. The choice of this baseline is to demonstrate the usefulness of acquiring population mobility in predicting the number of cases. We will also be using another simple baseline from^[149] where the noisy cases of the previous day are used as the current estimation. This experiment reflects a real-world example of utilizing the time-varying signals on one simplicial dimension as an influence factor upon another simplicial dimension.

The predicted population mobility and the predicted number of cases at three different time points using Algorithm 6.2 are illustrated in Fig. 6.7. From Fig. 6.7, it is clear that the recovered population mobility and the number of cases are close to the ground truth. In Fig. 6.8, we calculated the average absolute error for one node at each time point across all the nodes. By inspecting Fig. 6.8, we observe that our joint estimation framework has the lowest error most of the time. The results in this experiment indicate that our proposed JOE framework could utilize the dynamic information on the 1-simplices and then make a prediction on the 0-simplices to solve a real-world problem of online prediction of time-varying signals. The current approach to this real-world problem is a bit primitive. In the next section, we will discuss some of the potential improvements to our Hodge-NLMS algorithm and our joint estimation framework to solve similar problems.

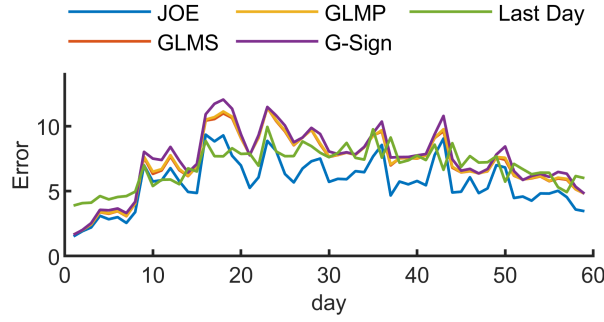


Figure 6.8 Average prediction error per region of daily COVID-19 cases in England .

6.6 Discussion and Future Work

This chapter presented the JOE framework to jointly estimate interacting time-varying simplicial signals across different simplicial dimensions using spectral and spatial TSP techniques. By treating graphs as a special case of a simplicial complex, time-varying simplicial signals co-existing on the nodes and edges of the graph can be processed by JOE through alternately executing the Hodge-NLMS for $k = 1$ and $k = 0$. The underlying online estimation algorithm of the JOE framework is the Hodge-NLMS algorithm, which is an adaptive algorithm backed by simplicial diffusion defined using TSP techniques. On the other hand, the Hodge-NLMS algorithm could operate as an online algorithm for estimating time-varying signals over a specific simplex. The JOE framework defines a time-varying regression data model where the signals in one simplex are under the influence of another higher-order simplicial signal. Experiments on both synthetic data and real data indicate that the JOE framework can efficiently solve the task of joint online estimation of time-varying simplicial signals.

There are many possible directions worth exploring for time-varying TSP and Hodge-Laplacian-based algorithms. Under our current setup, one limitation of our method is that the underlying topology is assumed to be known. One possible direction worth exploring is the possibility of combining our JOE framework with topology learning algorithms. This combination will further extend the predictive power of the Hodge-NLMS algorithm onto graphs that dynamically evolve the topology over time. Another limitation is that our current setup is not able to process multiple edges between two nodes. The extension of TSP approaches to multi-graphs will be another interesting direction. Another direction in which we can extend the JOE framework, instead of the Gaussian noise assumption, is to model the noise using non-Gaussian impulsive distributions such as the *alpha*-stable distribution, the Student's *t* distribution, or the Laplace distribution.

CHAPTER 7 TIME-VARYING GRAPH REPRESENTATION VIA DYNAMIC MULTI-HOP TOPOLOGIES

7.1 Introduction

Graph Signal Processing (GSP) and Graph Neural Network (GNN) methods have shown great success in multiple weather forecasting tasks, including the forecasting of nationwide temperature^[3], air quality^[113], and solar power^[150]. One early approach to online graph signal estimation was the proposal of the adaptive graph Least Mean Squares (GLMS) algorithm^[35]. The GLMS algorithm is based on an l_2 -norm optimization with the assumption of Gaussian noise, exploiting bandlimited spectral graph filters to conduct online estimations of graph signals on sampled observations. The Graph Diffusion LMS extends GLMS by replacing spectral graph filters with spatial diffusion^[30]. Approaches such as the Diffusion Sign-error LMS^[151] and Diffusion Sparse Sign with Variable Step-Size^[152] are adaptive sign algorithms but do not exploit the spectral or spatial graph information as seen in GSP. In GSP, the Adaptive Graph-Sign-Diffusion Algorithm was recently proposed by us, where a sign diffusion (or aggregation) was accomplished by defining a fixed binary update based on l_1 -norm optimization methods and graph (spatial) convolution on graphs^[153]. Please refer to Chapter 5, Section 5.7 for the details of the Graph-Sign-Diffusion Algorithm. This spatial diffusion on graphs is exactly graph aggregation.

By introducing nonlinear activation functions into GSP, architectures such as Graph Convolutional Neural Networks (GCN) and Graph Attention Networks (GAT) have extended the spatial GSP methods to machine learning tasks such as node classification, link prediction, and image classification^[18-19,77]. From the GSP perspective, these graph neural network (GNN) architectures can be generalized by node aggregations where the graph representations and operations are defined globally on the graph by a graph shift operator on the graph topology^[3]. There are several choices of graph shift operators on undirected graphs, for example, the (weighted and unweighted) adjacency matrix, the (weighted and unweighted) graph Laplacian matrix (with no normalization), or the normalized graph Laplacian matrix.

As discussed in Chapter 3 and Chapter 6, the GSP and GNN approaches mostly focused on the graph nodes and neglected higher-order structures. We can define more gen-

eralized signal representations on topologies that are beyond the graph nodes using TSP techniques. The solution to edge signal representation lies in Topological Signal Processing (TSP): spatial and spectral operations are generalized from graph nodes to simplicial complexes using operations defined by the Hodge Laplacians^[5-6]. Semi-supervised learning on the graph edges using the Hodge Laplacians and flow conservation is proposed in^[63]. Finite Impulse Response Filters on simplicial complexes were discussed in^[69,146], and convolutional filtering of signals on simplicial complexes was proposed in^[82]. An analogy of spectral wavelets on simplicial complexes called Hodgelets is proposed in^[67]. Neural network algorithms based on simplicial complexes and Hodge Laplacians can be found in^[75] and^[82].

Most of the previously discussed GSP and GNN algorithms on time-varying graph signals all assume the signal interaction represented by the underlying graph topology is fixed. However, assuming the topology is fixed as a universal approach may be an oversimplification when a more effective representation can be achieved using a dynamic topology^[154-155]. For example, in the pandemic prediction task, the underlying graph topology is a dynamic graph based on the daily population mobility among regions^[120]. Another example of representing time-varying interactions is a dynamic graph formed by thresholding a time-varying regression of gene activation that reflects one-to-one gene activation patterns over time^[64]. Thus, if the data interaction between nodes is dynamic, simply assigning static weights to these edges is insufficient to capture the dynamics and may lead to inferior performance in solving the targeted tasks^[155-156]. Additionally, the graph topology provided in datasets often goes through a graph sparsification process, which may significantly modify potentially crucial links that should have been formed between key nodes but end up missing^[156]. In other words, there may exist several hidden or latent edge connections that are not explicitly represented by the provided graph topology in datasets^[155]. Thus, what we truly need is an alternative perspective that allows for dynamic signals on the edges and potentially reveals the missing or hidden node interactions.

Instead of static edge weights, we propose the Dynamic Multi-Hop model to represent the change of interactions as time-varying signals on the graph edges under the influence of certain dynamics as a series of dynamic topological structures. The interactions (graph edges) are represented not only from the immediate 1-hop neighbors of each node but also for the nodes that are p -edges away with $p > 1$. Representation of time-varying signals

on both the graph nodes and the graph edges can be achieved by the utilization of TSP techniques. Additionally, we will introduce pruning techniques based on the Statistical Graphical Model to ensure that our formed topologies in Dynamic Multi-Hop have time-varying topologies and are sparse in the spatial domain, revealing the hidden dynamics within the data that were previously not represented by a fixed topology. Our aim for Dynamic Multi-Hop is that this dynamic topology representation will not only provide a more accurate representation of the interaction dynamics among node signals but also enable us to form a data model that will allow us to further analyze the interactions.

7.2 Dynamic Multi-Hop

7.2.1 1-Hop Node Signal Aggregation

Let us revisit the graph convolution shown in Chapter 3 because the algorithms mentioned above can be generalized using the graph convolution:

$$\mathbf{x}' = \mathbf{U}h(\mathbf{\Lambda})\mathbf{U}^T\mathbf{x} \approx \sum_{p=0}^P \theta_p \mathbf{L}^p \mathbf{x}, \quad (7.1)$$

where $\mathbf{U}$ is the eigenvector matrix of the graph Laplacian $\mathbf{L}$, $\mathbf{\Lambda}$ is the eigenvalue matrix of $\mathbf{L}$, and $h(\mathbf{\Lambda})$ is a graph filter. The expression after the approximation sign is obtained from the Chebyshev polynomial approximation^[31], and the derivations can be found in Chapter 3, Section 3.1. In the spatial domain, setting $p = 1$, equation (7.1) becomes

$$\mathbf{x}' = \mathbf{L}\mathbf{x} \in \mathbb{R}^N. \quad (7.2)$$

When (7.2) is viewed elementwise, it becomes

$$x'_i = \sum_{j=1}^{N_k} l_{ij} x_j, \quad (7.3)$$

where l_{ij} is the element on the i^{th} row and j^{th} column of the Graph Laplacian matrix $\mathbf{L}$. Looking at the definition of the graph Laplacian matrix, the matrix element l_{ij} is non-zero only when there is an adjacent node v_j to node v_i that is 1 edge away. So, if we repeat the element-wise operation shown in (7.3) for all the nodes, the summation essentially sums all the node signals (multiplied by -1) from all their adjacent nodes 1 edge away. As for the case of $i = j$, the summation will actually add the weighted signal from the i^{th} node itself. This means that (7.2) is an aggregation defined based on the adjacencies from each node that are in the 1-hop neighborhood. As we explained earlier, this process will be

repeated on all the nodes, so we can equivalently view (7.2) as all the node signals being diffused to their 1-hop neighbors.

7.2.2 P-Hop Node Signal Aggregation

Following the logic shown in the above paragraph, the p^{th} power of $\mathbf{L}$ in equation (5.18) aggregates the simplicial signal from the p -hop neighbors. Take the graph Laplacian as a specific example: repeating equation (7.2) p times, we will be aggregating the p -hop neighboring node signals. Doing this p -hop aggregation allows algorithms to aggregate signals that are p -neighbors away. Now, if we again break down the p^{th} power of the graph Laplacian matrix $\mathbf{L}$, we see that at positions where there was once a zero in $\mathbf{L}$, there is now a non-zero element in $\mathbf{L}^p$. When we have a convolution that consists of all the powers $1 \dots p$ and uses the adjacency matrix $\mathbf{A}$ instead of the Laplacian matrix, this formulation is known as the MixHop Graph Convolution^[157]:

$$\mathbf{x}' = \sum_{p=0}^P \theta_p \mathbf{A}^p \mathbf{x}. \quad (7.4)$$

7.2.3 Dense Topology Representation

Notice that we can establish a direct edge between the source node v_i and the target node v_j of a p -hops trajectory when the ij^{th} entry of $\mathbf{L}^p$ is non-zero, forming denser graph topologies as we increase the number of hops p . However, this dense graph formulation does not provide useful information if the edges are unweighted. Viewing this from a graph diffusion or aggregation perspective, because the edges between nodes are representations of the relationship and interaction of nodes, forming additional unweighted edges simply from expanding the computation graph by adding the number of hops is essentially adding irrelevant nodes into the aggregation. Viewing this from another perspective, simply adding unweighted edges by increasing the number of hops will result in a graph over-smoothing scenario where the resulting aggregation of all the nodes in a graph will have similar or even the same value^[158]. A visual comparison of the topology formed by a weighted graph Laplacian $\mathbf{L}$ with $\mathbf{L}^3$ and $\mathbf{L}^5$ is shown in Figure 7.1. This dataset is formed by a graph originally with 24 nodes and 38 edges, with a fixed topology shown as the original in Figure 7.1. The nodes and edge signals are formed synthetically following the approaches shown in Chapter 6.5.

In the next section, we will discuss how to alleviate the problems caused by fixed dense graph topology and provide the inference procedure for dynamic graph topology.

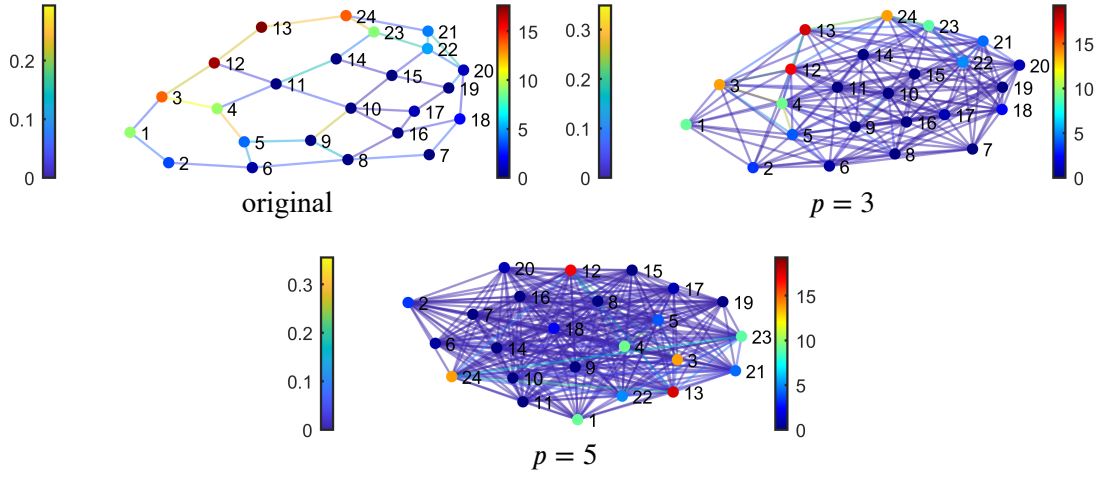


Figure 7.1 The original graph topology and the topologies formed by directly connecting p -hop neighbors. Left color bar: edges. Right color bar: nodes.

7.3 From Time-Varying Edge Signal To Dynamic Graphs

7.3.1 Time-Varying Edge Signal Representation

Now, in the case when the data interaction between nodes is dynamic, simply assigning fixed time-invariant weights to the edges is not feasible for capturing this data evolution pattern over time. A more suitable approach would be to consider the interactions between nodes as time-varying signals on the edges. In order to represent signals on the time-varying edges of the graph, we will be relying on techniques from TSP. We will begin edge signal representation on graphs with the simplicial convolution shown in Chapter 3:

$$\mathbf{x}'_k = \sum_{j=1}^{J_l} \gamma_{k,j} \mathbf{L}_{k,l}^j \mathbf{x}_k + \sum_{j=1}^{J_u} \theta_{k,j} \mathbf{L}_{k,u}^j \mathbf{x}_k + \xi_k \mathbf{I} \mathbf{x}_k, \quad (7.5)$$

where $\gamma_{k,j}$ are the weights of the lower Hodge Laplacian and $\theta_{k,j}$ are the weights for the upper Hodge Laplacian. In the spatial domain, similar to the GSP case, the above equation is the simplicial diffusion because the matrix multiplication

$$\mathbf{x}'_k = \mathbf{L}_k \mathbf{x}_k \in \mathbb{R}^N, \quad (7.6)$$

becomes

$$x'_{k,i} = \sum_{j=1}^{N_k} l_{k,ij} x_{k,i}, \quad (7.7)$$

when viewing it element-wise, where l_{ij} is the element on the i^{th} row and j^{th} column of the Hodge Laplacian matrix $\mathbf{L}$. Looking at the definition of the Hodge Laplacian matrix,

the matrix element l_{ij} is non-zero only when there is a $k + 1$ or $k - 1$ simplex that introduces the adjacency among k -simplices. So, if we repeat the element-wise operation shown in (7.7) for all the k -simplices, the summation essentially sums all the simplicial signals (multiplied by -1) from all the adjacent k -simplices. As for the case of $i = j$, the summation will add the weighted signal from the i^{th} k -simplex itself. This means that (7.6) is an aggregation defined based on the upper and lower adjacencies from the 1-hop neighborhood. Since this aggregation will be repeated on all the k -simplices, we can equivalently view (7.6) as all the signals being diffused to their 1-hop neighbors. Specifying (7.6) onto the graph edges, we can set $k = 1$. In scenarios where the triangle signals are unavailable, we can discard the upper Hodge Laplacian and only use the lower Hodge Laplacian formed by the node-to-edge incidence.

Now, to make (7.6) able to process time-varying signals, we can take a similar approach to that seen in the JOE framework shown in Chapter 6, Section 6.2:

$$\mathbf{x}_1[t + 1] = \mathbf{x}_1[t] + \Delta_1[t], \quad (7.8)$$

where $\Delta_1[t]$ is the amount of change that will lead the current edge signal $\mathbf{x}_1[t]$ to the next step edge signal $\mathbf{x}_1[t + 1]$. Assuming that we are using the Hodge-NLMS algorithm shown in Chapter 6, the edge signal estimation algorithm is

$$\hat{\mathbf{x}}_1[t + 1] = \hat{\mathbf{x}}_1[t] + \mu_1 \hat{\mathbf{H}}_1 \mathbf{e}_1[t] + r_1 \mathbf{R}_1 \hat{\mathbf{x}}_1[t], \quad (7.9)$$

where $\hat{\mathbf{H}}_1 = \sum_{p=0}^P \hat{\theta}_p \mathbf{L}_1^p$ as shown in (6.4) in Chapter 6, Section 6.2.1. For the choices of simplicial aggregation, please refer to Chapter 6.2.5. We should emphasize that the Hodge-NLMS in (7.9) is an online algorithm, meaning that at each time instance, we will have a different estimate of the time-varying edge signal $\mathbf{x}_1[t]$.

Now, in the absence of triangles, we can plug the online edge signal estimation obtained from (7.9) into (7.1) by replacing the static graph Laplacian $\mathbf{L}$ with a dynamic one:

$$\hat{\mathbf{L}}[t] = \hat{\mathbf{B}}_1^T \hat{\mathbf{B}}_1, \quad (7.10)$$

where $\hat{\mathbf{L}}_1[t]$ is a time-varying regression matrix in the form of a graph topology containing an edge signal. The term $\hat{\mathbf{B}}_1$ are defined similar to Chapter 6, Section 6.2.1

$$\hat{\mathbf{B}}_1 = (\mathbf{B}_1 \text{diag}(\text{sign}(\mathbf{x}_1[t]) \circ \sqrt{|\mathbf{x}_1[t]|})). \quad (7.11)$$

7.3.2 Topology Pruning

Visual inspection of Figure 7.1 shows that the graph topology gets denser as we increase the number of hops p . However, in GSP, a sparse topology is desired. Thus, our topologies have to be pruned in order to maintain sparsity.

Inspired by the Statistical Graphical Model (SGM)^[95], where topologies are formed by calculating the inverse of the covariance matrix among data and then taking a threshold to eliminate edges between weakly correlated nodes, we apply a similar threshold-based pruning approach in Dynamic Multi-Hop. Using the SGM, we can assume that the weights of the underlying graph topology are obtained from the inverse of the covariance matrix of a multivariate Gaussian distribution. Then, we can prune the edges in each of the topologies formed by $\hat{\mathbf{L}}^p[t]$ using a threshold that will keep only the edges that have stronger signals. Another approach we can take is to design a threshold in which a percentage of edges is eliminated. We denote the pruned version of $\hat{\mathbf{L}}^p[t]$ as $\bar{\mathbf{L}}^{(p)}[t]$, corresponding to the graph denoted as $\bar{\mathcal{G}}$.

Compared to the original graph topology $\mathcal{G}$, each $\bar{\mathcal{G}}[t]$ at different time instances will have only a few additional edges that represent the hidden node interactions that were previously absent in $\mathcal{G}$. The key to obtaining a sparse yet representative graph $\bar{\mathcal{G}}[t]$ is done by carefully choosing the threshold to prune $\hat{\mathbf{L}}^p[t]$, so each $\bar{\mathbf{L}}^{(p)}[t]$ should only have a few additional edges compared to $\mathcal{G}$ that is important. It should be emphasized that we can get a dynamic graph through pruning $\mathcal{G}$ only when the edge signals are time-varying. The reason behind this is that pruning the edges on a fixed graph at different time instances always results in cutting the same edges at different time instances. However, if we prune edges based on the evolving numerical value of the time-varying edge signals, then we are likely to cut different edges at different time instances. The simultaneous utilization of time-varyingness, SGM, graph pruning, and TSP is the unique characteristic of Dynamic Multi-Hop.

Here we will give a demonstration in Figure 7.2 of the combination of keeping the top-1 correlated edges along with a threshold of 0.05 at two different time instances for $p = 3$ and $p = 5$. It is clear from Figure 7.2 that the topologies for the pruned $\bar{\mathbf{L}}^{(3)}[0]$ and $\bar{\mathbf{L}}^{(6)}[0]$ are different from $\bar{\mathbf{L}}^{(3)}[30]$ and $\bar{\mathbf{L}}^{(6)}[30]$, which proves that our Dynamic Multi-Hop model is dynamic and evolves over time.

Looking back into the initial graph convolution formulation, the pruned topologies

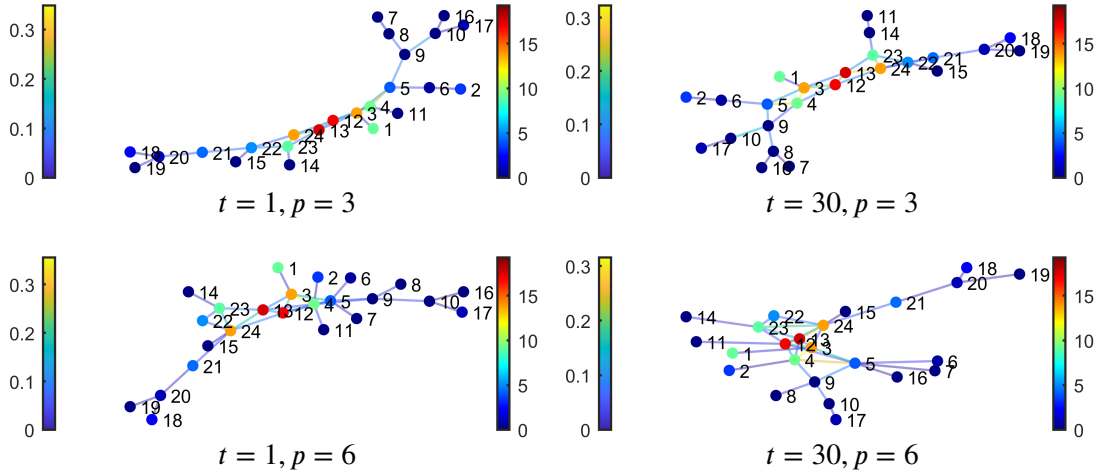


Figure 7.2 The topologies formed at two time instances $t = 0$ and $t = 30$ by directly connecting p -hop neighbors then pruning the edges. Left color bar: edges. Right color bar: nodes.

from different hops should be merged together to define convolution:

$$\mathbf{U}h(\Lambda[t])\mathbf{U}^T\mathbf{x} \approx \sum_{p=0}^P \theta_p \bar{\mathbf{L}}^{(p)}\mathbf{x}. \quad (7.12)$$

An illustration of the merged topology that resulted in a dynamic graph is shown in Figure 7.3. The merged topologies are indeed dynamic as well. Notice that for demonstration purposes, we set $\theta_p = 1$ for all the weights, but in reality, the weights are calculated by approximating the frequency filter using the Chebyshev polynomial approximation discussed in Chapter 3.

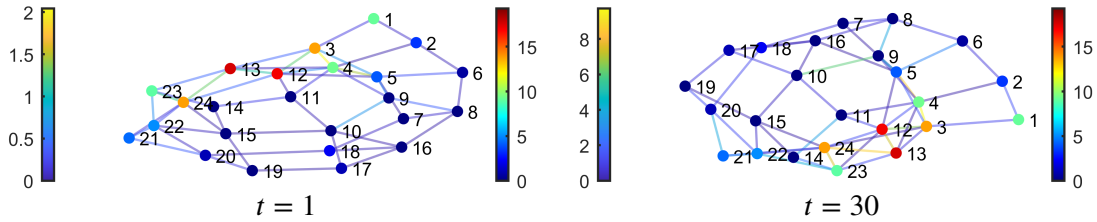


Figure 7.3 The topologies formed at two time instances $t = 0$ and $t = 30$ by directly merging all $\bar{\mathbf{L}}^{(p)}$ s for $p = 1 \dots 6$. Left color bar: edges. Right color bar: nodes.

7.4 Applying Dynamic Multi-Hop to Brain Networks

In this subsection, we will switch gears and delve into the application aspect of our Dynamic Multi-Hop in the field of functional brain connectivity analysis. In short, functional brain connectivity analysis interprets connections through brain regions in terms of the statistical dependencies between signals emitted from distinct regions of the brain.

Employing graph-based methods to assess brain connectivity opens up innovative avenues for examining how functional deficits in the brain correlate with structural disruptions implicated in various brain disorders^[159]. Nevertheless, it is important to acknowledge that assumptions of noiseless data and unchanging graph topologies hinder the effectiveness of current methodologies. Connectivity within the brain can be measured by transforming neighborhood relationships into a connectivity matrix, where each row and column represents different Regions of Interest (ROI) in the brain. A static and fixed graphical model can be obtained from brain fMRI by turning fMRI images into a graph of brain regions, aiming to identify the connections between specific brain areas^[160]. This process is similar to the statistical graphical model, in which correlations between pairs of regions (nodes) are determined, but it is done over the full length of the fMRI scanning period, losing the time dimension of the data. Using the Dynamic Multi-Hop model, we aim to reveal a series of dynamic graph topologies from the fMRI scans to represent the dynamic interactions among different brain regions.

7.4.1 Dataset Formulation

Our Dynamic Multi-Hop model will be applied to the normal and Autism subjects selected from the Autism Brain Imaging Data Exchange (ABIDE) dataset, in which the fMRI data were gathered from different internationally shared open databases, 1112 subjects in total^[161]. In the experiments, we selected the data that their division of ROI follows the Harvard-Oxford (HO) atlas, which gives us 111 different regions that will later correspond to $N_0 = 111$ nodes. Then, the signals within each ROI will naturally form a time-varying node signal $\mathbf{x}_0[t]$. We selected a normal subject and an Autism subject within the datasets, all with $T = 196$ time steps.

To generate a fixed topology, we use a combination of the threshold-based method similar to what is shown in^[160] and the Planar Maximally Filtered Graph^[162]. The correlation among 111 nodes across the 196 time steps is calculated throughout the 196 time steps and recorded in the correlation matrix $\mathbf{C}_F$. Using all the $N_0 \times N_0$ elements in $\mathbf{C}_F$, we can define a graph adjacency matrix $\mathbf{A}_F$, where the ij^{th} non-diagonal element is defined as $\mathbf{A}_{F,ij} = \text{abs}(\text{sign}(\mathbf{C}_{F,ij}))$ and the i^{th} diagonal element is defined as $\mathbf{A}_{F,ii} = 0$. This results in an unweighted and undirected fully connected graph $\mathcal{G}_F$ where each node has $N_0 - 1$ neighbors, which clearly does not satisfy the graph sparsity assumption^[3]. To formulate sparse edge connections, we only connect each node with its 1-hop neighbors that have the top-3 largest absolute correlation and also connect the nodes that have an absolute

correlation greater than a threshold of 0.95. The choice of using the top-3 most (absolute) correlated nodes is done by following the suggestion in^[162]. Viewing the absolute correlation from another aspect, we not only kept the connections between similar nodes that give us the data homophily but also preserved the data heterophily^[163]. The formulated sparse graph with fixed topology will be denoted as $\mathcal{G}$, with edge set $\mathcal{E} = \{e_1, \dots, e_{N_1}\}$. Then, to generate the time-varying signals on the edges within the edge set $\mathcal{E}$, we will use a sliding window of length $W = 10$ with overlap $T_p = 1$ to calculate the correlation matrices $\mathbf{C}[t]$ again. However, we only keep the entries that correspond to the edges within the edge set $\mathcal{E}$ to enforce sparsity. We take the absolute value of the correlation matrices $\mathbf{C}[t]$ to avoid negative edge weights, resulting in a time-varying edge signal $\mathbf{x}_1[t]|_{t=(W+1)\dots T}$.

In later subsections, we will be conducting experiments on the dataset formed above, which consists of the generated topology $\mathcal{G}$, the node signals $\mathbf{x}_0[t]$, and the edge signals $\mathbf{x}_1[t]$. The topology of $\mathcal{G}$ will be represented by the adjacency matrix $\mathbf{A}$. The corresponding graph Laplacian matrix will be denoted as $\mathbf{L}$. We should emphasize that in this case, the signals on the edges are fixed, but the signals on the nodes are time-varying. A demonstration of the graph $\mathcal{G}$ constructed on a normal subject and an Autism subject is shown in Figure 7.4. By looking at Figure 7.4, we can already see some distinctions between normal subjects and Autism subjects when using the same topology-forming techniques and the same parameters.

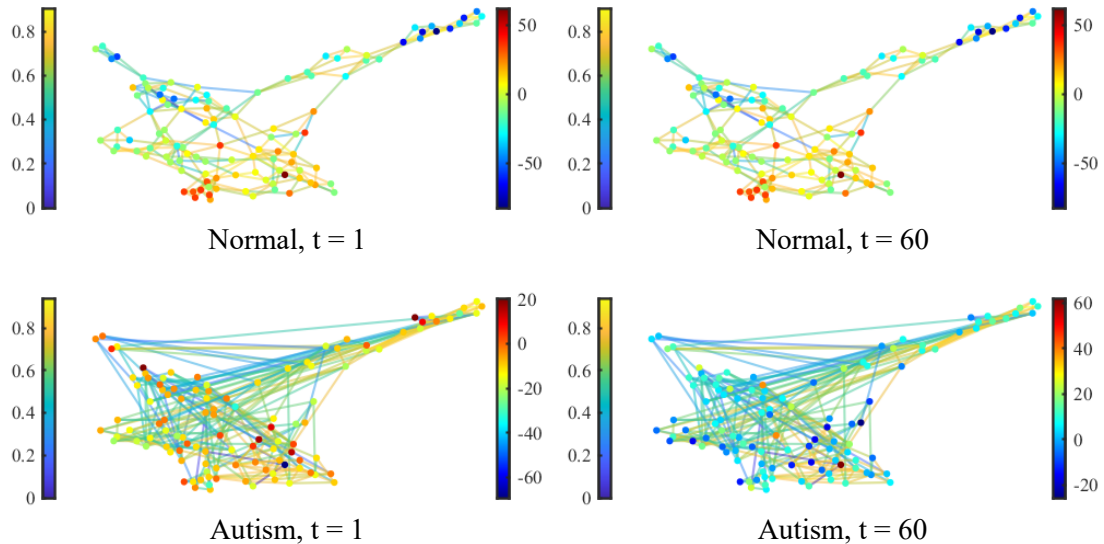


Figure 7.4 The formed graph topologies by taking the correlation of all node data and then pruning. The edge signals are fixed, but the node signals are time-varying.

7.4.2 Network Analysis Using Dynamic Multi-Hop

First, we will map the edge signals onto the topology of $\mathcal{G}$ by replacing the corresponding elements in the adjacency matrix $\mathbf{A}$ with the signal elements in $\mathbf{x}_1$. This, in a sense, will give us a series of time-varying weighted Laplacian matrices $\mathbf{A}[t]$. However, we should emphasize again that following the TSP interpretation, this can also be interpreted as having time-varying signals on the edge. Following the approaches shown in Section 7.2, we can use the summation $\sum_{p=0}^P \mathbf{A}^p$ to stack up all the p^{th} power of $\mathbf{A}[t]$ to form a denser time-varying graph that forms connections where all the nodes will be connected to new edges linking to their p -hop neighbors in the original graph $\mathcal{G}$. A visualization of the dense graphs at $t = 1$ and $t = 60$ for $p = 3$ and $p = 6$ are shown in Figure 7.5 (normal subject) and Figure 7.6 (Autism subject). In terms of brain network analysis, what we are doing is essentially trying to infer the hidden and time-varying interactions between brain regions that were not captured by the fixed graph topology. However, from Figure 7.5 and Figure 7.6, we can see that the number of edges with low correlation values is significantly greater than the number of edges with large correlation values. This dense structure is less desirable in both GSP and brain network analysis, as the dense structure may affect the performance of the downstream task.

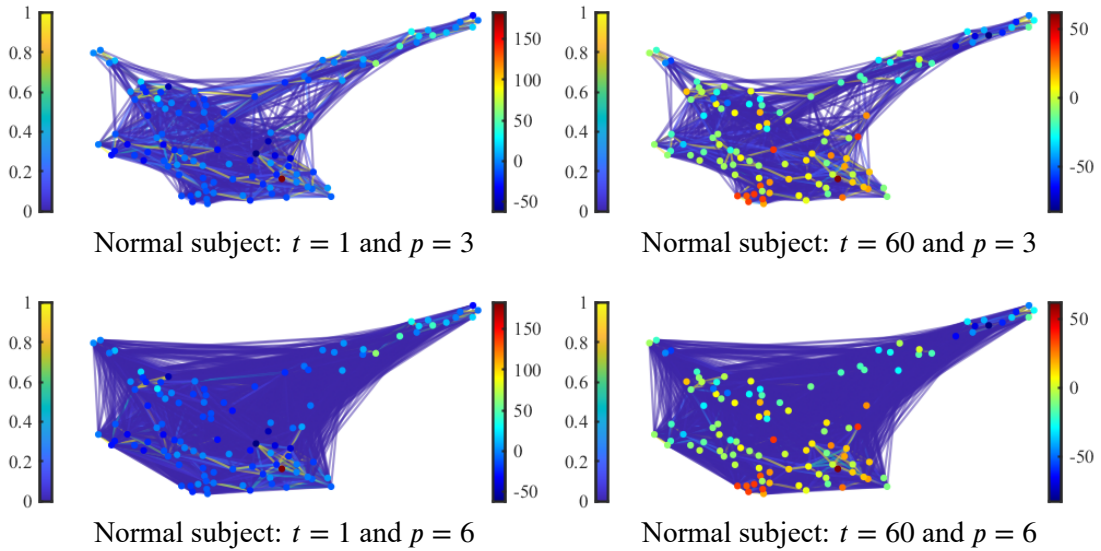


Figure 7.5 Topologies formed by directly connecting p -hop neighbors of a normal subject. Left color bar: edges. Right color bar: nodes.

The graph topology gets denser when we increase the hops p as expected. Notice that for the dense graphs in Figure 7.5 and Figure 7.6, the distinctions between normal and Autism subjects are not apparent anymore, which may lead to difficulties in processing

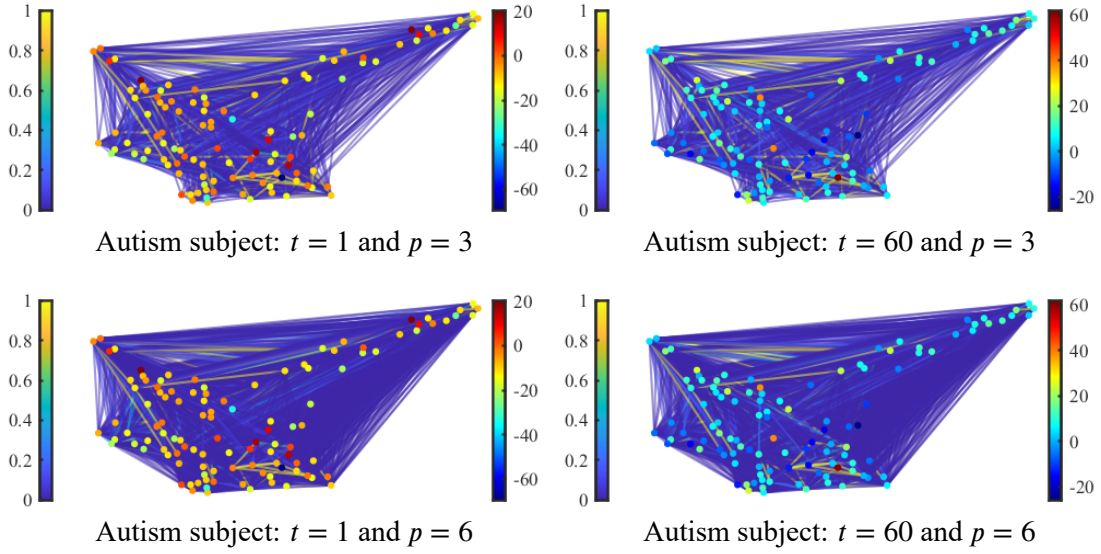


Figure 7.6 Topologies formed by directly connecting p -hop neighbors of an Autism subject. Left color bar: edges. Right color bar: nodes.

these signals. To mitigate the issue, we can follow our Dynamic Multi-hop approach again with the topology pruning procedures discussed in Section 7.3.2. The pruned topologies at $t = 1$ and $t = 60$ for $p = 3$ and $p = 6$ are shown in Figure 7.7 (normal subject) and Figure 7.8 (Autism subject). By inspecting Figure 7.7 and Figure 7.8, we see that the pruned topologies for both normal and Autism subjects become more sparse compared to the dense topologies in Figure 7.5 and Figure 7.6. Comparing their topologies from time $t = 1$ to $t = 60$, we can see that there is not only a change in the signals on the edges, but also dynamic edge connections formed by our Dynamic Multi-Hop approach. This is more apparent when comparing Figure 7.4 with Figure 7.7 and Figure 7.8, where different edges are formed on top of the given topology when $t = 1$ and $t = 60$. We also give a visual comparison to show the difference between the graph topology captured by Dynamic Multi-Hop and the original graph in Figure 7.9.

To further confirm that Dynamic Multi-Hop gives us a dynamic topology over time, we will be calculating the unweighted node degree over time. So, at each time instance, the unweighted node degree will give us the number of edges connected to that node. If this statistic changes at each time instance, then we can confirm that the number of edges that each node has is also changing. We plotted the average unweighted node degree over time in Figure 7.10. Visually analyzing Figure 7.10, we can confirm that for both subjects, the average unweighted node degree indeed changes over time, which confirms that Dynamic Multi-Hop generates dynamic graph topologies.

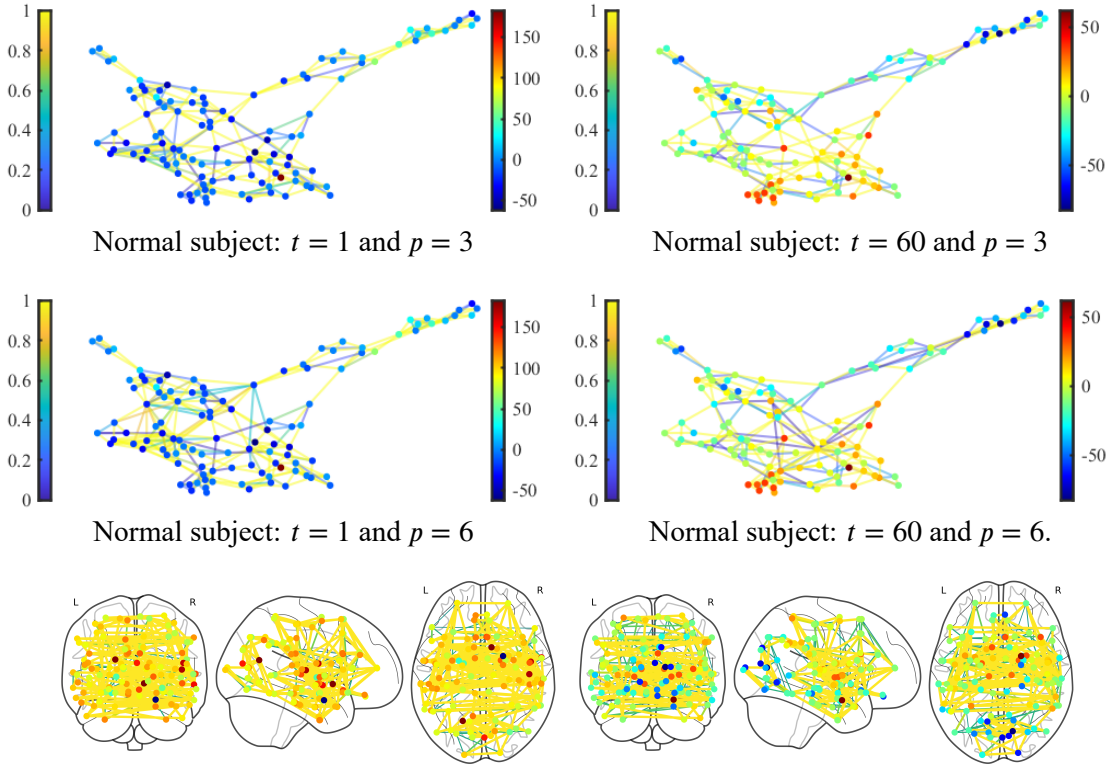
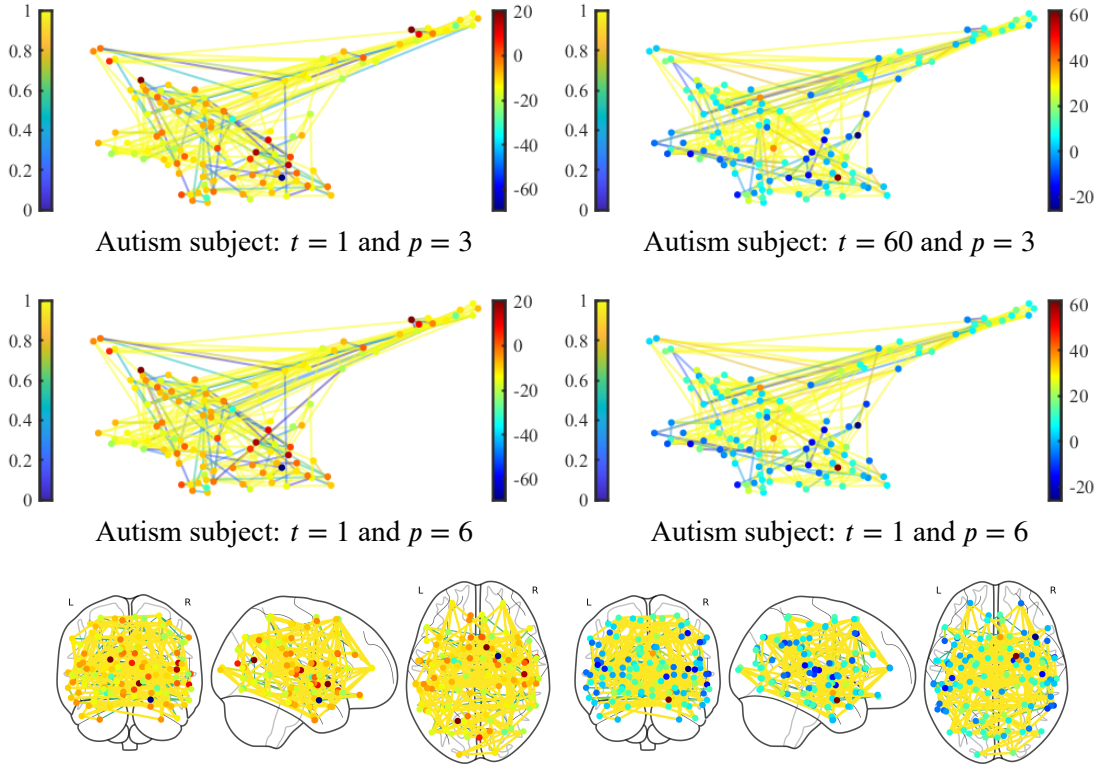


Figure 7.7 Topologies formed from a normal subject by Dynamic Multi-hop. Left color bar: edges. Right color bar: nodes.

7.5 Discussion and Limitations

In this chapter, we proposed the Dynamic Multi-Hop model, which represents the change of interactions as time-varying edge connections, outputting a dynamic series of topological structures. Applying the Dynamic Multi-Hop model to synthetic and brain networks, we are able to confirm that the Dynamic Multi-Hop model outputs a dynamic topology. These topologies can be used in other GSP algorithms for tasks like graph signal denoising and graph signal online estimation.

There is still room for improvement in the Dynamic Multi-Hop Model. The selection of the threshold in the pruning is still an ad hoc approach. We would like to derive more mathematical ways to find a threshold that is more meaningful. We would also like to conduct a search to find more real-world applications that are suitable for the Dynamic Multi-Hop model we proposed. Another aspect that we can improve is that, instead of the edge signal estimation done by the Hodge-NLMS algorithm, which is not derived for this particular model, we could develop an algorithm that is more suitable for Dynamic Multi-Hop. The last limitation is that the topologies are formed with the knowledge of all the



Mapping on brain images, $t = 1$ and $p = 6$. Mapping on brain images, $t = 60$ and $p = 6$.

Figure 7.8 Topologies formed from an Autism subject by Dynamic Multi-hop. Left color bar: edges. Right color bar: nodes.

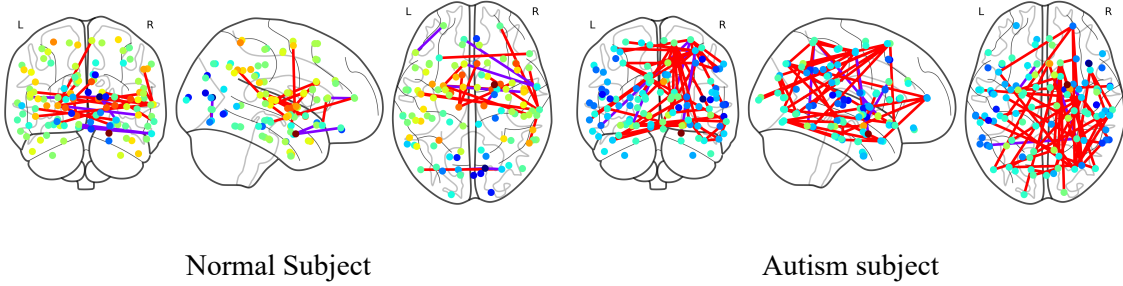


Figure 7.9 Comparison between the topology at $t = 52$ and $t = 60$ mapped on the brain image. The purple edges have appeared at $t = 52$ but not in $t = 60$ and the red edges edges have appeared at $t = 60$ but not in $t = 52$.

observations, but in reality, full observation of the data is sometimes not obtainable. Since the current setting assumes full observation, it is not suitable to directly apply Dynamic Multi-Hop to the current machine-learning paradigm. Taking the brain network as an example, assuming we have a node signal that contains missing observations, a more realistic approach is to use a learning algorithm first to infer the missing node signals from past known observations and then apply Dynamic Multi-Hop; this approach will be

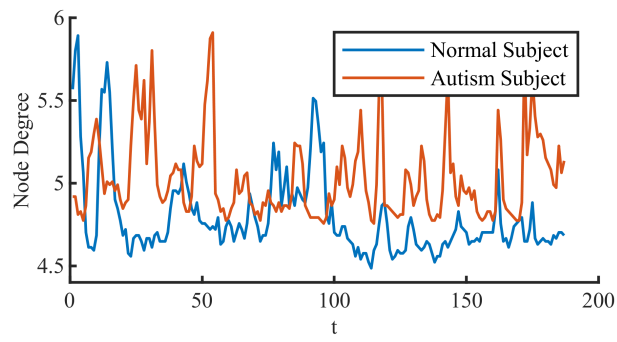


Figure 7.10 The average unweighted node degree over time.

worth our attention to investigate further.

CHAPTER 8 CONCLUSIONS AND FUTURE WORK

8.1 Conclusion

Let us revisit the four challenges that we would encounter and aim to surpass in the processing of multidimensional time-varying signals over graphs: the representation of topological irregularity, the estimation of unknown data based on known data, the removal of noise present in the data, and the extraction of the cross-space-time variation. We successfully integrated classical signal processing techniques with graph theory and machine learning concepts, proposing a suite of innovative approaches to address the aforementioned challenges. Moreover, the algorithms and methodologies proposed in this dissertation have not only resolved the four identified problems but also went beyond, delivering additional outcomes through our innovative integration of classical signal processing techniques, graph theory, and machine learning.

The first success among many of our research outputs is in the field of adaptive graph filters that led to the emergence of the Graph Normalized Least Mean p^{th} (GNLMP) algorithm and the Graph-Sign algorithm. Previous adaptive GSP algorithms have a Gaussian noise assumption; our algorithms aim to conduct an online estimation of partially observed time-varying graph node signals corrupted by impulsive noise. Built upon the combination of adaptive filters with Graph Signal Processing (GSP), the GNLMP algorithm and the Graph-Sign algorithm stand out for their stability and robustness under impulsive noise. The development of GNLMP and Graph-Sign contributed greatly to the advancements and innovations in the field of GSP. The GNLMP algorithm provided an algorithm focusing on the accuracy of the estimation, where a time-varying normalization is applied in the spectral domain to each adaptive update derived from the optimization results of a minimum dispersion loss. The Graph-Sign algorithm reduces the computational complexity and excels in terms of efficiency while maintaining high robustness under various noise distributions. By addressing the issues of robustness and computational complexity, the Graph-Sign algorithm and the GNLMP algorithm push the field of GSP forward, offering novel insights and methodologies that significantly enhance our problem-solving capabilities for more complicated GSP tasks.

Further pushing the limits and going beyond the graph nodes by incorporating Topological Signal Processing (TSP) tools, we have proposed the Hodge-NLMS algorithm,

which is an adaptive filter operating on simplicial complexes. The Hodge-NLMS algorithm is able to operate as an online algorithm for estimating time-varying signals over a specific simplex by itself, elevating the field of GSP by considering higher-order data representation than graph nodes. On the other hand, the interacting time-varying simplicial signals across different simplicial dimensions can be jointly estimated using spectral and spatial TSP techniques under our proposed Joint Online Estimation (JOE) framework. Stepping forward into the scenario of time-varying signals on graphs, we can use JOE to jointly process the time-varying signals co-existing on the graph nodes and the graph edges by specifying the simplicial complexes seen in JOE to be nodes and edges. The JOE framework from this research has fundamentally transformed the field of GSP since the JOE framework is the first framework to consider the joint estimation of interacting and time-varying node and edge signals.

Taking the graph topology for granted and using the fixed graph topology from a dataset may be an easy approach to complete the task at hand. However, there may be latent structures and dynamic relationships that are not captured by the already existing static graph topology. Our proposed Dynamic Multi-Hop model is a data-driven model that seeks to reveal and represent these changes and interactions in a time-varying manner, outputting a series of dynamic graph topologies that evolve over time. The additional information provided by the dynamic graphs could potentially serve as candidates for the implementation of graph-related applications and increase the performance of machine learning tasks on graphs.

The proposed methods have already shown success in multiple real-world signal processing and interdisciplinary applications. The GNLMP algorithm and the Graph-Sign algorithm have demonstrated great performance at forecasting meteorological data such as wind speed and hourly temperature. The JOE frameworks have proven successful at jointly estimating signals on the nodes and edges of transportation networks. JOE is also able to make predictions of the next-day COVID-19 cases in a population mobility graph by jointly considering the past number of cases and the population mobility. The Dynamic Multi-Hop is applied to brain network analysis: we represented the brain activities of normal and autistic subjects in the form of dynamic graphs. Other applications of the research proposed in this dissertation include citation estimation on a citation network and ocean drifter classification on an ocean-drifter map.

8.2 Future Work

The topic of interest discussed in this dissertation remains a promising avenue for further exploration. Most of the methodologies proposed in this dissertation are sign-invariant models, in which they take whatever undirected graph topology is input and use it as a computation graph by defining a graph or a simplicial shift operator and forming batch outputs on all the nodes and edges at the end. A more interesting model would be to individually consider the local structures of the graph, possibly incorporating the signs and shifts, and define different computations for different local structures. Our recent attempts at extending the Message Passing concept onto the Graph-Sign algorithm to define the computations locally at each node have proven successful^[164]. We would like to investigate further in this research direction in the future.

Even though we are able to formulate techniques on higher-order structures such as simplicial complexes, one limitation is that our current algorithms do not assume multiple edges connecting two nodes. We believe that these challenges can be solved by a collective solution of defining a time-varying and equivariant algorithm using message passing. Additionally, these higher-order models need not have a fixed structure. For example, in the Dynamic Multi-Hop model, we are only considering the nodes and edges. It would be interesting to explore the future of forming a higher-order dynamic structure, producing models that are more expressive than the current graph models at representing relationships that are beyond pairwise interactions.

Most of the spectral filters used in the GSP are predefined. However, a predefined filter based on assumptions may not reflect the real data behavior or may be sub-optimal in the downstream tasks. Thus, a filter learned by a machine learning or even deep learning algorithm could potentially give us filters that improve the performance of our algorithms. On the other hand, most Graph Neural Network (GNN) algorithms focus on time-invariant tasks. GNNs such as the Spatio-Temporal Graph Convolutional Neural Network assume the training data is clean of noise and has no missing observations. These limitations hinder the effectiveness of using GSP or GNNs in some real-world applications. Our initial effort in this direction is the LMS-GNN, which is our ongoing work that was recently proposed as a GNN that combines the advantages of adaptive graph filters and neural networks^[165]. At each time step, the resulting LMS-GNN architecture is trained using the residuals of the estimation, which allows the LMS-GNN to obtain a trained filter that will adaptively make predictions in the opposite direction of the error, similar to what is

seen in adaptive filters. The filters trained by the NN components are fed back into the adaptive graph filter components as a time-varying bandlimited filter. The adaptive GSP backbone of LMS-GNN allows it to update based on the estimation error at each time step, while the GNN components of the LMS-GNN learn and update the filter coefficients at each time step. Additionally, following this line of work, we have recently made the GNN analogues of the graph Least Mean p^{th} algorithm and Graph-Sign algorithm. We named them LMP-GNN and Sign-GNN, respectively^[166]. These two novel GNNs follow the same forward and backward propagation logic as LMS-GNN, but are defined using the minimum dispersion criterion and have shown robustness under impulsive noise scenarios.

Lastly, the proposed methodologies are built on assumptions such as graph smoothness (homophily), graph stationarity, and graph bandlimitedness. These assumptions may or may not hold true for a given task. Thus, developing graph algorithms without these assumptions, or even in opposition to these assumptions, is worth exploring. Building on top of the great application outcomes of the research completed in this dissertation, we seek to explore new applications, thereby enhancing the societal and scientific impact.

REFERENCES

- [1] Sandryhaila A, Moura J M. Big data analysis with signal processing on graphs: Representation and processing of massive data sets with irregular structure[J]. IEEE Signal Processing Magazine, 2014, 31(5): 80 - 90.
- [2] Shuman D, Narang S, Frossard P, et al. The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains[J]. IEEE Signal Processing Magazine, 2013, 30(3): 83-98.
- [3] Ortega A, Frossard P, Kovačević J, et al. Graph signal processing: Overview, challenges, and applications[J]. Proceedings of the IEEE, 2018, 106(5): 808-828.
- [4] Dong X, Thanou D, Toni L, et al. Graph signal processing for machine learning: A review and new perspectives[J]. IEEE Signal Processing Magazine, 2020, 37(6): 117-127.
- [5] Barbarossa S, Sardellitti S. Topological signal processing over simplicial complexes[J]. IEEE Transactions on Signal Processing, 2020, 68: 2992-3007.
- [6] Barbarossa S, Sardellitti S. Topological signal processing: Making sense of data building on multiway relations[J]. IEEE Signal Processing Magazine, 2020, 37(6): 174-183.
- [7] Bick C, Gross E, Harrington H A, et al. What are higher-order networks?[J]. SIAM Review, 2023, 65(3): 686-731.
- [8] Roddenberry T M, Schaub M T, Hajij M. Signal processing on cell complexes[C]//ICASSP. 2022: 8852-8856.
- [9] Yan Y, Kuruoglu E E, Altinkaya M A. Adaptive sign algorithm for graph signal processing[J]. Signal Processing, 2022, 200: 108662.
- [10] Yan Y, Adel R, Kuruoglu E E. Graph normalized-lmp algorithm for signal estimation under impulsive noise[J]. Journal of Signal Processing Systems, 2022.
- [11] Shuman D I, Narang S K, Frossard P, et al. The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains[J]. IEEE Signal Processing Magazine, 2013, 30(3): 83-98.
- [12] Huang W, Bolton T A W, Medaglia J D, et al. A graph signal processing perspective on functional brain imaging[J]. Proceedings of the IEEE, 2018, 106(5): 868-885.
- [13] Epskamp S, Waldorp L J, Möttus R, et al. The gaussian graphical model in cross-sectional and time-series data[J]. Multivariate Behavioral Research, 2018, 53(4): 453-480.
- [14] Yang J, Peng J. Estimating time-varying graphical models[J]. Journal of Computational and Graphical Statistics, 2020, 29(1): 191-202.
- [15] Stabler B, Bar-Gera H, Sall E. Transportation networks for research[M/OL]. GitHub, 2016. <https://github.com/bstabler/TransportationNetworks>.
- [16] Jabłoński I. Graph signal processing in applications to sensor networks, smart grids, and smart cities[J]. IEEE Sensors Journal, 2017, 17(23): 7659 - 7666.

REFERENCES

- [17] Li Z, Xiao Z, Lan C. Anomalous sensor detection based on nonlinear graph filter[C]//GlobalSIP. IEEE, 2019: 1-4.
- [18] Defferrard M, Bresson X, Vandergheynst P. Convolutional neural networks on graphs with fast localized spectral filtering[J]. Advances in neural information processing systems, 2016, 29.
- [19] Kipf T N, Welling M. Semi-supervised classification with graph convolutional networks[A]. 2016.
- [20] Xie T, Wang B, Kuo C C. GraphHop: An enhanced label propagation method for node classification[J]. IEEE Transactions on Neural Networks and Learning Systems, 2022: 1-15.
- [21] Bruna J, Zaremba W, Szlam A, et al. Spectral networks and locally connected networks on graphs[J]. ICLR, 2014.
- [22] Tremblay N, Borgnat P. Graph wavelets for multiscale community mining[J]. IEEE Transactions on Signal Process, 2014, 62(20): 5227-5239.
- [23] Isufi E, Loukas A, Simonetto A, et al. Autoregressive moving average graph filtering[J]. IEEE Transactions on Signal Processing, 2016, 65(2): 274-288.
- [24] Mei J, Moura J M F. Signal processing on graphs: Causal modeling of unstructured data[J]. IEEE Transactions on Signal Processing, 2017, 65(8): 2077-2092.
- [25] Grover A, Leskovec J. node2vec: Scalable feature learning for networks[C]//SIGKDD. 2016: 855-864.
- [26] Veličković P, Cucurull G, Casanova A, et al. Graph Attention Networks[J]. ICLR, 2018.
- [27] Godsil C, Royle G F. Algebraic graph theory: volume 207[M]. Springer Science & Business Media, 2001.
- [28] Hammond D K, Vandergheynst P, Gribonval R. Wavelets on graphs via spectral graph theory [J]. Applied and Computational Harmonic Analysis, 2011, 30(2): 129-150.
- [29] Shi X, Feng H, Zhai M, et al. Infinite impulse response graph filters in wireless sensor networks [J]. IEEE Signal Processing Letters, 2015, 22(8): 1113-1117.
- [30] Nassif R, Richard C, Chen J, et al. A graph diffusion lms strategy for adaptive graph signal processing[C]//Asilomar. 2017: 1973-1976.
- [31] Shuman D I, Vandergheynst P, Frossard P. Chebyshev polynomial approximation for distributed signal processing[C]//DCOSS. 2011: 1-8.
- [32] Lee S H, Ji F, Tay W P. Sgat: Simplicial graph attention network[C]//IJCAI, 2022: 3192-3200.
- [33] Kipf T N, Welling M. Semi-supervised classification with graph convolutional networks[J]. ICLR, 2017.
- [34] Diniz P. Adaptive filtering: Algorithms and practical implementation[M]. Springer, 2008.
- [35] D. Lorenzo P, Barbarossa S, Banelli P, et al. Adaptive least mean squares estimation of graph signals[J]. IEEE Transactions on Signal and Information Processing over Networks., 2016, 2 (4): 555 - 568.
- [36] Spelta M J M, Martins W A. Normalized lms algorithm and data-selective strategies for adaptive graph signal estimation[J]. Signal Processing, 2020, 167: 107326.

REFERENCES

- [37] Di Lorenzo P, Banelli P, Isufi E, et al. Adaptive graph signal processing: Algorithms and optimal sampling strategies[J]. IEEE Transactions on Signal Processing, 2018, 66(13): 3584-3598.
- [38] Hua F, Nassif R, Richard C, et al. Online distributed learning over graphs with multitask graph-filter models[J]. IEEE Transactions on Signal and Information Processing over Networks, 2020, 6: 63-77.
- [39] Elias V R M, Gogineni V C, Martins W A, et al. Adaptive graph filters in reproducing kernel hilbert spaces: Design and performance analysis[J]. IEEE Transactions on Signal and Information Processing over Networks, 2021, 7: 62-74.
- [40] Box G E, Jenkins G M, Reinsel G C, et al. Time series analysis: forecasting and control[M]. John Wiley & Sons, 2015.
- [41] Isufi E, Loukas A, Perraudin N, et al. Forecasting Time Series With VARMA Recursions on Graphs[J]. IEEE Transactions on Signal Processing, 2019, 67(18): 4870-4885.
- [42] Isufi E, Loukas A, Simonetto A, et al. Autoregressive moving average graph filtering[J]. IEEE Transactions on Signal Processing, 2017, 65(2): 274-288.
- [43] Engle R F. Autoregressive conditional heteroscedasticity with estimates of the variance of united kingdom inflation[J]. Econometrica: Journal of the Econometric Society, 1982: 987-1007.
- [44] Bollerslev T. Generalized autoregressive conditional heteroskedasticity[J]. Journal of Econometrics, 1986, 31(3): 307-327.
- [45] Nelson D B. Conditional heteroskedasticity in asset returns: A new approach[J]. Econometrica: Journal of the Econometric Society, 1991: 347-370.
- [46] Glosten L R, Jagannathan R, Runkle D E. On the relation between the expected value and the volatility of the nominal excess return on stocks[J]. The Journal of Finance, 1993, 48(5): 1779-1801.
- [47] Hong J, Yan Y, Kuruoglu E E, et al. Multivariate time series forecasting with GARCH models on graphs[J]. IEEE Transactions on Signal and Information Processing over Networks., 2023, 9: 557-568.
- [48] Kalman R E. A new approach to linear filtering and prediction problems[Z]. 1960.
- [49] Shi L. Kalman filtering over graphs: Theory and applications[J]. IEEE Transactions on Automatic Control, 2009, 54(9): 2230-2234.
- [50] Sagi G, Shlezinger N, Routtenberg T. Extended Kalman filter for graph signals in nonlinear dynamic systems[C]//ICASSP. 2023: 1-5.
- [51] Li W, Fu X, Zhang B, et al. Unscented Kalman filter of graph signals[J]. Automatica, 2023, 148: 110796.
- [52] Grassi F, Loukas A, Perraudin N, et al. A time-vertex signal processing framework: Scalable processing and meaningful representations for time-series on graphs[J]. IEEE Transactions on Signal Processing, 2018, 66(3): 817-829.
- [53] Chen Y, So H C, Kuruoglu E E. Variance analysis of unbiased least l_p -norm estimator in non-gaussian noise[J]. Signal Processing, 2016, 122: 190-203.
- [54] Brock F V. A nonlinear filter to remove impulse noise from meteorological data[J]. Journal of Atmospheric and Oceanic Technology, 1986, 3(1): 51 - 58.

REFERENCES

- [55] Karakuş O, Kuruoğlu E E, Altınkaya M A. Modelling impulsive noise in indoor powerline communication systems[J]. *Signal, image and video processing*, 2020, 14(8): 1655-1661.
- [56] Kuruoğlu E E, Rayner P J, Fitzgerald W J. Least lp-norm impulsive noise cancellation with polynomial filters[J]. *Signal Processing*, 1998, 69(1): 1-14.
- [57] Wang W, Sun Q. Robust adaptive estimation of graph signals based on Welsch loss[J]. *Symmetry*, 2022, 14(2): 426.
- [58] Shao M, Nikias C. Signal processing with fractional lower order moments[J]. *Proceedings of the IEEE*, 1993, 81(7): 986-1010.
- [59] Nguyen N, Doğançay K, Wang W. Adaptive estimation and sparse sampling for graph signals in alpha-stable noise[J]. *Digital Signal Processing*, 2020, 105: 102782.
- [60] Ribeiro M I. Kalman and extended Kalman filters: Concept, derivation and properties[J]. *Institute for Systems and Robotics*, 2004, 43(46): 3736-3741.
- [61] Alippi C, Zambon D. Graph kalman filters[A]. 2023.
- [62] Wan E A, Van Der Merwe R. The unscented Kalman filter[J]. *Kalman filtering and neural networks*, 2001: 221-280.
- [63] Jia J, Schaub M T, Segarra S, et al. Graph-based semi-supervised and active learning for edge flows[C]//KDD. *ACM*, 2019: 761–771.
- [64] Ancherbak S, Kuruoglu E E, Vingron M. Time-dependent gene network modelling by sequential monte carlo[J]. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 2016, 13(6): 1183-1193.
- [65] Han K, Eve G, Friesz T L. Computing dynamic user equilibria on large-scale networks with software implementation[J]. *Networks and Spatial Economics*, 2019, 19(3): 869-902.
- [66] Martino A, Giuliani A, Rizzi A. (hyper) graph embedding and classification via simplicial complexes[J]. *Algorithms*, 2019, 12(11): 223.
- [67] Roddenberry T M, Frantzen F, Schaub M T, et al. Hodgelets: Localized spectral representations of flows on simplicial complexes[C]//ICASSP. 2022: 5922-5926.
- [68] Battiloro C, Di Lorenzo P, Barbarossa S. Topological slepians: Maximally localized representations of signals over simplicial complexes[M]. *arXiv*, 2022.
- [69] Yang M, Isufi E, Schaub M T, et al. Finite impulse response filters for simplicial complexes [C]//EUSIPCO. 2021: 2005-2009.
- [70] Essl G. Topological iir filters over simplicial topologies via sheaves[J]. *IEEE Signal Processing Letters*, 2020, 27: 1215-1219.
- [71] Yang M, Isufi E, Schaub M T, et al. Simplicial convolutional filters[J]. *IEEE Transactions on Signal Processing*, 2022, 70: 4633-4648.
- [72] Kadambari S K, Francis R, Chepuri S P. Distributed denoising over simplicial complexes using chebyshev polynomial approximation[C]//EUSIPCO. 2022: 822-826.
- [73] Money R, Krishnan J, Beferull-L. B, et al. Online edge flow imputation on networks[J]. *IEEE Signal Processing Letters*, 2023, 30: 115-119.

REFERENCES

- [74] Krishnan J, Money R, Beferull-Lozano B, et al. Simplicial vector autoregressive model for streaming edge flows[C]//ICASSP. 2023: 1-5.
- [75] Roddenberry T M, Segarra S. Hodgenet: Graph neural networks for edge data[C]//ACSSC. 2019: 220-224.
- [76] Schaub M T, Segarra S. Flow smoothing and denoising: Graph signal processing in the edge-space[C]//GlobalSIP. 2018: 735-739.
- [77] Chamberlain B P, Shirobokov S, Rossi E, et al. Graph neural networks for link prediction with subgraph sketching[J]. ICLR, 2022.
- [78] Gilmer J, Schoenholz S S, Riley P F, et al. Neural message passing for quantum chemistry[C]//ICML. PMLR, 2017: 1263-1272.
- [79] Fey M, Lenssen J E. Fast graph representation learning with PyTorch geometric[J]. ICLR Workshop, 2019.
- [80] Yu B, Yin H, Zhu Z. Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting[C]//IJCAI. 2018.
- [81] Lewenfus G, Martins W A, Chatzinotas S, et al. Joint forecasting and interpolation of time-varying graph signals using deep learning[J]. IEEE Transactions on Signal and Information Processing over Networks., 2020, 6: 761-773.
- [82] Yang M, Isufi E, Leus G. Simplicial convolutional neural networks[C]//ICASSP. 2022: 8847-8851.
- [83] Giusti L, Battiloro C, Di Lorenzo P, et al. Simplicial attention neural networks[M]. arXiv, 2022.
- [84] Ebli S, Defferrard M, Spreemann G. Simplicial neural networks[J]. NeurIPS Workshop TDA and Beyond, 2020.
- [85] Yang M, Isufi E, Leus G. Simplicial convolutional neural networks[C]//ICASSP. 2022: 8847-8851.
- [86] Goh C W J, Bodnar C, Lio P. Simplicial attention networks[J]. ICLR Workshop on Geometrical and Topological Representation Learning, 2022.
- [87] Giusti L, Battiloro C, Di Lorenzo P, et al. Simplicial attention neural networks[M]. arXiv, 2022.
- [88] Wu F, Souza A, Zhang T, et al. Simplifying graph convolutional networks[C]//Proceedings of Machine Learning Research: volume 97 ICML. 2019: 6861-6871.
- [89] Wang J, Wang Y, Yang Z, et al. Bi-gcn: Binary graph convolutional network[J]. CVPR, 2021: 1561-1570.
- [90] Li Q, Han Z, Wu X M. Deeper insights into graph convolutional networks for semi-supervised learning[C]//AAAI: volume 32. 2018.
- [91] Liu M, Gao H, Ji S. Towards deeper graph neural networks[C]//KDD. 2020: 338-348.
- [92] Lauritzen S L. Graphical models: volume 17[M]. Clarendon Press, 1996.
- [93] Yuan M, Lin Y. Model selection and estimation in the gaussian graphical model[J]. Biometrika, 2007, 94(1): 19-35.
- [94] Altenbuchinger M, Weihs A, Quackenbush J, et al. Gaussian and mixed graphical models as (multi-)omics data analysis tools[J]. Biochimica et Biophysica Acta - Gene Regulatory Mechanisms, 2020, 1863(6): 194418.

REFERENCES

- [95] Zhang C, Florêncio D, Chou P A. Graph signal processing-a probabilistic framework[J]. Microsoft Res., Redmond, WA, USA, Tech. Rep. MSR-TR-2015-31, 2015.
- [96] Dong X, Thanou D, Frossard P, et al. Learning laplacian matrix in smooth graph signal representations[J]. IEEE Transactions on Signal Processing, 2016, 64(23): 6160-6173.
- [97] Margetic H P, Thanou D, Frossard P. Graph learning under sparsity priors[C]//ICASSP. 2017: 6523-6527.
- [98] Belilovsky E, Kastner K, Varoquaux G, et al. Learning to discover sparse graphical models[C]//ICML. PMLR, 2017: 440-448.
- [99] Giannakis G B, Shen Y, Karanikolas G V. Topology identification and learning over graphs: Accounting for nonlinearities and dynamics[J]. Proceedings of the IEEE, 2018, 106(5): 787-807.
- [100] Saboksayr S S, Mateos G, Cetin M. Online discriminative graph learning from multi-class smooth signals[J]. Signal Processing, 2021, 186: 108101.
- [101] Liu Y, Yang L, You K, et al. Graph learning based on spatiotemporal smoothness for time-varying graph signal[J]. IEEE Access, 2019, 7: 62372-62386.
- [102] Liu Y, Yang L, Wenbin G, et al. Spatiotemporal smoothness-based graph learning method for sensor networks[C]//WCNC. IEEE, 2019: 1-6.
- [103] Yamada K, Tanaka Y, Ortega A. Time-varying graph learning based on sparseness of temporal variation[C]//ICASSP. 2019: 5411-5415.
- [104] Coutino M, Isufi E, Maehara T, et al. State-space network topology identification from partial observations[J]. IEEE Transactions on Signal and Information Processing over Networks, 2020, 6: 211-225.
- [105] Friedman J, Hastie T, Tibshirani R. Sparse inverse covariance estimation with the graphical lasso[J]. Biostatistics, 2008, 9(3): 432-441.
- [106] Giraldo J H, Mahmood A, Garcia-Garcia B, et al. Reconstruction of time-varying graph signals via sobolev smoothness[J]. IEEE Transactions on Signal and Information Processing over Networks, 2022, 8: 201-214.
- [107] Natali A, Isufi E, Coutino M, et al. Online graph learning from time-varying structural equation models[C]//Asilomar. IEEE, 2021: 1579-1585.
- [108] Natali A, Isufi E, Coutino M, et al. Learning time-varying graphs from online data[J]. IEEE Open Journal of Signal Processing, 2022, 3: 212-228.
- [109] Zaman B, Ramos L M L, Romero D, et al. Online topology identification from vector autoregressive time series[J]. IEEE Transactions on Signal Processing, 2020, 69: 210-225.
- [110] Sardellitti S, Barbarossa S, Di Lorenzo P. Online learning of time-varying signals and graphs [C]//ICASSP. 2021: 5230-5234.
- [111] Natali A, Coutino M, Isufi E, et al. Online time-varying topology identification via prediction-correction algorithms[C]//ICASSP. 2021: 5400-5404.
- [112] Shuman D I, Narang S K, Frossard P, et al. The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains[J]. IEEE Signal Processing Magazine, 2013, 30(3): 83-98.

REFERENCES

- [113] Wang C, Zhu Y, Zang T, et al. Modeling inter-station relationships with attentive temporal graph convolutional network for air quality prediction[C]//WSDM. New York, NY, USA, 2021: 616–634.
- [114] Jiang B, Huang Y, Panahi A, et al. Dynamic graph learning: A structure-driven approach[J]. *Mathematics*, 2021, 9(2): 168.
- [115] Hu W, Pang J, Liu X, et al. Graph signal processing for geometric data and beyond: Theory and applications[J]. *IEEE Transactions on Multimedia*, 2021(Early Access).
- [116] Li M M, Huang K, Zitnik M. Graph representation learning in biomedicine and healthcare[J]. *Nature Biomedical Engineering*, 2022: 1-17.
- [117] Yu B, Yin H, Zhu Z. Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting[J]. *IJCAI*, 2018.
- [118] Schaub M T, Benson A R, Horn P, et al. Random walks on simplicial complexes and the normalized hodge 1-laplacian[J]. *SIAM Review*, 2020, 62(2): 353-391.
- [119] McCallum A K, Nigam K, Rennie J, et al. Automating the construction of internet portals with machine learning[J]. *Information Retrieval* 3, 2000: 127–163.
- [120] Panagopoulos G, Nikolentzos G, Vazirgiannis M. Transfer graph neural networks for pandemic forecasting[J]. *AAAI*, 2021, 35(6): 4838-4845.
- [121] Colonnese S, Petti M, Farina L, et al. Protein-protein interaction prediction via graph signal processing[J]. *IEEE Access*, 2021, 9: 142681-142692.
- [122] Stanković L, Sejdić E. Vertex-frequency analysis of graph signals[M]. Springer, 2019.
- [123] Kutlay M, Gagula-Palalic S. Application of machine learning in healthcare: Analysis on MHEALTH dataset[J]. *Southeast Europe Journal of Soft Computing*, 2016, 4: 46-51.
- [124] Micheloyannis S, Pachou E, Stam C J, et al. Using graph theoretical analysis of multi channel EEG to evaluate the neural efficiency hypothesis[J]. *Neuroscience Letters*, 2006, 402(3): 273-277.
- [125] Arguez et. al. [J]. NOAA’s U.S. Climate Normals (1981-2010), 2010.
- [126] Lim L H. Hodge laplacians on graphs[J]. *SIAM Review*, 2020, 62(3): 685-715.
- [127] Bodnar C, Frasca F, Wang Y, et al. Weisfeiler and Lehman go topological: Message passing simplicial networks[C]//ICML. PMLR, 2021: 1026-1037.
- [128] Yan Y, Kuruoglu E E. Binarized simplicial convolutional neural networks[C]//2023.
- [129] Ammar W, Groeneveld D, Bhagavatula C, et al. Construction of the literature graph in semantic scholar[J]. *Proceedings of the North American Chapter of the Association for Computational Linguistics*, 2018.
- [130] Courbariaux M, Hubara I, Soudry D, et al. Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1[J]. *Algorithms*, 2016, 12(11).
- [131] Diniz P. Adaptive filtering: Algorithms and practical implementation[M]. Springer, 2008.
- [132] Di Lorenzo P, Banelli P, Barbarossa S, et al. Distributed adaptive learning of graph signals[J]. *IEEE Transactions on Signal Processing*, 2017, 65(16): 4193-4208.

REFERENCES

- [133] Banerjee S, Agrawal M. Underwater acoustic communication in the presence of heavy-tailed impulsive noise with bi-parameter cauchy-gaussian mixture model[C]//SYMPOL. 2013: 1-7.
- [134] Li Q, Ben Y, Naqvi S M, et al. Robust student's t based cooperative navigation for autonomous underwater vehicles[J]. IEEE Transactions on Instrumentation and Measurement, 2018, 67(8): 1762-1777.
- [135] Lee I, Liu X, Zhou C, et al. Noise-enhanced detection of subthreshold signals with carbon nanotubes[J]. IEEE Transactions on Nanotechnology, 2006, 5(6): 613-627.
- [136] Wang N, Yeung D Y. Bayesian robust matrix factorization for image and video processing[C]//ICCV. 2013: 1785-1792.
- [137] Wang L. The L1 penalized lad estimator for high dimensional linear regression[J]. Journal of Multivariate Analysis, 2013, 120: 135-151.
- [138] Di Lorenzo P, Banelli P, Barbarossa S. Optimal sampling strategies for adaptive learning of graph signals[C]//EUSIPCO. 2017: 1684-1688.
- [139] Shomorony I, Avestimehr S. Sampling large data on graphs[J]. GlobalSIP, 2014.
- [140] Bhatia V, Mulgrew B, Georgiadis A T. Stochastic gradient algorithms for equalisation in α -stable noise[J]. Signal Processing, 2006, 86(4): 835-845.
- [141] Swami A, Sadler B M. On some detection and estimation problems in heavy-tailed noise[J]. Signal Processing, 2002, 82(12): 1829-1846.
- [142] Sahmoudi M, Abed-Meraim K, Benidir M. Blind separation of heavy-tailed signals using normalized statistics[C]//Independent Component Analysis and Blind Signal Separation. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004.
- [143] Sayed A H, Nascimento V H. Energy conservation and the learning ability of LMS adaptive filters[J]. Least-Mean-Square Adaptive Filters, Wiley, 2003.
- [144] Tremblay N, Gonçalves P, Borgnat P. Chapter 11 - design of graph filters and filterbanks[M]// Djurić P M, Richard C. Cooperative and Graph Signal Processing. Academic Press, 2018: 299-324.
- [145] Yan Y, Xie T, Kuruoglu E E. Joint online estimation of time-varying simplicial signals on graphs [J]. submitted to IEEE Transactions on Signal and Information Processing Over Networks, 2023.
- [146] Sardellitti S, Barbarossa S. Robust signal processing over simplicial complexes[C]//ICASSP. 2022: 8857-8861.
- [147] Yang Q, Lyu Y, Li X, et al. Adaptive distributed synchronization of heterogeneous multi-agent systems over directed graphs with time-varying edge weights[J]. Journal of the Franklin Institute, 2021, 358(4): 2434-2452.
- [148] Bick C, Gross E, Harrington H A, et al. What are higher-order networks?[J]. SIAM Review, 2023, 65(3): 686-731.
- [149] Panagopoulos G, Nikolentzos G, Vazirgiannis M. Transfer graph neural networks for pandemic forecasting[J]. AAAI, 2021, 35(6): 4838-4845.
- [150] Simeunovic J, Schubnel B, Alet P J, et al. Spatio-temporal graph neural networks for multi-site pv power forecasting[J]. IEEE Transactions on Sustainable Energy, 2022, 13(2): 1210-1220.

REFERENCES

- [151] Ni J, Chen J, Chen X. Diffusion sign-error lms algorithm[J]. Signal Processing, 2016, 128: 142-149.
- [152] Chen F, Liu X, Duan S, et al. Diffusion sparse sign algorithm with variable step-size[J]. Circuits, Systems, and Signal Processing, 2019, 38(4): 1736-1750.
- [153] Yan Y, Kuruoglu E E. Fast and robust wind speed prediction under impulsive noise via adaptive graph-sign diffusion[C]//IEEE CAI. 2023: 302-305.
- [154] Kazi A, Cosmo L, Ahmadi S A, et al. Differentiable graph module (dgm) for graph convolutional networks[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2022, 45(2): 1606-1617.
- [155] de Ocariz Borde H S, Kazi A, Barbero F, et al. Latent graph inference using product manifolds [C]//ICLR. 2022.
- [156] Lu J, Xu Y, Wang H, et al. Latent graph inference with limited supervision[J]. Advances in Neural Information Processing Systems, 2024, 36.
- [157] Abu-El-Haija S, Perozzi B, Kapoor A, et al. Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing[C]//ICML. PMLR, 2019: 21-29.
- [158] Chen D, Lin Y, Li W, et al. Measuring and relieving the over-smoothing problem for graph neural networks from the topological view[J]. Proceedings of the AAAI, 2020, 34: 3438-3445.
- [159] Zhao F, Kuruoglu E E. Sequential monte carlo graph convolutional network for dynamic brain connectivity[Z]. 2023.
- [160] Li X, Zhou Y, Dvornek N, et al. Braingnn: Interpretable brain graph neural network for fMRI analysis[J]. Medical Image Analysis, 2021, 74: 102233.
- [161] Craddock C, Benhajali Y, Chu C, et al. The neuro bureau preprocessing initiative: open sharing of preprocessed neuroimaging data and derivatives[J]. Frontiers in Neuroinformatics, 2013, 7 (27): 5.
- [162] Tumminello M, Aste T, Di Matteo T, et al. A tool for filtering information in complex systems [J]. Proceedings of the National Academy of Sciences, 2005, 102(30): 10421-10426.
- [163] Zhu J, Rossi R A, Rao A, et al. Graph neural networks with heterophily[C]//Proceedings of the AAAI: volume 35. 2021: 11168-11176.
- [164] Peng C, Yan Y, KURUOGLU E. Adaptive message passing sign algorithm[C]//Temporal Graph Learning Workshop @ NeurIPS 2023. 2023.
- [165] Yan Y, Peng C, Kuruoglu E E. Adaptive least mean squares graph neural networks and online graph signal estimation[C]//under review for IEEE Signal Processing Letters. 2023.
- [166] Peng C, Yan Y, KURUOGLU E. Adaptive least mean p^{th} power graph neural networks”, under review for *Neurocomputing*[C]//under review for Neurocomputing. 2024.

ACKNOWLEDGEMENTS

First and foremost, I would like to express my sincere gratitude to my supervisor, Professor Ercan Engin Kuruoglu, for the support, guidance, and valuable insights throughout the four years of my Ph.D. life. The facilities and resources provided in the Time-Varying Data Science Lab are fundamental to the success of my work.

I am grateful to Tsinghua University for providing me with an opportunity to study and research at one of the most famous and influential universities worldwide. Being a student at Tsinghua University is exactly my childhood fantasy, and this fantasy has been fulfilled. I greatly appreciate the support from our department, TBSI, for giving me the opportunity to study at the University of California, Berkeley. This study experience at the University of California, Berkeley has significantly enriched my academic and personal growth. Thank you to all the professors I have met during my academic journey. Your invaluable guidance, inspiration, and support enrich my academic experience. An earnest thank you to all my labmates and classmates, for their collaboration, insightful discussions, and the moral support that kept me motivated.

Last but not least, I cannot express enough thanks to my family for their love, understanding, and endless support. The belief, trust, and encouragement I received from my family are the driving forces for me to pursue my goal of completing my Ph.D. study. To my parents, Shishen Yan and Xiao Wang, and my wife, Fangyuan Lyu, thank you for being my cornerstone and my source of strength.